

SRINIVAS



UNIVERSITY

**TeleDrive — Edge-Based Riding Behaviour Analysis Using
Smartphone Inertial Sensors and On-Device Machine Learning**

Submitted to Srinivas University on partial Completion of Sixth
Semester

Bachelor of Computer Application By

**Sharan S
(03SU23DS019)**

VI Semester BCA

Under the Guidance of

Asst.Prof. Sachin M

Faculty of

Institute of Computer Science and Information Sciences

Srinivas University

Pandeshwar , Mangalore

2023-2027



SRINIVAS UNIVERSITY

Srinivas Nagar, Mukka- 574 146, Surathikal, Mangalore, Phone: 0824-2477456
(Private University Established by Karnataka Govt. ACT No.42 of 2013, Recognized by UGC,
New Delhi & Member of Association of Indian Universities, New Delhi)
Web: www.srinivasuniversity.ac.in, Email: info@srinivasuniversity.edu.in
Administrative Office: GHS Road, Mangalore-01, Phone 0824-2425966



INSTITUTE OF COMPUTER SCIENCE & INFORMATION SCIENCES

CERTIFICATE

This is to certify that the project work entitled
“.....”

.....” carried out by

Mr./Ms.....

(Registration No.....), a bonafide student of this institution, in partial fulfillment for the award of Degree **B.C.A/ M.C.A** at Srinivas University, New Campus, Pandeshwara, Mangaluru – 575 001, during the academic year 2023 -2027. It is also certified that all corrections/suggestions indicated during internal assessment have been incorporated in the Report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Name & Signature of the Guide

Name & Signature of
the Dean

External Viva:

Name of the Examiners

Signature with Date

1.

2

DECLARATION

We hereby declare that the project work detail which is being presented in this report is in fulfilment of the requirement for the award of degree of Bachelor of Computer Application.

We hereby declare that we have undertaken project work on “TELEDRIIVE” under the guidance of **Asst.Prof. Sachin M**, Professor, Department of Computer Science and Information Science, Srinivas University, Mangalore.

We hereby declare that this project work report is of our own work and the best of our knowledge and belief the matter embedded in report has not been submitted by us for the award of other degree to this or any other university.

Place: Mangalore

**Sharan S
(03SU23DS019)**

Date: 20-04-2026

ACKNOWLEDGEMENT

Any success—whether large or small—is rarely achieved alone. Behind every accomplishment lies the support, encouragement, and guidance of many individuals. The fulfilment and pride that come with completing this project would not be complete without acknowledging those who contributed to its realization.

We extend our sincere appreciation to **Dr. Subramanya Bhat**, Dean of the Institute, for ensuring we had access to the necessary resources and for nurturing a stimulating and goal-oriented academic atmosphere.

Our heartfelt thanks also go to **Dr. P. Sridhara Acharya**, Professor and Head of the Department, Institute of Computer Science and Information Science, Srinivas University, Mangalore, for his insightful guidance, constant motivation, and unwavering support throughout my project work.

We would also like to acknowledge the teaching and non-teaching staff of the Department of Computer Science and Information Science for their assistance, both direct and indirect, which played a vital role in the successful completion of the project.

Lastly, we express our deep gratitude to our **parents, friends, and well-wishers** for their steadfast support, motivation, and encouragement throughout our academic journey and during this project endeavour.

Mr. Sharan S
(03SU23DS019)

CONTENTS

CERTIFICATE	2
DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT	13
LIST OF FIGURES	14
LIST OF TABLES.....	15
CHAPTER 1 SYNOPSIS.....	18
1.1 Introduction of the Project	18
1.2 Problem Statement	20
1.3 Objective of the Project	21
1.4 Category.....	22
1.5 Tools and Platforms	22
1.6 Hardware Requirements.....	23
1.6.1 Development System	23
1.6.2 Android Device	23
1.7 Software Requirements	24
1.8 Client Requirements	25
CHAPTER 2 SOFTWARE REQUIREMENTS SPECIFICATION	26
2.1 Introduction.....	26
2.2 Purpose.....	27
2.3 Scope.....	27
2.3.1 In Scope.....	27
2.3.2 Out of Scope	29
2.4 Definitions, Acronyms, and Abbreviations.....	30
2.5 References	32
2.6 Overview	33
2.7 Overall System Description.....	33
2.8 Product Perspective	35
2.9 Product Functions.....	36
2.10 User Characteristics.....	38
2.11 General Constraints	38

2.12 Assumptions and Dependencies	39
2.13 Specific Requirements	40
2.13.1 External Interface Requirements	40
2.13.2 User Interface	41
2.13.3 Software Interface.....	42
2.13.4 Hardware Interface	43
2.13.5 Communication Interface.....	43
2.14 Functional Requirements.....	44
2.15 Non-Functional Requirements	48
2.16 Performance Requirements	50
2.17 Design Constraints.....	51
2.18 System Attributes.....	52
2.19 Other Requirements	53
2.19.1 Safety Requirements	53
2.19.2 Security Requirements	54
CHAPTER 3 SYSTEM DESIGN	55
3.1 Introduction.....	55
3.2 Data Flow Diagram — Level 0 (Context Diagram)	56
3.2.1 External Entities.....	57
3.2.2 Data Store	57
3.2.3 Data Flows	58
3.3 Data Flow Diagram (DFD) Overview	59
3.4 Rules Regarding DFD Constraints.....	60
3.5 DFD Symbols	62
3.6 Data Flow Diagram — Level 1 (Detailed DFD)	63
3.6.1 Process Summary.....	64
3.6.2 Data Stores	65
3.6.3 Sensor Data Acquisition Module	65
3.6.4 Signal Processing and Feature Extraction Module	66
3.6.5 Event Classification Module	67
3.6.6 Event Validation and Confirmation Module	68
3.6.7 Eco Scoring and Fuel Estimation Module	69
3.6.8 Evidence Capture Module	70

3.6.9 Trip Persistence Module	71
3.6.10 Intelligence and Analytics Module	72
3.6.11 User Interface and Trip Display Module	73
3.7 Process Descriptions	75
CHAPTER 4 DATABASE DESIGN	76
4.1 Introduction.....	76
4.2 ER Diagram.....	76
4.2.1 Data Objects	78
4.2.2 Attributes	78
4.2.3 Relationships	81
4.2.4 Cardinality	82
4.3 Data Models and Interfaces	83
CHAPTER 5 DETAILED DESIGN.....	84
5.1 System Context	84
5.2 Technology Stack	85
5.3 System Workflow	87
5.4 High-Level Architecture	91
5.5 Core Module Design	94
5.5.1 SensorService	94
5.5.2 TeleDriveProcessor.....	96
5.5.3 RuleBasedAnalyzer	98
5.5.4 MLAnalyzer.....	99
5.5.5 EcoScoreEngine	100
5.5.6 FuelEstimator	101
5.5.7 EvidenceManager and EvidenceCleanupWorker	102
5.5.8 InsightEngine.....	103
5.5.9 HotspotDetector.....	104
5.5.10 EventMapActivity	104
5.6 Inter-Module Communication	105
5.7 Design Rationale	106
5.7.1 Why no cloud backend?	106
5.7.2 Why FusedLocationProviderClient instead of LocationManager?	106
5.7.3 Why SharedPreferences instead of Room/SQLite?	106

5.7.4 Why three detection modes instead of one?	106
5.7.5 Why an event-confirmation state machine?	106
CHAPTER 6 CODING	107
6.1 Introduction.....	107
6.2 Development Environment	108
6.3 Sensor Data Acquisition	109
6.4 Signal Pre-processing.....	112
6.5 Feature Extraction	114
6.6 Rule-Based Event Detection	116
6.7 Machine Learning Model Architecture and Training.....	120
6.7.1 Why CNN Over Rule-Based Only?	120
6.7.2 Model Architecture	121
6.7.3 Dataset Construction.....	122
6.7.4 Training Performance (Offline Evaluation).....	123
6.7.5 Final Model Evaluation (Real-World Validation).....	123
6.8 TFLite Inference Integration	125
6.9 Hybrid Detection Implementation	127
6.10 Event Confirmation and Cooldown Logic	128
6.10.1 State Machine Logic	128
6.10.2 Cooldown Enforcement.....	128
6.10.3 Evidence Trigger Condition	129
6.11 Eco Score and Fuel Estimation	129
6.11.1 Eco Score Computation	129
6.11.2 Fuel Estimation.....	132
6.12 Evidence Capture.....	133
6.13 Local Storage and Persistence Implementation.....	135
6.14 Module File Reference	136
6.14.1 User Interface and Navigation Modules	136
6.14.2 Core Processing and Intelligence Modules.....	138
6.14.3 Storage, Utility, and Integration Modules	139
6.15 ML Pipeline and Model Specification	140
6.15.1 Overview of the TeleDrive ML Subsystem	140
6.15.2 Data Collection and Dataset Construction	141

6.15.3 Windowing, Labelling, and Class Balancing	142
6.15.4 Final Model Identity	143
6.15.5 Input and Output Specification	144
6.15.6 Training Configuration	146
6.15.7 Evaluation and Validation Results	147
6.15.8 TFLite Conversion and Deployment Artefacts.....	148
6.15.9 Android Runtime Inference Flow	149
6.15.10 Hybrid Integration with Rule-Based Detection	150
6.15.11 Resource Characteristics and Current Boundaries	151
CHAPTER 7 TESTING	152
7.1 Introduction.....	152
7.2 Testing Objectives	152
7.3 Testing Steps	153
7.4 Unit Testing	154
7.4.1 Onboarding and App Start Validation.....	154
7.4.2 User Profile Setup	155
7.4.3 Live Trip Alerts and Coaching Tips	157
7.4.4 Analytics Dashboard and Score Trends	158
7.4.5 Profile Management.....	159
7.4.6 Evidence Gallery View and Delete	160
7.4.7 Stop Trip and Session Cleanup	161
7.4.8 ML Dataset Validation and Training Data Pipeline.....	162
7.5 Integration Testing.....	163
7.5.1 Service Initialisation and Permission Flow	163
7.5.2 Onboarding-to-Dashboard Pipeline.....	165
7.5.3 Sensor Service and Background Processing.....	166
7.5.4 Main Dashboard and Navigation Flow	168
7.6 Validation Testing.....	169
7.6.1 Input Validation	169
7.6.2 Requirements Traceability.....	170
7.7 Output Testing	171
7.8 User Acceptance Testing	173
7.8.1 Test Environment.....	173

7.8.2 UAT Results	173
7.8.3 User Feedback Summary	174
7.9 System Testing.....	175
7.9.1 System Testing Tables.....	175
7.10 Test Cases and Results Summary.....	177
CHAPTER 8 USER INTERFACE	178
8.1 Introduction.....	178
8.2 User Interface	178
8.3 UI Screens and Their Roles:	179
8.4 First Launch Screen.....	180
8.5 Permission Setup Screen.....	181
8.6 User Profile Setup Screen	182
8.7 Main Dashboard Screen	183
8.8 Live Trip Monitoring Screen	184
8.9 Event Evidence Gallery Screen	185
8.10 Event Evidence Detail Screen.....	186
8.11 Evidence Delete Confirmation Dialog	187
CHAPTER 9 USER MANUAL.....	188
9.1 Getting Started.....	188
9.2 System Requirements	189
9.3 Installation and Launch	190
9.4 User Profile Setup	191
9.5 Main Dashboard	192
9.6 Starting a Trip.....	193
9.7 During a Trip	194
9.8 Stopping a Trip	195
9.9 Ride Summary Screen	195
9.10 Trip History	196
9.11 Analytics Screen	197
9.12 Event Evidence Gallery	198
9.13 Event Map.....	199
9.14 Permissions and Troubleshooting.....	200
9.15 Safe Usage Guidelines.....	202

CHAPTER 10 CONCLUSION.....	203
10.1 Introduction.....	203
10.2 Result and Analysis	204
10.2.1 Controlled Scoring Validation	204
10.2.2 Empirical Scoring Properties.....	204
10.3 Trip Summary Results.....	206
10.3.1 Ride Summary (low score)	206
10.3.2 Ride Summary (100 score).....	208
10.3.3 Trip History Screen.....	209
10.3.4 Trip Details Analysis	212
10.3.5 Event Map Analysis	216
10.4 Eco Score Behaviour Analysis	218
10.5 Machine Learning Evaluation Analysis	219
10.5.1 Real-World Validation	219
10.5.2 Interpretation	219
10.6 Real-World System Performance Analysis	220
10.6.1 Processing Latency	220
10.6.2 Classification Performance	220
10.6.3 Resource Consumption.....	220
10.7 Overall System Performance Summary.....	221
10.8 Comparative Analysis with Existing Solutions.....	222
10.9 Engineering Iterations Applied.....	223
10.10 Discussion of Findings	224
10.11 Limitations.....	225
10.11.1 Introduction.....	225
10.11.2 Current System Limitations	226
10.11.3 Technical Constraints	228
10.11.4 Deployment and Validation Limitations	230
10.11.5 Privacy and Ethical Considerations	231
10.12 Future Enhancements	233
10.13 Summary of Work Completed	235
10.14 Key Technical Contributions.....	236
10.15 Major Findings and Outcomes	237

10.16 Practical Relevance	239
10.17 Final Concluding Statement	240
CHAPTER 11 BIBLIOGRAPHY	241
11.1 BOOK REFERENCES	241
11.2 WEB REFERENCES	241
CHAPTER 12 PLAGARISM REPORT	242
GLOSSARY	244

ABSTRACT

Two-wheeler riders, particularly those working under delivery and time-constrained conditions, often adopt aggressive riding behaviour that increases fuel consumption, mechanical wear, and safety risk. Existing telematics solutions are frequently unsuitable for individual riders because they depend on dedicated hardware, fleet contracts, or cloud-based subscriptions. Smartphone-based alternatives are commonly designed for four-wheelers and may not account for the vibration patterns and dynamics of two-wheelers.

TeleDrive is a standalone Android application that addresses this gap by performing real-time riding behaviour analysis using the smartphone's built-in accelerometer, gyroscope, GPS, and camera hardware. The application runs as a Foreground Service, acquires inertial data at approximately 50 Hz with periodic GPS updates, processes one-second sensor windows locally, and extracts eight statistical features for classification.

The system provides three user-selectable detection modes: Rule-Based, AI Assist, and Hybrid. In the current implementation, Hybrid mode is the default. The deployed 1D-CNN TensorFlow Lite model was trained on 115,250 class-balanced samples derived from 473,800 raw readings collected across 48 real two-wheeler ride sessions. Reported evaluation results indicate practical performance suitable for real-time use on consumer Android devices. All classification outputs pass through an event-confirmation state machine with hysteresis and cooldown logic before scoring, display, and evidence handling occur.

When a confirmed non-NORMAL event is validated and the camera cooldown has elapsed, the rear camera captures a timestamped JPEG image as contextual evidence. At trip completion, the system generates an eco score (0–100), estimated fuel impact, financial loss estimate, behavioural insights, trip history, and route-based review through the Event Map interface. All sensing, inference, scoring, evidence management, and storage occur on-device. TeleDrive supports Android 8.0 (API 26) and later and requires no cloud backend or external telematics hardware for its core pipeline.

Keywords: mobile telematics, Android Foreground Service, inertial measurement unit, riding behaviour analysis, 1D-CNN, TensorFlow Lite, edge AI, eco score, two-wheeler safety, event map, hotspot detection.

LIST OF FIGURES

Figure 3.1 Level 0 Data Flow Diagram (Context Diagram) of the TeleDrive System	56
Figure 3.2 Level 1 Data Flow Diagram of the TeleDrive System	63
Figure 4.1 Entity Relationship Diagram of the TeleDrive System	76
Figure 5.1 End-to-End System Workflow	90
Figure 5.2 High-Level Layered Architecture of TeleDrive	93
Figure 6.1 Signal Pre-processing Pipeline	112
Figure 6.2 1D-CNN Model Architecture	121
Figure 6.3 Confusion Matrix — Final 1D-CNN Model (Real-World Validation).	124
Figure 8.1 First Launch Screen	180
Figure 8.2 Runtime Permission Request During Initial Setup	181
Figure 8.3 Vehicle and User Profile Configuration Screen	182
Figure 8.4 Main Dashboard Screen	183
Figure 8.5 Live Trip Monitoring Screen — Real-Time Riding Behaviour Detection During Active Trip	184
Figure 8.6 Event Evidence Gallery Screen	185
Figure 8.7 Event Evidence Detail and Export Screen	186
Figure 8.8 Evidence Deletion Confirmation Dialog	187
Figure 10.1 Ride Summary Screen — Sample Output (Score: 93)	206
Figure 10.2 Ride Summary Screen (Score: 100)	208
Figure 10.3 Trip History Screen Showing Stored Trips and Aggregate Average Score ...	209
Figure 10.4 Trip Details Screen Showing Perfect Eco Score for Smooth Riding Behaviour	212
Figure 10.5 Trip Details Screen Showing Reduced Eco Score for Aggressive Riding Behaviour	213
Figure 10.6 Event Map Interface of TeleDrive	216
Figure 10.7 TeleDrive Event Map Showing Unsafe Events, Hotspot Detection, and Evidence	217
Figure 10.8 Eco Score Trend — Last 7 Trips (Analytics Screen)	218
Figure 10.9 TeleDrive System — Consolidated Performance Evaluation	221

LIST OF TABLES

Table 2.1 Definitions, Acronyms, and Abbreviations	30
Table 2.2 Product Functions Summary.....	34
Table 2.3 User Interface Requirements.....	41
Table 2.4 Software Interface Requirements	42
Table 2.5 Hardware Interface Requirements	43
Table 2.6 Communication Interface Requirements.....	43
Table 2.7 Functional Requirements	44
Table 2.8 Non-Functional Requirements	48
Table 2.9 Performance Requirements.....	50
Table 2.10 Design Constraints.....	51
Table 2.11 Safety Requirements	53
Table 2.12 Security Requirements	54
Table 3.1 External Entities	57
Table 3.2 Data Store.....	57
Table 3.3 Data Flows.....	58
Table 3.4 DFD Symbols	62
Table 3.5 Process Summary	64
Table 3.6 Data Stores	65
Table 3.7 Process Descriptions	75
Table 4.1 Data Objects	78
Table 4.2 UserProfile Attributes.....	78
Table 4.3 TripSummary Attributes	79
Table 4.4 TripEvent Attributes (nested within TripSummary)	80
Table 4.5 EvidenceItem Attributes	80
Table 4.6 Cardinality	82
Table 4.7 Data Models and Interfaces.....	83
Table 5.1 Technology Stack	85
Table 5.2 High-Level Architecture	91
Table 5.3 SensorService Key Parameters	95
Table 5.4 Processing stages	97
Table 5.5 Extracted features (FeatureVector)	97
Table 5.6 RuleBasedAnalyzer Thresholds	98
Table 5.7 Model assets (bundled in APK assets)	99
Table 5.8 per-event degradation penalties.....	101
Table 5.9 InsightEngine evaluates four analytical dimensions per trip	103
Table 5.10 Inter-Module Communication	105
Table 6.1 Development Environment.....	108
Table 6.2 ML Training Dataset Class Distribution (After Balancing).....	122
Table 6.3 Offline Training Evaluation Metrics (Balanced Hold-Out Test Set).....	123
Table 6.4 User Interface and Navigation Modules	136
Table 6.5 Core Processing and Intelligence Modules.....	138

Table 6.6 Storage, Utility, and Integration Modules.....	139
Table 6.7 TeleDrive ML Subsystem Summary	140
Table 6.8 Data Collection Summary	141
Table 6.9 Dataset Preparation Summary	142
Table 6.10 Final Model Identity	143
Table 6.11 Input Specification.....	144
Table 6.12 Input Feature Channels	145
Table 6.13 Output Specification	145
Table 6.14 Output Classes	146
Table 6.15 Training Configuration	146
Table 6.16 Evaluation and Validation Summary	147
Table 6.17 Deployment Artefacts	148
Table 6.18 Runtime Inference Flow	149
Table 6.19 Hybrid Decision Rules	150
Table 6.20 Runtime Resource Characteristics	151
Table 7.1 Unit Test Cases — Onboarding and App Start Validation	154
Table 7.2 Unit Test Cases — User Profile Setup	155
Table 7.3 Unit Test Cases — Live Trip Alerts and Coaching Tips.....	157
Table 7.4 Unit Test Cases — Analytics Dashboard and Score Trends	158
Table 7.5 Unit Test Cases — Profile Management	159
Table 7.6 Unit Test Cases — Evidence Gallery View and Delete	160
Table 7.7 Unit Test Cases — Stop Trip and Session Cleanup.....	161
Table 7.8 Unit Test Cases — ML Dataset Validation and Training Data Pipeline	162
Table 7.9 Integration Test Cases — Service Initialisation and Permission Flow	163
Table 7.10 Integration Test Cases — Onboarding-to-Dashboard Pipeline	165
Table 7.11 Integration Test Cases — Sensor Service and Background Processing.....	166
Table 7.12 Integration Test Cases — Main Dashboard and Navigation Flow	168
Table 7.13 Input Validation	169
Table 7.14 Functional Requirements Traceability	170
Table 7.15 Output Test Cases	171
Table 7.16 Test Environment.....	173
Table 7.17 UAT Results.....	173
Table 7.18 System Test Cases and Results.....	175
Table 8.1 User Interface Requirements.....	178
Table 8.2 UI Screens and Their Roles	179
Table 9.1 Practical System Requirements for TeleDrive.....	189
Table 9.2 Event Map Marker Legend	199
Table 9.3 Permissions and Troubleshooting Guide	200
Table 10.1 Controlled Scoring Validation Tests.....	204
Table 10.2 Output-Validation Rides — Full Trip History	210
Table 10.3 Real Trip Test Results.....	214
Table 10.4 Offline Classification Performance.....	219
Table 10.5 Comparative Feature Summary.....	222
Table 10.6 Engineering Issues and Applied Fixes	223

Table 10.7 Current System Limitations	226
Table 10.8 Technical Constraints	228
Table 10.9 Deployment and Validation Limitations	230
Table 10.10 Future Enhancements for TeleDrive.....	233

CHAPTER 1 SYNOPSIS

Title of the Project

TeleDrive — Edge-Based Riding Behaviour Analysis Using Smartphone Inertial Sensors and On-Device Machine Learning

1.1 Introduction of the Project

Two-wheelers are widely used for daily commuting, business, and delivery services. The rapid expansion of the gig economy has introduced significant time pressure on riders working for platforms such as Swiggy, Zomato, and Zepto. Factors including traffic congestion, delays, and poor road conditions often lead to aggressive riding behaviours such as harsh acceleration, abrupt braking, and unsafe manoeuvres. These behaviours contribute to increased fuel consumption, vehicle wear, and accident risk, while the associated financial and safety impacts often remain unnoticed due to the lack of accessible real-time monitoring systems.

This issue is supported by global road-safety data. According to the World Health Organization, approximately 1.19 million people die annually in road traffic accidents worldwide, with a significant proportion involving vulnerable road users such as motorcyclists. Existing telematics solutions are largely unsuitable for individual riders, as they typically depend on external hardware, subscription models, or continuous cloud connectivity. Moreover, most smartphone-based solutions are designed for four-wheelers and do not accurately capture the motion characteristics of two-wheelers.

To address this gap, TeleDrive is developed as a smartphone-based telematics system that performs real-time riding behaviour analysis using built-in sensors such as the accelerometer, gyroscope, GPS, and camera. The application operates as an Android Foreground Service, collecting sensor data and processing it locally in one-second sliding windows. A signal-conditioning pipeline consisting of spike rejection, median filtering, and moving average is applied, followed by statistical feature extraction for classification.

Riding behaviour is classified into Normal, Harsh Acceleration, Harsh Braking, and Unstable Ride using three detection modes: a speed-adaptive rule-based engine, an on-device one-dimensional Convolutional Neural Network implemented using TensorFlow Lite, and a hybrid mode combining both approaches. The CNN model is trained on 115,250 samples derived from real-world ride data and achieves 96.0% accuracy with a macro F1-score of 0.92 in offline evaluation, along with 98.76% accuracy on labelled real-world validation data.

To enhance reliability, the system incorporates an event-confirmation state machine with hysteresis and cooldown logic. Confirmed events trigger automatic image capture using the device camera for contextual evidence. At the end of each trip, TeleDrive generates a comprehensive ride summary including an eco score (0–100) based on a diminishing-penalty model, estimated fuel consumption, financial loss, and behavioural insights derived from historical data. An interactive Event Map provides visualization of routes, detected events, and high-risk locations.

All processing, including sensing, analysis, inference, and storage, is performed entirely on-device, ensuring privacy, low cost, and independence from network connectivity. TeleDrive is designed for Android 8.0 (API 26) and above, making it deployable on widely available mid-range smartphones.

1.2 Problem Statement

Road accidents and hidden financial losses caused by aggressive riding behaviour represent a significant and growing challenge in urban environments, particularly among two-wheeler users.

In India, the scale of this issue is substantial. The Ministry of Road Transport and Highways reported 4,61,312 road accidents and 1,68,491 deaths in 2022, highlighting road safety as a major public-health concern. Two-wheeler riders are disproportionately affected, with nearly half of all road accident deaths involving this group.

Several key challenges contribute to this problem:

Time-pressure-driven unsafe riding. The growth of gig-economy platforms such as Zomato, Swiggy, and Zepto has introduced strict delivery timelines. Riders must manage multiple constraints-including traffic congestion, order delays, and navigation-within a fixed time window. This often leads to aggressive behaviours such as rapid acceleration, late braking, and unsafe manoeuvres, increasing crash risk and reducing ride stability.

Invisible cumulative cost of aggressive riding. Frequent harsh acceleration and braking increase fuel consumption and accelerate wear of vehicle components such as brakes, tyres, and chains. These costs directly reduce rider earnings, yet remain largely unquantified due to the absence of practical monitoring tools.

Absence of real-time feedback. Riders lack systems that provide immediate feedback on unsafe behaviour during a trip. Existing post-trip statistics, when available, do not support real-time behavioural correction.

Economic inaccessibility of existing solutions. Commercial telematics systems such as Vamosys, Letstrack, and Fleetx are designed for fleet-level deployment, requiring external hardware and subscription fees, making them unsuitable for individual riders.

Inaccuracy of existing smartphone solutions. Most mobile applications are designed for four-wheelers and are not calibrated for the motion dynamics of two-wheelers, resulting in unreliable detection and higher false positives.

Cloud dependency and privacy concerns. Many systems rely on cloud-based processing, leading to latency, data costs, and concerns regarding continuous tracking of user location and behaviour.

Lack of spatial and evidence-based analytics. Existing solutions typically provide only summary statistics and lack features such as event mapping, route visualization, evidence capture, and hotspot identification necessary for actionable insights.

These limitations highlight the need for a low-cost, smartphone-based, and privacy-preserving solution that provides accurate, real-time riding behaviour analysis tailored specifically for two-wheelers. TeleDrive is developed to address these challenges.

1.3 Objective of the Project

The TeleDrive system is designed to monitor and improve riding behaviour using a smartphone-based approach. The main objectives of the project are:

1. To monitor riding behaviour in real time using smartphone sensors.
2. To detect harsh acceleration, harsh braking, and unstable riding events.
3. To improve detection accuracy using a combination of rule-based and machine learning techniques.
4. To generate an eco score and provide trip performance insights.
5. To estimate fuel usage and riding efficiency.
6. To maintain trip history for analysis and comparison.
7. To capture contextual evidence for significant riding events.
8. To provide an interactive Event Map with route tracking and event markers.
9. To identify repeated high-risk zones using hotspot detection.
10. To offer an affordable solution without requiring external hardware.

1.4 Category

- Mobile Application Development
- Artificial Intelligence
- Machine Learning
- Android Application
- Edge Computing
- Smart Transportation
- Safety Analytics

1.5 Tools and Platforms

Tool / Technology	Purpose
Android Studio	Application development environment
Kotlin	Primary programming language
Jetpack Compose	User interface development
TensorFlow Lite	On-device machine learning inference
Python	Model training and data analysis
OpenStreetMap / osmdroid	Event map integration
CameraX	Evidence image capture
Gradle	Build automation
Git	Version control

1.6 Hardware Requirements

1.6.1 Development System

Component	Minimum Specification
Processor	Intel Core i5 / AMD Ryzen 5
RAM	8 GB
Storage	20 GB free
Operating System	Windows 10 / 11

1.6.2 Android Device

Component	Minimum Specification
Android Version	8.0 (API 26) or later
Sensors	Accelerometer, Gyroscope, GPS
Camera	Rear-facing camera
RAM	3 GB
Storage	100 MB free

1.7 Software Requirements

- Windows 10 / 11
- Android Studio
- JDK 11
- Kotlin SDK
- Gradle
- TensorFlow Lite Libraries
- Python 3.x
- NumPy, Pandas, scikit-learn
- CameraX Library
- Gson Library
- WorkManager
- Google Play Services Location AP

1.8 Client Requirements

The client or end user requires a simple and reliable application that can help improve riding behaviour without expensive devices or subscriptions. The system should be easy to use, accurate, and suitable for everyday riders.

Client expectations include:

1. Real-time riding behaviour monitoring
2. Accurate unsafe event detection
3. Minimal user interaction required during active trips - trip initiation and termination through single-touch controls
4. Clear score and analytics reports
5. Fuel and cost awareness
6. Event location tracking through a map
7. Safe local data storage
8. Fast performance on normal Android phones
9. Affordable and practical solution for riders and delivery workers
10. Privacy-focused offline functionality.

CHAPTER 2 SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Introduction

This chapter specifies the functional and non-functional requirements of the TeleDrive system, an Android-based application for real-time two-wheeler riding behaviour analysis. The system integrates smartphone sensor data with on-device processing and edge machine learning to detect riding events and generate trip-level insights.

The requirements defined in this chapter describe the expected behaviour of core system components, including sensor data acquisition, signal processing, event detection, scoring, evidence capture, and data storage. Each requirement is expressed in measurable and testable terms and is aligned with the implemented modules in the final system.

The requirements documented in this chapter were derived from four sources: analysis of the two-wheeler riding domain and the gig-economy delivery ecosystem; a review of existing telematics solutions and their limitations for individual riders; empirical data collected across 48 real ride sessions during the development phase; and iterative testing against the implemented Android application. Each requirement is traceable to a specific system module and can be verified against the delivered software artefacts.

The remainder of this chapter is organised as follows: sections 2.2 through 2.6 define the purpose, scope, terminology, references, and document overview. Sections 2.7 through 2.12 describe the overall system, product functions, user characteristics, constraints, and assumptions. Section 2.13 specifies external interface requirements. Section 2.14 enumerates the functional requirements. Sections 2.15 through 2.18 cover performance requirements, design constraints, system quality attributes, and other requirements, including safety and security.

2.2 Purpose

This document specifies the functional and non-functional requirements of the TeleDrive system, including its operational constraints and expected behaviour during real-time riding analysis. It provides a structured description of how the system acquires sensor data, processes it, detects riding events, and generates outputs such as eco score, insights, and evidence

The specification acts as a reference for validating system behaviour and supports the design of test cases used to evaluate the implemented application. It is intended to guide evaluation, future modification, and extension of the system by clearly describing its components and their interactions.

2.3 Scope

TeleDrive is a standalone Android mobile application that acquires inertial sensor and GPS data from smartphone hardware, processes that data using on-device signal-processing algorithms and machine learning inference, stores all results locally, and presents them through a declarative Jetpack Compose user interface. The system has no server-side components and no required network connectivity for its core detection, scoring, and storage pipeline. Map tile rendering through osmdroid requires a one-time internet connection for initial cache population; all subsequent map views are served offline.

2.3.1 In Scope

- Real-time accelerometer and gyroscope data collection through Android SensorManager at TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE.
- GPS speed, location, bearing, and distance tracking through FusedLocationProviderClient with PRIORITY_HIGH_ACCURACY, one-second interval, and 0.5-metre minimum displacement.
- Signal pre-processing through spike rejection, three-point median filtering, and five-point moving average smoothing.

- Eight-feature extraction for each one-second sensor window assembled into a FeatureVector.
- Three user-selectable detection modes: Rule-Based, AI Assist (1D-CNN via TensorFlow Lite), and Hybrid (default).
- Event-confirmation state machine with hysteresis-based entry and exit thresholds.
- Eco score computation (0–100) using a diminishing-penalty model.
- Fuel consumption estimation and financial loss calculation in Indian Rupees.
- Automatic photographic evidence capture via CameraX on confirmed events.
- Automatic evidence cleanup using WorkManager (daily at 03:00, files older than 24 hours).
- Interactive Event Map with osmdroid route polyline, colour-coded event markers, pixel-based clustering, sliding bottom sheet with per-event coaching and ± 8 -second temporally matched evidence, and multi-trip hotspot detection.
- Trip history persistence using SharedPreferences with Gson JSON serialisation.
- Comparative, trend, and streak-based riding insights through InsightEngine.
- Analytics dashboard with eco score trend chart for the last seven trips.
- Onboarding flow with user and vehicle profile setup.
- Target users: delivery riders, daily commuters, students, and small-scale fleet users.

2.3.2 Out of Scope

- Continuous video recording during trips.
- Cloud storage or remote analytics.
- Multi-user accounts or social features.
- Live fleet dashboards or third-party remote monitoring.
- OBD-II or external vehicle hardware integration.
- Turn-by-turn navigation.
- Automatic crash detection with SOS alerts.
- Live fuel-price API integration.
- iOS, Wear OS, or tablet support.

2.4 Definitions, Acronyms and Abbreviations

Table 2.1 Definitions, Acronyms and Abbreviations

Term	Definition
IMU	Inertial Measurement Unit - the combination of accelerometer and gyroscope sensors used for motion measurement.
FusedLocation ProviderClient (FLPC)	Google Play Services API that combines GPS, Wi-Fi and cellular signals to deliver optimised location updates.
Edge AI	Machine learning inference executed locally on the user's device rather than on a remote server.
TensorFlow Lite (TFLite)	A lightweight runtime for deploying trained machine learning models on mobile and embedded devices.
1D-CNN	One-Dimensional Convolutional Neural Network a deep learning architecture that applies convolutional kernels along a single spatial axis (time).
FeatureVector	An eight-element data class containing statistical descriptors extracted from one second of sensor data.
Eco Score	A 0–100 quality metric representing overall riding smoothness for a single trip.
Foreground Service	An Android service type that runs with a persistent notification and elevated process priority, preventing OS termination during active trips.
Event Map	An interactive osmdroid-based map screen that displays the trip route, event markers, coaching, evidence, and risk hotspots.
Hotspot	A geographic location where unsafe riding events have been detected across two or more distinct trips within a 150-metre radius.
Haversine Distance	A formula for calculating the great-circle distance between two points on a sphere given their latitude and longitude.

LiveDataBus	A lightweight singleton callback mechanism used within TeleDrive for real-time UI updates during an active trip.
Detection Mode	One of three classification strategies: Rule-Based, AI Assist (ML) or Hybrid.
Hysteresis	A state machine design where the threshold for entering a state differs from the threshold for leaving it, used to suppress rapid state flickering.
API	Application Programming Interface.
GPS	Global Positioning System.
SRS	Software Requirements Specification.
IEEE	Institute of Electrical and Electronics Engineers.
UI	User Interface.
JPEG	Joint Photographic Experts Group - the image compression format used for evidence photographs.
JSON	JavaScript Object Notation - the serialisation format used for trip and profile persistence.
Hz	Hertz - unit of frequency (samples per second).
km/h	Kilometres per hour.
km/L	Kilometres per litre.
EMA	Exponential Moving Average.

2.5 References

IEEE. *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. 1998.

Google. *Android Developer Documentation: SensorManager*. 2024.
URL: <https://developer.android.com/reference/android/hardware/SensorManager>

Google. *Android Developer Documentation: Foreground Services*. 2024.
URL: <https://developer.android.com/guide/components/foreground-services>

Google. *Android Developer Documentation: FusedLocationProviderClient*. 2024.
URL: <https://developer.android.com/training/location>

Google. *TensorFlow Lite: Deploy Machine Learning on Mobile and Edge Devices*. 2024.
URL: <https://www.tensorflow.org/lite>

JetBrains. *Kotlin Documentation*. 2024.
URL: <https://kotlinlang.org/docs/home.html>

Google. *Jetpack Compose Documentation*. 2024.
URL: <https://developer.android.com/jetpack/compose>

osmdroid Contributors. *osmdroid: OpenStreetMap Tools for Android*. 2024.
URL: <https://github.com/osmdroid/osmdroid>

Google. *Android Developer Documentation: CameraX Overview*. 2024.
URL: <https://developer.android.com/training/camerax>

Google. *Android Developer Documentation: WorkManager*. 2024.
URL: <https://developer.android.com/topic/libraries/architecture/workmanager>

2.6 Overview

This SRS document is organised into three logical segments. The first segment (Sections 2.1 through 2.6) provides introductory material, establishes the document's purpose and scope, and defines the terminology used throughout. The second segment (Sections 2.7 through 2.12) describes the product from a general perspective: what the system does, who uses it, what constraints it operates under, and what external dependencies it relies upon. The third segment (Sections 2.13 through 2.18) specifies detailed requirements: external interfaces, functional behaviour, performance targets, design constraints, system quality attributes, and safety and security provisions.

The requirements are numbered sequentially within each category (FR-01 through FR-32 for functional requirements, PR-01 through PR-10 for performance requirements, and related identifiers for other categories) to support traceability between this specification and the system's implementation, testing, and validation artefacts.

2.7 Overall System Description

TeleDrive is designed for individual two-wheeler riders, including gig-economy delivery workers, daily commuters, and students, who wish to monitor, quantify, and improve their riding behaviour without purchasing dedicated hardware or subscribing to a fleet telematics service. The system performs all core functions - data acquisition, signal processing, event detection, eco scoring, fuel and financial estimation, evidence capture, insight generation, spatial visualisation, and data persistence autonomously after the rider initiates a trip. No continuous user interaction is required once a trip has started. The system terminates monitoring and presents results when the rider stops the trip.

Table 2.2 Product Functions Summary

Function	Description
Trip Recording	Continuous sensor and GPS collection as an Android Foreground Service
Signal Processing	Three-stage pipeline (spike rejection, median filter, moving average) with eight-feature extraction
Event Detection	Identifies harsh acceleration, harsh braking, and unstable riding via speed-adaptive thresholds
AI Classification	On-device 1D-CNN inference through TensorFlow Lite (AI Assist mode)
Hybrid Consensus	Parallel rule + ML execution with confidence-gated fusion (default mode)
Event Confirmation	State machine with hysteresis entry/exit and UI flicker suppression
Eco Scoring	Real-time score (0–100) with logarithmic diminishing-returns penalty model
Fuel & Cost Estimation	Per-event mileage degradation and financial loss in INR
Evidence Capture	Automatic rear-camera JPEG on confirmed high-severity events
Evidence Cleanup	WorkManager-scheduled daily purge of evidence older than 24 hours
Event Map	Interactive route visualisation with markers, clustering, coaching, evidence matching, and hotspot detection
Trip History	Persistent record of all completed trips with drill-down
Analytics	Multi-trip eco score trend chart and aggregate statistics
Contextual Insights	Comparative, trend, and streak analysis across trip history
Onboarding	One-time user and vehicle profile setup for fuel calibration

2.8 Product Perspective

TeleDrive is a self contained, single-device application. It does not form part of a larger system, does not communicate with any backend server, and does not depend on any external hardware beyond the Android smartphone itself. The application replaces the role traditionally served by dedicated fleet telematics hardware (OBD-II dongles, GPS tracker devices) and cloud analytics platforms by consolidating data acquisition, processing, classification, scoring, evidence management, and visualisation within a single Android APK.

The system operates within the standard Android application lifecycle. During an active trip, it runs as a Foreground Service to maintain sensor access and prevent OS termination. Between trips, no background processing occurs except for the daily evidence cleanup worker scheduled through WorkManager.

From an ecosystem perspective, TeleDrive occupies the following position:

Hardware Layer: The application relies entirely on the device's built-in TYPE_LINEAR_ACCELERATION sensor, TYPE_GYROSCOPE sensor, GPS receiver (accessed through FusedLocationProviderClient), and rear camera (accessed through CameraX). No external sensors, Bluetooth peripherals, or OBD-II adapters are required.

Platform Layer: The application targets Android 8.0 (API 26) through Android 16 (API 36) and requires Google Play Services for FusedLocationProviderClient.

Data Layer: All trip data, user profiles, and evidence photographs are stored locally on the device. SharedPreferences with Gson JSON serialisation is used for structured data, while the filesystem is used for evidence images.

Network Layer: The core detection, scoring, and storage pipeline is fully offline. Network connectivity is required only for the initial download of OpenStreetMap tiles for the Event Map; once cached, map rendering is offline.

2.9 Product Functions

1. **Real-time sensor data acquisition** at approximately 50 Hz from TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE through Android SensorManager, merged with GPS speed, location, and bearing from Google Play Services' FusedLocationProviderClient at one-second intervals with high-accuracy priority and 0.5-metre minimum displacement.
2. **Signal pre-processing pipeline:** spike rejection (samples with 3D magnitude $>15 \text{ m/s}^2$ discarded) $\rightarrow$ three-point sliding median filter on forward acceleration $\rightarrow$ five-point moving average smoothing.
3. **Feature extraction:** eight statistical descriptors per one-second window assembled into a FeatureVector - mean forward acceleration, peak forward acceleration, minimum forward acceleration, horizontal acceleration standard deviation, mean gyroscope magnitude, peak gyroscope magnitude, gyroscope jitter (standard deviation of per-sample gyro magnitude), and vertical-axis standard deviation (road-roughness indicator).
4. **Three-mode event classification:**
 - *Rule-Based:* speed-adaptive threshold engine with five-window sustained-detection buffers, GPS speed-trend validation, magnitude-dominance checks (20% margin), variance-window filtering, and pattern-persistence gating ($\geq 30\%$ mini-window corroboration).
 - *AI Assist (ML):* 1D-CNN TensorFlow Lite model (50×9 input, four-class softmax) with speed-aware safety filters and prediction buffer smoothing.
 - *Hybrid (default):* both engines run in parallel; agreement floors confidence at 85%; disagreement resolved by physics-priority protocol (rule wins for directional events, ML governs unstable-ride suppression with 88% confidence gate).
5. **Event-confirmation state machine** with consecutive-detection entry threshold, hysteresis-based exit (holds event state until sufficient normal windows are observed), and a stricter UI display threshold to prevent visual flicker.

6. **Eco score computation** using a diminishing-penalty model base penalties of 8 (acceleration), 10 (braking), and 9 (instability) points, scaled by normalised severity and dampened by a $1/\sqrt{n}$ factor per event occurrence.
7. **Fuel consumption estimation** through FuelEstimator (per-event mileage penalties: -0.4 km/L per harsh acceleration, -0.3 per harsh brake, -0.5 per instability; effective mileage clamped to 30–70 km/L) and **financial loss calculation** from cumulative fuel wastage at configurable petrol price (default ₹105/L).
8. **Photographic evidence capture** through an invisible 1×1 -pixel CameraX activity with CAPTURE_MODE_MINIMIZE_LATENCY and flash disabled, triggered on confirmed non-NORMAL events, subject to cooldown; files stored with event-type-encoded timestamps.
9. **Trip history persistence and retrieval** using SharedPreferences with Gson JSON serialisation.
10. **Contextual insight generation** from historical trip comparison through a rule-based InsightEngine - comparative (vs. last trip), trend (vs. last 7 trips), streak (consecutive smooth rides ≥ 85) - returning the top 3 insights ranked by severity.
11. **Interactive Event Map** through osmdroid with OpenStreetMap MAPNIK tiles — dual-layer route polyline, colour-coded circle markers with event-type letters (A/B/U/S/E), pixel-based clustering, sliding bottom sheet with per-event coaching and ± 8 -second temporally matched evidence, and hotspot toggle overlay (150-metre Haversine clustering, minimum 2 distinct trips).

2.10 User Characteristics

The intended user is an individual adult two-wheeler rider. No prior technical knowledge of telematics, sensor fusion, or machine learning is required. The primary interaction model consists of four touch actions: starting a trip, optionally selecting a detection mode (Rule-Based, AI Assist, or Hybrid), stopping the trip, and reviewing results. All subsequent analysis, evidence capture, scoring, and summary generation are fully automated.

A secondary user profile, small fleet operators managing a limited number of delivery riders, may use the application by installing it on each rider's device and reviewing individual trip histories. No centralised fleet dashboard is provided in the current version.

2.11 General Constraints

- **Platform constraint.** The application targets Android 8.0 (API 26) through Android 16 (API 36). iOS, Wear OS, and other mobile platforms are out of scope.
- **Sensor hardware constraint.** The device must include functional `TYPE_LINEAR_ACCELERATION` and `TYPE_GYROSCOPE` hardware sensors. Devices with only a basic accelerometer (without gyroscope) or virtual/software-emulated sensors cannot run the full detection pipeline.
- **Mounting orientation constraint.** The system assumes the phone is mounted in portrait orientation on the handlebars with the device X-axis approximately aligned with the vehicle's forward direction of travel. Significant rotation from this orientation reduces forward-acceleration extraction accuracy. Automatic orientation calibration is not implemented in this version.
- **Speed gate constraint.** Event detection is suppressed below speed-specific minimum thresholds: 15 km/h for harsh acceleration, 12 km/h for harsh braking, and 8 km/h for unstable riding. Trips conducted exclusively below these thresholds will record no events of the corresponding type.

- **Storage constraint.** Evidence photographs older than 24 hours are automatically purged by EvidenceCleanupWorker. Long-term archival requires manual export by the user before the cleanup window.
- **GPS availability constraint.** Fuel estimation, distance calculation, speed-trend validation, and map rendering require GPS signal. Indoor or GPS-denied environments will produce incomplete trip summaries and suppress speed-dependent event detection.
- **Map tile constraint.** The Event Map requires an initial internet connection to fetch and cache OpenStreetMap tiles. Once cached, tiles are served offline. Without this initial fetch, the map screen displays without a base layer

2.12 Assumptions and Dependencies

- The Android device clock is accurate to at least 10 ms resolution, required for the duplicate-guard timestamp comparison in SensorService.onSensorChanged().
- Google Play Services is installed and up to date on the target device, which is required for FusedLocationProviderClient. Devices without Play Services may not receive GPS data.
- TensorFlow Lite runtime (version 2.17.0) is available through the bundled AAR dependency. No separate TFLite installation is required on API 26+.
- The device camera and camera permission are granted. If camera permission is denied, evidence capture degrades gracefully — the event is still detected and scored, but no photograph is captured. The application does not crash.
- GPS is enabled and accessible. If GPS becomes unavailable mid-trip, the system defaults speed to zero, which activates speed gates and suppresses event detection until GPS is restored. Distance accumulation pauses during the outage.
- The Gson library (bundled as a Gradle dependency) correctly serialises and deserialises TripSummary, TripEvent, and UserProfile data classes, including nullable fields for backward compatibility.
- SharedPreferences storage is adequate for the expected trip volume. At approximately 2–4 KB per trip summary (including serialised event lists), the system supports several thousand trips before approaching practical SharedPreferences limits. For higher volumes, migration to Room/SQLite is identified as a future enhancement.

2.13 Specific Requirements

This section specifies the detailed technical requirements of TeleDrive, organised into external interface requirements (Sections 3.13.1–3.13.5), functional requirements (Section 3.14), performance requirements (Section 3.15), design constraints (Section 3.16), system quality attributes (Section 3.17), and other requirements, including safety and security (Section 3.18).

All requirements are stated in verifiable terms and are traceable to specific modules in the implemented system.

2.13.1 External Interface Requirements

TeleDrive interfaces with four categories of external entities: the user through the graphical interface, the Android software platform through its APIs, the device hardware through sensor and camera subsystems, and network services through tile download. Each interface category is specified in the subsections that follow.

2.13.2 User Interface

Table 2.3 User Interface Requirements

ID	Requirement
UI-01	The application shall present all screens using Jetpack Compose with Material 3 design language and a dark colour theme optimised for outdoor visibility.
UI-02	The onboarding screen shall collect the rider's name, vehicle name, engine displacement (CC), and expected mileage (km/L) before allowing access to the main dashboard.
UI-03	The main dashboard (MainActivity) shall provide controls to start a trip, toggle AI Assist mode, and navigate to Trips, Analytics, Profile, and Evidence screens.
UI-04	The live trip screen (LiveTripActivity) shall display real-time speed, current event state (NORMAL / HARSH_ACCELERATION / HARSH_BRAKING / UNSTABLE_RIDE), detection source label (RULE / ML / HYBRID), confidence percentage, eco score, and a contextual coaching tip — all updated via LiveDataBus callbacks.
UI-05	The ride summary screen (RideSummaryActivity) shall display eco score with colour coding (green 90–100, yellow 75–89, red 0–74), distance, duration, average speed, fuel consumed, financial loss, event counts by type, the top three behavioural insights, and a link to the most recent evidence photograph.
UI-06	The Event Map screen (EventMapActivity) shall render a full-screen osmdroid map with a dual-layer route polyline, colour-coded circle markers with single-letter type identifiers (A, B, U, S, E), a sliding bottom sheet with event detail and temporally matched evidence, and a floating action button to toggle the hotspot overlay.
UI-07	The analytics screen (AnalyticsActivity) shall display an eco score trend chart for the last seven trips and aggregate statistics, including average score, total trips, and total events.
UI-08	The evidence gallery screen (EventEvidenceActivity) shall display captured evidence photographs sorted newest-first, capped at 50 items, with options to export to the device gallery or delete individual items.
UI-09	The profile screen (ProfileActivity) shall allow editing of the rider's name, vehicle name, engine displacement, and expected mileage after initial onboarding.

2.13.3 Software Interface

Table 2.4 Software Interface Requirements

ID	Requirement
SI-01	The application shall use Android SensorManager API to register listeners for TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE at SENSOR_DELAY_GAME frequency (~50 Hz).
SI-02	The application shall use Google Play Services FusedLocationProviderClient (play-services-location 21.0.1) with PRIORITY_HIGH_ACCURACY, 1000 ms interval, 500 ms fastest interval, and 0.5-metre smallest displacement for GPS data acquisition.
SI-03	The application shall use TensorFlow Lite runtime (version 2.17.0) with GPU delegate support for on-device CNN inference. The model, scaler parameters, and label mapping shall be loaded from bundled APK assets.
SI-04	The application shall use CameraX library (version 1.3.x) for programmatic image capture with CAPTURE_MODE_MINIMIZE_LATENCY and FLASH_MODE_OFF.
SI-05	The application shall use osmdroid library (version 6.1.18) with OpenStreetMap MAPNIK tile source for map rendering.
SI-06	The application shall use Gson library (version 2.10.1) for JSON serialisation and deserialisation of TripSummary, TripEvent, and UserProfile objects to and from SharedPreferences.
SI-07	The application shall use Android WorkManager (version 2.9.0) for scheduling periodic background evidence cleanup.
SI-08	The application shall target compileSdk 36, targetSdk 36, and minSdk 26, compiled against JVM target 11 using Kotlin 2.0.21.

2.13.4 Hardware Interface

Table 2.5 Hardware Interface Requirements

ID	Requirement
HI-01	The device shall provide a hardware TYPE_LINEAR_ACCELERATION sensor (accelerometer with gravity compensation). The application reads three-axis linear acceleration (ax, ay, az) values from this sensor.
HI-02	The device shall provide a hardware TYPE_GYROSCOPE sensor. The application reads three-axis angular velocity (gx, gy, gz) values from this sensor.
HI-03	The device shall provide a GPS receiver accessible through FusedLocationProviderClient. The application derives speed, bearing, latitude, longitude, and cumulative distance from GPS updates.
HI-04	The device shall provide a rear-facing camera accessible through the CameraX API. The application captures JPEG still images as event evidence.
HI-05	The device shall provide sufficient internal or external storage for evidence photographs (estimated 50–200 KB per JPEG) and SharedPreferences data (2–4 KB per trip summary).

2.13.5 Communication Interface

Table 2.6 Communication Interface Requirements

ID	Requirement
CI-01	The application shall download OpenStreetMap MAPNIK tiles over HTTPS for initial map cache population. Once tiles are cached locally, no further network communication is required for map rendering.
CI-02	The core detection, scoring, evidence capture, and data storage pipeline shall operate without any network connectivity. No data shall be transmitted to any external server during normal operation.
CI-03	The application shall not require or implement any Bluetooth, NFC, USB, or other device-to-device communication protocol.

2.14 Functional Requirements

Table 2.7 Functional Requirements

ID	Requirement	Implementation Reference
FR-01	The system shall collect TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE sensor data continuously during a trip at a minimum rate of 50 Hz	SensorService.onCreate(), SENSOR_DELAY_GAME
FR-02	The system shall operate as an Android Foreground Service with a visible persistent notification (foregroundServiceType = location dataSync), ensuring sensor collection is not interrupted by the OS	SensorService.startForeground() nd()
FR-03	The system shall accumulate sensor samples into fixed one-second windows of approximately 50 samples for processing	windowBuffer, WINDOW_DURATION_MS = 1000 ms
FR-04	The system shall acquire GPS speed, location, bearing, and distance through FusedLocationProviderClient at 1-second intervals with PRIORITY_HIGH_ACCURACY and 0.5 m minimum displacement, applying a low-pass speed filter ($\alpha = 0.4$)	LocationService, FusedLocationProviderClient
FR-05	The system shall apply a three-stage signal pre-processing pipeline to each window: spike rejection (3D magnitude > 15 m/s ²), three-point median filter, and five-point moving average	TeleDriveProcessor.extractFeatures()
FR-06	The system shall extract an eight-feature FeatureVector from each processed window: mean forward acceleration, peak forward acceleration, minimum forward acceleration, horizontal acceleration std, mean gyro magnitude, peak gyro magnitude, gyro jitter (gyroStd), and vertical-axis std (azStd)	TeleDriveProcessor.extractFeatures()
FR-07	In Rule-Based mode, the system shall detect harsh acceleration when peak forward acceleration exceeds a speed-adaptive threshold (2.5 m/s ² at high speed, 3.5 at medium, 4.5 at low), with peak > min × 1.2 (magnitude	SensorService.processWindow()

	dominance), $\text{std} > 1.0$, and GPS speed trend > 1.0 km/h over the five-window buffer	
FR-08	In Rule-Based mode, the system shall detect harsh braking when minimum forward acceleration falls below a speed-adaptive threshold (-2.0 m/s ² at high speed, -3.0 at medium, -4.0 at low), with $ \text{min} > \text{peak} \times 1.2$, std below a variance ceiling, and GPS speed trend < -3.0 km/h	SensorService.processWindow()
FR-09	In Rule-Based mode, the system shall detect unstable riding when horizontal acceleration std exceeds a dynamically adapted threshold (vibration baseline EMA + 1.0, floored at 1.8/2.5/4.5 by speed band) for ≥ 3 consecutive windows, with gyroStd > 0.22 rad/s and total energy > 1.2	SensorService.processWindow(), vibrationBaselineEMA
FR-10	The system shall enforce speed-specific minimum gates: 15 km/h for acceleration detection, 12 km/h for braking detection, 8 km/h for unstable detection	SensorService, minSpeedFor* constants
FR-11	The system shall validate detections through a pattern-persistence gate requiring $\geq 30\%$ of recent 100 ms mini-windows to corroborate the event type before confirmation	checkPatternPersistence()
FR-12	In AI Assist mode, the system shall classify each window using an on-device 1D-CNN TFLite model (50 timesteps $\times$ 9 features), producing one of four class labels with a softmax confidence score, subject to speed-aware safety filters and prediction buffer smoothing	MLAnalyzer, ModelHelper
FR-13	In Hybrid mode (default), the system shall run both rule and ML engines in parallel, with a confidence-gated consensus: agreement floors confidence at 85%; rule takes precedence for acceleration/braking; ML-only unstable requires $\geq 88\%$ confidence; rule-only unstable is suppressed when ML predicts Normal	SensorService, DetectionMode.HYBRID_MODE
FR-14	The system shall confirm events through a state machine requiring EVENT_CONFIRM_THRESHOLD (1) consecutive same-type detections to enter event state, and NORMAL_CONFIRM_THRESHOLD (2) consecutive normal windows to exit, with a stricter UI threshold (confirm + 1) to prevent display flicker	SensorService, state machine

FR-15	The system shall enforce EVENT_COOLDOWN of 2000 ms between consecutive detection updates and a same-event spam guard of 2000 ms	SensorService, EventDetector
FR-16	The system shall compute an eco score (0–100) using a logarithmic diminishing-returns penalty model with base penalties of 8 (acceleration), 10 (braking), and 9 (instability) points, scaled by normalised severity (0.7–1.0) and dampened by a $1/\sqrt{n}$ factor per event occurrence	EcoScoreEngine.processEvent()
FR-17	The system shall estimate fuel consumption by adjusting user-declared base mileage with per-event penalties (–0.4 km/L per harsh acceleration, –0.3 per harsh brake, –0.5 per instability), clamped to 30–70 km/L effective range	FuelEstimator
FR-18	The system shall calculate financial loss from cumulative fuel wastage at a configurable petrol price (default ₹105/L)	EcoScoreEngine.getFinancialLoss()
FR-19	The system shall capture a still JPEG photograph from the device rear camera through an invisible 1×1-pixel CameraX activity (CAPTURE_MODE_MINIMIZE_LATENCY, flash off) upon confirmation of any high-severity event, saved as event_<type>_<timestamp>.jpg	CameraControllerActivity, EvidenceManager
FR-20	The system shall automatically delete evidence images older than 24 hours through a WorkManager periodic task scheduled daily at 03:00, with battery-not-low constraint	EvidenceCleanupWorker
FR-21	The system shall present a ride summary screen at trip end showing eco score, financial loss, distance, average speed, fuel used, duration, event counts (acceleration, braking, instability), contextual driving insight, and most recent evidence photograph	RideSummaryActivity
FR-22	The system shall persist trip history on-device using SharedPreferences with Gson serialisation and allow the user to browse past trips with drill-down to detail	TripStorage, TripsActivity, TripDetailActivity
FR-23	The system shall display an aggregate analytics screen with a multi-trip eco score trend chart	AnalyticsActivity
FR-24	The system shall generate contextual riding insights — comparative (vs. last trip, $\geq 30\%$ event count change), trend	InsightEngine

	(vs. last 7 trips), and streak (consecutive smooth rides with score ≥ 85) — returning the top 3 ranked by severity	
FR-25	The system shall render an interactive Event Map through osmdroid with OpenStreetMap MAPNIK tiles, featuring: dual-layer route polyline (glow + inner stroke), colour-coded circle markers with event-type letters (A/B/U/S/E), pixel-based clustering to prevent overlap, and a sliding bottom sheet with per-event coaching text and ± 8 -second temporally matched evidence	EventMapActivity
FR-26	The system shall detect risk hotspots by clustering events from multiple trips within a 150-metre Haversine radius, requiring contributions from ≥ 2 distinct trips, and display them as a toggleable overlay through FAB	HotspotDetector, EventMapActivity
FR-27	The system shall compute route analytics (distance, total events, event density per km, maximum severity, smooth-ride flag) for the Event Map display	RouteAnalyticsHelper
FR-28	The system shall collect a one-time user and vehicle profile during first launch (name, vehicle type, engine displacement, expected mileage) and use it for fuel calibration throughout the application	OnboardingActivity, UserProfileManager
FR-29	The system shall broadcast real-time event state, speed, acceleration standard deviation, detection source, confidence score, and contextual tip to the live trip screen during an active trip	LiveDataBus, LiveTripActivity
FR-30	The system shall log raw 50 Hz sensor windows with rule-engine labels to CSV when ML_TRAINING_MODE is enabled, for offline model training data collection	MLTrainingLogger, WindowLabeler

2.15 Non-Functional Requirements

Table 2.8 Non-Functional Requirements

ID	Category	Requirement	Justification
NFR-01	Performance	Feature extraction and event detection shall complete within one second (one window period) on a mid-range Android device	Required for real-time detection; pipeline involves arithmetic and array operations on 50 samples with no blocking I/O
NFR-02	Performance	TFLite inference shall complete in under 50 ms per window	Model v4 is a lightweight 1D-CNN (3 Conv1D blocks); TFLite optimises operator kernels for ARM acceleration
NFR-03	Accuracy	The rule-based engine shall produce fewer than 15% false positives on typical urban road conditions	Achieved through speed-adaptive thresholds, sustained detection, GPS speed-trend validation, magnitude-dominance checks, pattern-persistence gating, and dynamic vibration baseline adaptation
NFR-04	Accuracy	The ML model shall achieve $\geq 90\%$ recall for harsh acceleration events and $\geq 92\%$ macro-average F1 across all classes	Confirmed through held-out test evaluation
NFR-05	Reliability	The Foreground Service shall remain active throughout the trip without OS termination	Achieved through START_STICKY, foreground notification, and foregroundServiceType declaration
NFR-06	Memory	Sensor buffer memory consumption shall remain constant regardless of trip duration	Window and rolling buffers are bounded by fixed-size limits
NFR-07	Battery	Sensor collection and processing shall not cause abnormal battery drain over a multi-hour trip	Uses efficient sensors, 1 Hz GPS updates, and no continuous network activity during the core pipeline
NFR-08	Privacy	No personal data, sensor data, or trip data shall leave the device during core pipeline operation	No network calls are initiated by detection, scoring, or storage components

NFR-09	Usability	The rider shall not be required to interact with the application between tapping START TRIP and STOP TRIP	All detection, scoring, evidence capture, insight generation, and storage are automated
NFR-10	Storage	Evidence images older than 24 hours shall be automatically purged without user intervention	Enforced by EvidenceCleanupWorker through WorkManager
NFR-11	Compatibility	The application shall function on Android 8.0 (API 26) and later versions through API 36	Confirmed in project build configuration
NFR-12	Robustness	Duplicate sensor events shall be discarded to prevent double-counting	Implemented through timestamp and previous-value comparison
NFR-13	Robustness	GPS signal loss mid-trip shall not crash the application or corrupt trip data	Detection pauses safely and resumes automatically when signal returns
NFR-14	Robustness	Camera permission denial shall not prevent event detection or scoring	Evidence capture fails gracefully without affecting core functions
NFR-15	Scalability	The system shall support storage of several thousand trip summaries without degradation	Current storage is adequate; Room/SQLite migration is identified for future versions

2.16 Performance Requirements

Table 2.9 Performance Requirements

ID	Requirement
PR-01	The complete processing pipeline (signal conditioning, feature extraction, classification, state machine update, scoring) shall complete within 200 ms per one-second window to avoid data loss or window overlap.
PR-02	TFLite model inference shall complete within 50 ms per invocation on a mid-range Android device (Snapdragon 600-series or equivalent).
PR-03	The system shall sustain continuous operation for a minimum of 60 minutes without memory leaks, ANR (Application Not Responding) events, or service termination under normal riding conditions.
PR-04	Sensor data acquisition shall maintain an effective sampling rate of 40–60 Hz (SENSOR_DELAY_GAME) for both accelerometer and gyroscope throughout a trip session.
PR-05	GPS location updates shall be received at a rate of at least one update per second under open-sky conditions.
PR-06	Evidence photograph capture shall complete within 3 seconds from event confirmation, including CameraX initialisation, capture, and file write.
PR-07	Trip summary generation (eco score computation, fuel estimation, insight generation, location resolution) shall complete within 5 seconds of trip termination.
PR-08	The Event Map shall render the route polyline and up to 50 event markers within 2 seconds of screen launch.
PR-09	Trip history loading (deserialisation and display of up to 100 trips) shall complete within 3 seconds.
PR-10	The application's total resident memory footprint during an active trip shall not exceed 150 MB, including the TFLite model, sensor buffers, and UI composition.

2.17 Design Constraints

Table 2.10 Design Constraints

ID	Constraint
DC-01	The application shall be developed entirely in Kotlin using Jetpack Compose for the UI layer, targeting Android 8.0 (API 26) as the minimum supported platform version.
DC-02	All sensor processing, ML inference, scoring, and data persistence shall execute on-device. No cloud backend or remote API shall be required for the core pipeline.
DC-03	The ML model shall be deployed as a TensorFlow Lite FlatBuffer file bundled within the APK assets directory. No runtime model download or over-the-air model updates shall be supported.
DC-04	Trip data shall be persisted using SharedPreferences with Gson JSON serialisation. No relational database (Room/SQLite) shall be used in the current version.
DC-05	Map rendering shall use osmdroid with OpenStreetMap MAPNIK tiles. No proprietary map SDK (Google Maps, Mapbox) shall be used.
DC-06	The camera subsystem shall use CameraX with an invisible 1×1-pixel preview to avoid disrupting the rider during active trips.
DC-07	The Foreground Service shall declare foregroundServiceType = location.

2.18 System Attributes

Reliability: The system shall operate continuously throughout a trip without crashes, data loss, or service interruption. The event-confirmation state machine with hysteresis prevents transient sensor noise from producing false event detections. Graceful degradation is implemented for GPS outage (speed defaults to zero, activating speed gates) and camera permission denial (event detection continues without evidence capture).

Availability: The system shall be available for trip recording at any time the device has sufficient battery and sensor hardware. No server uptime, network connectivity, or external service availability is required for core operation. The only network-dependent feature (map tile download) caches tiles locally after first use, making subsequent access independent of connectivity.

Maintainability: The system follows a layered pipeline architecture with clearly defined module boundaries. Each processing stage (signal conditioning, feature extraction, classification, scoring, evidence, persistence) is encapsulated in a dedicated class with a single responsibility. Detection mode switching is managed through the AnalyzerProvider singleton, allowing the Rule-Based and ML classification engines to be replaced or updated independently without modifying adjacent pipeline stages.

Portability: The application targets a broad Android version range (API 26 through API 36), covering approximately 95% of active Android devices. The only platform-specific dependencies are Android SensorManager, FusedLocationProviderClient, CameraX, and WorkManager — all of which are standard Android Jetpack or Play Services components available across the supported API range.

Usability: The primary user interaction model requires only four actions: start trip, optionally select detection mode, stop trip, and review results. All intermediate processing — data acquisition, signal conditioning, classification, scoring, evidence capture, and insight generation — is fully automated and invisible to the user during the trip.

2.19 Other Requirements

This section specifies safety and security requirements that apply to the TeleDrive application.

2.19.1 Safety Requirements

Table 2.11 Safety Requirements

ID	Requirement
SF-01	The application shall not require any rider interaction during an active trip. All detection, scoring, and evidence capture shall be fully automated to avoid rider distraction.
SF-02	The live trip screen shall use a high-contrast dark theme with large text elements to minimise visual attention required if the rider glances at the screen.
SF-03	Evidence capture shall use an invisible 1×1-pixel preview with flash disabled to prevent sudden light flashes that could distract the rider or other road users.
SF-04	The event-confirmation state machine shall suppress single-window false positives to prevent unnecessary alerts that could cause rider anxiety or distraction.
SF-05	The speed gate mechanism shall suppress all event detection below minimum speed thresholds (8–15 km/h depending on event type) to prevent false detections during stationary or low-speed manoeuvres such as parking, traffic signals, or slow-speed turns.
SF-06	The system shall display a safety disclaimer during onboarding advising riders to mount the phone securely and not to interact with the device while riding.

2.19.2 Security Requirements

Table 2.12 Security Requirements

ID	Requirement
SE-01	All trip data, user profile data, and evidence photographs shall be stored exclusively on the local device. No data shall be transmitted to any external server, cloud service, or third-party endpoint.
SE-02	Evidence photographs shall be stored in the application's private external files directory, accessible only to the TeleDrive application unless explicitly exported by the user.
SE-03	Evidence photographs older than 24 hours shall be automatically deleted by EvidenceCleanupWorker to limit the retention window of location-correlated imagery.
SE-04	The application shall request only the minimum permissions required for operation: ACCESS_FINE_LOCATION, CAMERA, and POST_NOTIFICATIONS (API 33+). No contacts, microphone, phone state, or storage permissions beyond the application's own directory shall be requested.
SE-05	GPS location data shall be used only for speed calculation, distance accumulation, event geolocation, and reverse geocoding. Location data shall not be shared with any external entity.
SE-06	The user profile (name, vehicle details, mileage) shall be stored in SharedPreferences within the application's private data directory, inaccessible to other applications under standard Android sandboxing.
SE-07	The application shall not implement any remote access, remote control, or remote data retrieval capability. All data review occurs locally on the device by the rider.

CHAPTER 3 SYSTEM DESIGN

3.1 Introduction

This chapter presents the structured system design of TeleDrive using standard software engineering modelling techniques. The primary objective is to define how data originates from external sources, flows through the system's internal processing stages, and ultimately reaches the user as meaningful output. Two complementary modelling notations are employed: Data Flow Diagrams (DFDs) and an Entity-Relationship (ER) Diagram.

Data Flow Diagrams decompose the system into progressively finer levels of detail. A Level 0 context diagram establishes the system boundary and identifies all external entities that interact with TeleDrive. A Level 1 DFD expands the single system process into its constituent functional modules and traces every data flow between them. Module-level descriptions define the responsibility, input, output, and transformation logic of each process. Together, these diagrams provide a complete picture of data movement through the system without prescribing implementation detail — that responsibility belongs to the architecture and implementation chapters that follow.

The Entity-Relationship (ER) diagram represents the logical data model of the TeleDrive system, identifying the core data entities, their attributes, and the relationships and cardinality constraints between them. Although TeleDrive does not use a traditional relational database, the ER diagram serves as a conceptual design tool to validate data structure, consistency, and interaction across system modules.

In the implemented system, structured data such as trip history, event logs, and user preferences are persisted locally using Android SharedPreferences with Gson-based JSON serialisation, while captured evidence images are stored directly in the device file system. Therefore, the ER diagram reflects the logical organisation of data rather than a physical database schema, ensuring clarity in design and maintainability of the system.

All process names, data flow contents, data store structures, and entity attributes documented in this chapter are derived directly from the implemented Kotlin source code. They are traceable to the module reference presented in Chapter 5.

3.2 Data Flow Diagram — Level 0 (Context Diagram)

A Level 0 Data Flow Diagram, also known as a context diagram, represents the entire system as a single process and identifies all external entities that produce data consumed by the system or consume data produced by it. The context diagram establishes the system boundary — everything inside the central process is within scope; everything outside it is an external actor or resource.

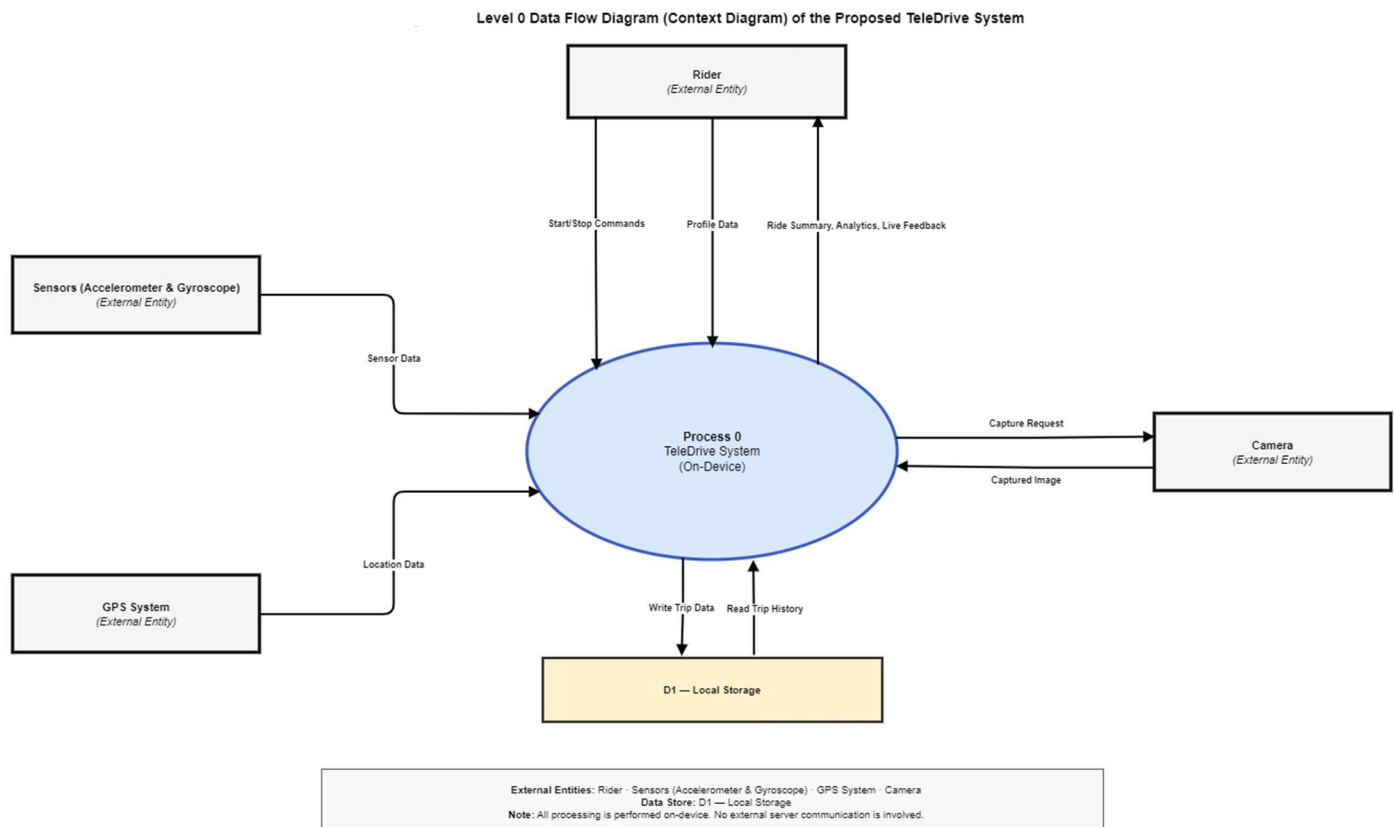


Figure 3.1 Level 0 Data Flow Diagram (Context Diagram) of the TeleDrive System

3.2.1 External Entities

The following four external entities interact with the TeleDrive system:

Table 3.1 External Entities

External Entity	Type	Role
Rider	Human actor	Initiates and terminates trips, selects detection mode, provides profile information, and reviews trip results, analytics, and evidence
Sensors (Accelerometer & Gyroscope)	Device hardware	Provide three-axis linear acceleration (ax, ay, az) and three-axis angular velocity (gx, gy, gz) at approximately 50 Hz via Android SensorManager
GPS System	Device hardware	Provides latitude, longitude, speed, bearing, and altitude at approximately 1 Hz via Google Play Services FusedLocationProviderClient
Camera	Device hardware	Captures JPEG still photographs on command from the system when an unsafe riding event is confirmed

3.2.2 Data Store

Table 3.2 Data Store

Store	ID	Description
Local Storage	D1	SharedPreferences (JSON via Gson) for TripSummary and UserProfile records; filesystem directory for evidence JPEG photographs

3.2.3 Data Flows

Table 3.3 Data Flows

From	To	Flow Name	Content
Rider	TeleDrive System	Start/Stop Commands	Trip start, trip stop, detection mode selection
Rider	TeleDrive System	Profile Data	Rider name, vehicle name, engine displacement, declared mileage
Sensors	TeleDrive System	Sensor Data	Raw ax, ay, az, gx, gy, gz samples at ~50 Hz
GPS System	TeleDrive System	Location Data	Speed, bearing, latitude, longitude at ~1 Hz
TeleDrive System	Camera	Capture Request	Event type identifier and target file path
Camera	TeleDrive System	Captured Image	JPEG photograph file
TeleDrive System	D1	Write Trip Data	Serialised TripSummary with nested TripEvent list; UserProfile JSON; evidence JPEG files
D1	TeleDrive System	Read Trip History	Previously stored trip summaries, user profile, evidence file listings
TeleDrive System	Rider	Ride Summary, Analytics, Live Feedback	Eco score, fuel consumed, financial loss, event counts, insights, coaching, Event Map, evidence gallery, real-time event state

All processing is performed on-device. No external server communication is involved.

3.3 Data Flow Diagram (DFD) Overview

TeleDrive's system design is documented using two structured notelling techniques. Data Flow Diagrams trace how sensor readings, GPS samples, and user actions travel through the detection pipeline — from hardware acquisition through signal conditioning, event classification, scoring, and final display. The ER diagram captures the three persistent data objects (UserProfile, TripSummary, EvidenceItem) and their logical associations, which are stored in SharedPreferences using Gson serialisation rather than in a relational database.

DFDs are independent of implementation technology; the same diagram applies regardless of whether the system is built in Kotlin, Java, or any other language.

DFDs are constructed at multiple levels of abstraction:

Level 0 (Context Diagram): The entire system is represented as a single process. All external entities and their data flows into and out of the system are identified. No internal structure is visible. This level answers the question: “What does the system interact with?”

Level 1 (Detailed DFD): The single system process is decomposed into its major functional processes. Data flows between processes, between processes and data stores, and between processes and external entities are fully enumerated. This level answers the question: “What are the major processing stages and how does data move between them?”

Level 2 and Below (Module-Level): Individual Level 1 processes are further decomposed into sub-processes. In the case of TeleDrive, Level 1 provides sufficient detail for the system's scope, and module-level descriptions in the process descriptions that follow, supplement the Level 1 diagram with narrative process specifications rather than additional graphical decomposition.

The DFD notation used in this document follows the Yourdon-DeMarco convention, which is the most widely adopted notation in academic software engineering.

3.4 Rules Regarding DFD Constraints

The following standard rules govern the construction and validation of the Data Flow Diagrams presented in this chapter. Every process must have at least one input and one output; no data may flow directly between external entities or directly between data stores, and a process must mediate all interactions with data stores. Process labels are intended to describe transformations, while data-flow labels identify the data being transferred. The Level 0 and Level 1 diagrams presented in Figure 3.1 and Figure 3.2, together with the accompanying process descriptions in this chapter, were checked against these constraints to ensure balanced and internally consistent representation of the implemented system.

Conservation of Data Flows: Every data flow entering a process must be accounted for in the process's output. No data may be silently consumed or created without a corresponding input or output flow.

No Direct Flow Between External Entities: Data cannot flow directly from one external entity to another. All data must pass through at least one process within the system boundary.

No Direct Flow Between Data Stores: Data cannot move directly from one data store to another. A process must mediate every data store interaction.

No Data Flow Directly from an External Entity to a Data Store: An external entity cannot read from or write to a data store without an intervening process.

Every Process Must Have at Least One Input and One Output: A process with only inputs is a "black hole" (data enters but never leaves). A process with only outputs is a "miracle" (data is produced from nothing). Both are design errors.

Processes Are Named with Verb Phrases: Process labels describe the transformation being performed (e.g., "Signal Processing & Feature Extraction") rather than the module or component name.

Data Flows Are Named with Noun Phrases: Flow labels describe the data content (e.g., "FeatureVector," "Confirmed Event") rather than the action being performed on the data.

Balancing Between Levels: The data flows entering and leaving a parent process at Level 0 must match the aggregate data flows entering and leaving the child processes at Level 1. No new external entities or flows may appear at a lower level without corresponding representation at the higher level.

Unique Naming: Every process, data store, and data flow carries a unique label. No two elements share the same name within a diagram.

All three representations in this chapter (Figure 3.1 at Level 0, Figure 3.2 at Level 1, and the module-level descriptions in this chapter) have been validated against these rules.

3.5 DFD Symbols

The following table defines the standard Yourdon-DeMarco notation symbols used in Figures 3.1 and 3.2.

Table 3.4 DFD Symbols

Symbol	Shape	Name	Description
□	Rectangle	External Entity	A source or destination of data that exists outside the system boundary. External entities are not controlled by the system. In TeleDrive: Rider, Sensors, GPS System, Camera.
○	Circle (or rounded rectangle)	Process	A function that transforms input data into output data. Each process is labelled with a unique identifier and a verb-phrase name. In TeleDrive: Sensor Data Acquisition, Signal Processing, Event Classification, etc.
═	Open-ended rectangle (two parallel lines)	Data Store	A repository where data is held for later retrieval. Data stores are labelled with a "D" prefix and a unique number. In TeleDrive: D0 (Sensor Window Buffer), D1 (Trip Data Store), D2 (Evidence Storage).
→	Arrow	Data Flow	A pipeline through which data of known composition moves from a source to a destination. The arrow is labelled with a noun phrase describing the data content.

3.6 Data Flow Diagram — Level 1 (Detailed DFD)

Figure 3.2 presents the Level 1 Data Flow Diagram of TeleDrive. It decomposes the single system process shown in Figure 3.1 into the major functional processes responsible for sensor acquisition, signal processing, classification, event validation, scoring, evidence capture, persistence, analytics, and user-facing output.

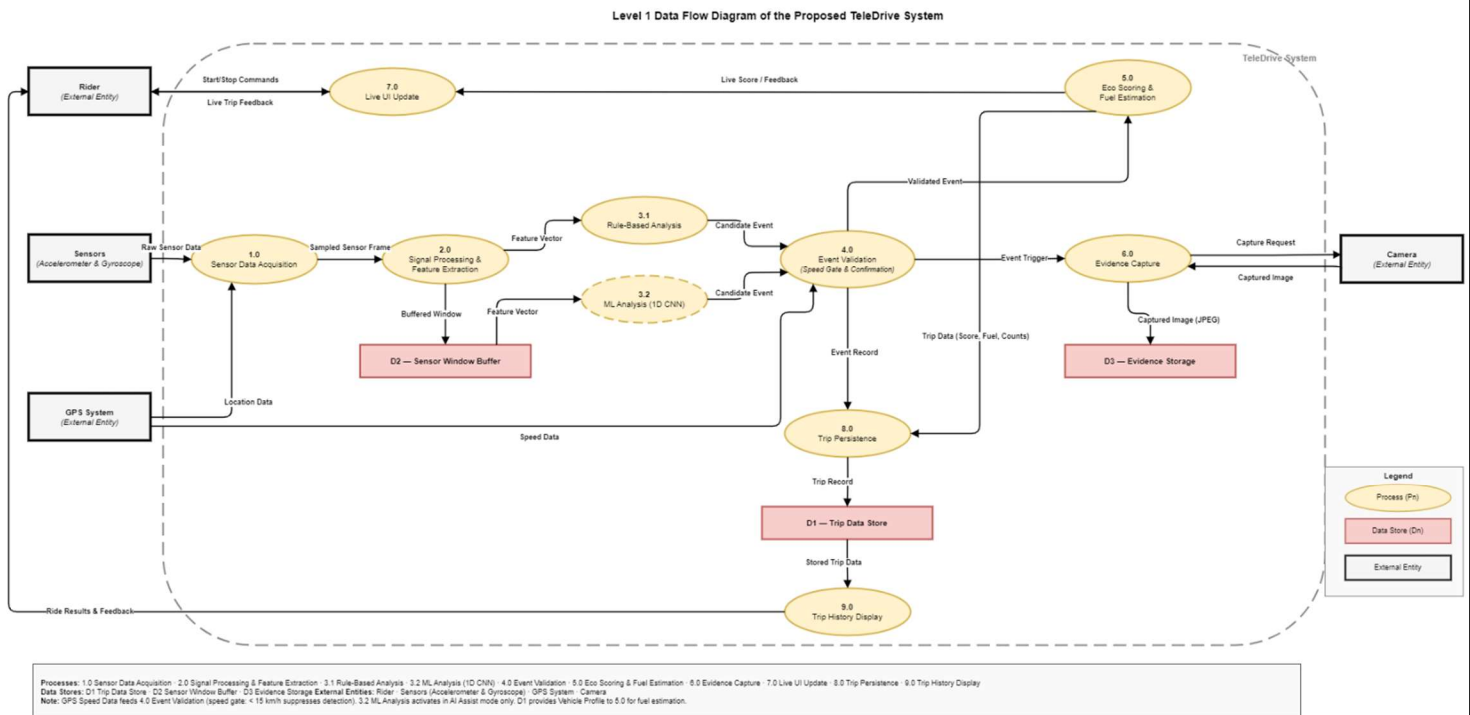


Figure 3.2 Level 1 Data Flow Diagram of the TeleDrive System

3.6.1 Process Summary

Table 3.5 Process Summary

Process ID	Process Name	Description
1.1	Sensor Data Acquisition	Receives raw sensor callbacks and GPS updates; accumulates samples into one-second windows
2.0	Signal Processing & Feature Extraction	Applies spike rejection, median filtering, and moving average; extracts eight statistical features
3.1	Rule-Based Analysis	Classifies events using speed-adaptive physics-derived thresholds
3.2	ML Analysis (1D CNN)	Classifies events using TensorFlow Lite convolutional neural network inference
4.1	Event Validation / State & Confirmation	Applies event-confirmation state machine with hysteresis and cooldown enforcement
5.0	Eco Scoring & Fuel Estimation	Computes eco score, fuel consumption, and financial loss from confirmed events
6.0	Evidence Capture	Triggers and manages photographic evidence capture for confirmed events
7.0	Trip Persistence	Serialises and stores trip summaries, user profiles, and location cache
8.0	Trip History Display	Retrieves and presents historical trip data and analytics to the rider

3.6.2 Data Stores

Table 3.6 Data Stores

Store ID	Name	Content
D0	Sensor Window Buffer	Temporary in-memory buffer of ~50 SensorSample objects per one-second window
D1	Trip Data Store	SharedPreferences JSON containing TripSummary records with nested TripEvent lists, UserProfile, location cache, and aggregate score
D2	Evidence Storage	Filesystem directory containing JPEG evidence photographs with event-type-encoded filenames

The subsections that follow describe each process in detail.

3.6.3 Sensor Data Acquisition Module

Process ID: 1.1

Input:

- Raw accelerometer callbacks (ax, ay, az) at ~50 Hz from the Sensors external entity
- Raw gyroscope callbacks (gx, gy, gz) at ~50 Hz from the Sensors external entity
- GPS location updates (speed, bearing, latitude, longitude) at ~1 Hz from the GPS System external entity

Processing:

On each TYPE_LINEAR_ACCELERATION callback, the most recent gyroscope values and GPS speed and heading are fused into a SensorSample data structure containing nine fields: timestamp, ax, ay, az, gx, gy, gz, heading, and speed. A duplicate-guard mechanism discards any accelerometer sample whose values match the previous reading within a 10-millisecond window. Valid samples are accumulated into the Sensor Window Buffer (D0). When 1000 milliseconds of elaps

ed time have passed since the current window started, the buffer is dispatched as a complete one-second window to the Signal Processing module and cleared.

Output:

- One-second sensor window: a list of approximately 50 SensorSample objects written to D0 and dispatched to Process 2.0
- Individual SensorSample objects fed to ML Analysis (Process 3.2) for its rolling 50-sample buffer
- Current GPS speed forwarded to Event Validation (Process 4.1) for speed-gate checks

Connected Data Stores: D0 (Sensor Window Buffer — write)

3.6.4 Signal Processing and Feature Extraction Module

Process ID: 2.0

Input:

- One-second sensor window (list of ~50 SensorSample objects) from Process 1.1

Processing:

The window passes through a three-stage signal-conditioning pipeline. Stage 1 (spike rejection): any sample whose three-axis acceleration magnitude exceeds 15 m/s^2 is discarded, eliminating impulse artefacts from road surface impacts. Stage 2 (median filter): a three-point sliding median filter is applied to the forward acceleration axis (ax), removing isolated impulse noise without blurring real signal transitions. Stage 3 (moving average): a five-point moving average attenuates residual lower-frequency noise. After filtering, eight statistical features are computed and assembled into a FeatureVector: mean forward acceleration, peak forward acceleration, minimum forward acceleration, horizontal acceleration standard deviation, mean gyroscope magnitude, peak gyroscope magnitude, gyroscope jitter (standard deviation of per-sample gyroscope magnitude), and vertical-axis standard deviation (road-roughness indicator).

Output:

- FeatureVector (eight statistical descriptors) forwarded to Process 3.1 (Rule-Based Analysis) and Process 3.2 (ML Analysis)

Connected Data Stores: None (stateless transformation)

3.6.5 Event Classification Module

Process IDs: 3.1 (Rule-Based Analysis) and 3.2 (ML Analysis)

Input:

- FeatureVector from Process 2.0
- Current GPS speed from Process 1.1
- Raw SensorSample objects from Process 1.1 (for ML buffer population)
- Current detection mode setting from the Rider (via UI)

Processing:

The FeatureVector is routed through the active detection mode. In Rule-Based mode (Process 3.1), speed-adaptive thresholds are applied: harsh acceleration requires peak forward acceleration $>2.5 \text{ m/s}^2$ at speed $\geq 15 \text{ km/h}$; harsh braking requires minimum forward acceleration $<-3.5 \text{ m/s}^2$ at speed $\geq 12 \text{ km/h}$; unstable riding requires acceleration standard deviation >3.0 or mean gyroscope >2.0 at speed $\geq 8 \text{ km/h}$. Additional checks include magnitude-dominance validation (20% margin), variance-window filtering, GPS speed-trend analysis, and pattern-persistence gating ($\geq 30\%$ mini-window corroboration).

In AI Assist mode (Process 3.2), a rolling buffer of the 50 most recent SensorSample objects is used to construct a (1, 50, 9) input tensor. The nine features per timestep are: ax, ay, az, gx, gy, gz, acceleration magnitude, jerk magnitude, and GPS speed. Each feature is normalised using pre-computed mean and standard deviation values. The normalised tensor is passed to a one-dimension

al Convolutional Neural Network via TensorFlow Lite inference. The class with the highest softmax probability is selected.

In Hybrid mode (default), both engines execute in parallel. The agreement between the two classifiers floors confidence at 85%. Disagreement is resolved by a physics-priority protocol: the rule engine prevails for directional events (acceleration, braking); the ML engine governs unstable-ride suppression with an 88% confidence gate.

Output:

- Candidate DrivingEvent (type, severity, confidence, source) forwarded to Process 4.1

Connected Data Stores: None

3.6.6 Event Validation and Confirmation Module

Process ID: 4.1

Input:

- Candidate DrivingEvent from Process 3.1 or 3.2
- Current GPS speed from Process 1.1

Processing:

All classification outputs pass through an event-confirmation state machine with hysteresis. A non-NORMAL event must persist for at least one consecutive detection window (`EVENT_CONFIRM_THRESHOLD = 1`) before being promoted to a confirmed event. Once confirmed, the event state is held until at least two consecutive NORMAL windows are observed (`NORMAL_CONFIRM_THRESHOLD = 2`), preventing premature state clearing and UI flicker. A 2-second event cooldown and a 4-second scoring cooldown are enforced between consecutive scored events to prevent rapid-fire duplicate detections. Speed-gate validation suppresses all events below the minimum speed thresholds (15 km/h for acceleration, 12 km/h for braking, 8 km/h for instability).

Output:

- Confirmed DrivingEvent forwarded to Process 5.0 (Eco Scoring) and Process 6.0 (Evidence Capture)
- Real-time event state, speed, confidence, detection source, and coaching tip forwarded to the UI (Live Feedback to Rider)

Connected Data Stores: None (stateful in-memory only)

3.6.7 Eco Scoring and Fuel Estimation Module

Process ID: 5.0

Input:

- Confirmed DrivingEvent (type, severity) from Process 4.1
- Trip distance from Process 1.1 (GPS-derived cumulative distance)
- Rider's declared base mileage from UserProfile (read from D1)

Processing:

The eco score begins at 100 at trip start and is decremented by a penalty per confirmed event. The penalty follows a diminishing-penalty model: $\text{penalty} = [\text{basePenalty} \times \text{normalizedSeverity} \times (1/\sqrt{n})]$, where basePenalty is 8 for harsh acceleration, 10 for harsh braking, and 9 for instability; normalizedSeverity is the event severity divided by 10, clamped to the range [0.7, 1.0]; and n is the cumulative count of events of that specific type. The square-root decay ensures that the first event of each type incurs the full penalty while subsequent events of the same type apply progressively smaller deductions.

The fuel consumption is estimated by adjusting the rider's declared base mileage with per-event degradation penalties: -0.4 km/L per harsh acceleration, -0.3 km/L per harsh brake, and -0.5 km/L per instability event. The resulting effective mileage is clamped to a plausible range of 30–70 km/L. Total fuel consumed is computed as $\text{distance (km)} \div \text{effective mileage (km/L)}$. Financial 1

loss is computed as cumulative fuel wastage multiplied by the configurable petrol price (default ₹ 105/L).

Output:

- Updated eco score (0–100) forwarded to UI (Live Feedback)
- Final score, fuel consumed, and financial loss forwarded to Process 7.0 (Trip Persistence) at trip end

Connected Data Stores: D1 (reads UserProfile for base mileage)

3.6.8 Evidence Capture Module

Process ID: 6.0

Input:

- Event trigger from Process 4.1: event type and timestamp for each confirmed non-NORMAL event, subject to a 3-second camera cooldown

Processing:

When a confirmed non-NORMAL event is received and the cooldown period has elapsed, the system launches an invisible 1×1-pixel camera activity that captures a JPEG photograph from the device rear camera with minimum-latency capture mode and flash disabled. The photograph is saved to the Evidence Storage (D2) with a filename encoding the event type and timestamp in the format `evidence/event_<typeSlug>_<timestamp>.jpg`.

A periodic cleanup worker executes every 24 hours (targeting 03:00 local time) and deletes all evidence files older than 24 hours to manage storage consumption. The evidence manager provides file scanning (newest-first, capped at 50 items for display), individual deletion, and export to the device public gallery.

Output:

- JPEG evidence file written to D2 (Evidence Storage)
- Evidence file listings provided to Process 8.0 (Trip History Display) for the evidence gallery and Event Map bottom sheet

Connected Data Stores: D2 (Evidence Storage — write and read)

3.6.9 Trip Persistence Module

Process ID: 7.0

Input:

- Completed TripSummary object containing eco score, distance, duration, average speed, event counts by type, nested list of TripEvent objects (each with type, latitude, longitude, timestamp, severity, speed), and a behavioural insight tip — from Processes 1.1, 4.1, and 5.0
- UserProfile data from the Rider (via UI): name, vehicle name, engine displacement, declared mileage
- Reverse-geocoded location names for trip start and end coordinates

Processing:

The TripSummary object is serialised to JSON using Gson and appended to the SharedPreferences trip history array stored in D1. The complete list is re-serialised on each save. The UserProfile is stored as a singleton JSON object in a separate SharedPreferences key. A location cache stores resolved human-readable place names to avoid redundant geocoding requests. Trip retrieval supports sorted access (most recent first), indexed lookup by UUID, and deletion by UUID. An aggregate average eco score is computed and cached for display on the analytics screen.

Output:

- Persisted JSON records written to D1 (Trip Data Store)
- Human-readable location names appended to TripSummary before persistence

Connected Data Stores: D1 (Trip Data Store — write and read)

3.6.10 Intelligence and Analytics Module

Input:

- Current TripSummary from Process 7.0
- List of historical TripSummary objects from D1
- Rider's declared mileage from UserProfile in D1

Processing:

The insight engine evaluates four analytical dimensions against the completed trip: comparative analysis (current trip versus previous trip), trend analysis (current versus average of last seven trips), streak analysis (consecutive trips scoring 85 or above), and milestone detection. Each generated insight carries a category (acceleration, braking, stability, fuel efficiency, overall improvement, streak, or milestone), a severity level (positive, neutral, warning, or critical), and an improvement suggestion. Insights are ranked by severity and the top three are returned for display.

The hotspot detector clusters TripEvent objects from all stored trips spatially using the Haversine distance formula. Two events are considered co-located if they fall within a 150-metre radius. A cluster is promoted to a risk hotspot only if it contains events contributed by at least two distinct trips, preventing a single bad ride from producing a false hotspot. Each hotspot records the geographic centre, dominant event type, total event count, and contributing trip count.

The route analytics helper computes per-trip metrics: total distance, total events, event density per kilometre, maximum event severity, and a smooth-ride boolean flag.

Output:

- Top three TripInsight objects forwarded to the UI (Ride Summary screen)
- List of RiskHotspot objects forwarded to the UI (Event Map hotspot overlay)
- RouteAnalytics metrics forwarded to the UI (Event Map header)

Connected Data Stores: D1 (Trip Data Store — read)

3.6.11 User Interface and Trip Display Module

Process IDs: 1.0 (Live UI) and 8.0 (Trip History Display)

Input:

- Real-time event state, speed, eco score, confidence, detection source, and coaching tip from Process 4.1 (via callback)
- Scoring results from Process 5.0
- Intelligence and analytics outputs from the Intelligence module
- Trip summaries and user profile from D1 (via Process 7.0)
- Evidence file listings from D2 (via Process 6.0)

Processing:

The user interface layer renders structured data received from all lower-layer processes. No computation, classification, scoring, or persistence is performed in this module. During an active trip, the live trip screen subscribes to a singleton callback that delivers event label, speed, standard deviation, coaching tip, confidence, and detection source at each window cycle. At trip end, the ride summary screen receives the completed trip results through Android Intent extras. The trip history screen reads all stored trip summaries from D1 and presents them sorted by recency. The Event Map screen renders the trip route with event markers, per-event coaching, temporally matched

evidence, and hotspot overlay. The analytics screen computes a trend chart from the last seven trips. The evidence gallery screen reads evidence files from D2.

Output:

- Visual screens presented to the Rider: main dashboard, live trip monitor, ride summary, trip history, trip detail, Event Map, analytics, profile, evidence gallery, and onboarding

Connected Data Stores: D1 (Trip Data Store — read), D2 (Evidence Storage — read)

3.7 Process Descriptions

Table 3.7 Process Descriptions

Process ID	Process Name	Input	Output
P1	Sensor Acquisition	Raw hardware interrupts at ~50 Hz	SensorSample struct
P2	Signal Pre-processing	List of 50 SensorSample objects	FeatureVector (8 features)
P3	Behaviour Analysis	FeatureVector, GPS speed	DrivingEvent (type, severity, confidence, source)
P4	Event Validation	DrivingEvent, rolling window buffers	Confirmed DrivingEvent or NORMAL
P5	Scoring	Confirmed DrivingEvent	Updated eco score, fuel estimate
P6	Evidence Capture	HIGH severity event trigger	JPEG file saved to D3
P7	Live UI Update	Event state, speed, tip	LiveDataBus broadcast to UI thread
P8	Persistence	Trip summary data on trip end	TripSummary written to D1
P9	UI Presentation	Stored TripSummary, EvidenceItem list	Rendered screens for user

CHAPTER 4 DATABASE DESIGN

4.1 Introduction

The TeleDrive system does not use a relational database. Trip history and user profile data are persisted using Android SharedPreferences with Gson JSON serialisation. Evidence photographs are stored directly on the device filesystem. Despite the absence of a formal database, a logical data model is presented using ER notation to document the system's persistent data structures, their attributes, and the relationships between them. This model serves as a design reference and enables verification that all required data elements are captured and correctly associated.

Three persistent data entities exist in the system: UserProfile, TripSummary, and EvidenceItem. Their relationships, attributes, and cardinality constraints are described in the subsections that follow.

4.2 ER Diagram

Figure 4.1 presents the Entity-Relationship Diagram for the TeleDrive system, illustrating the principal persistent data objects, their attributes, and the logical 1:N relationships used for documentation and design verification.

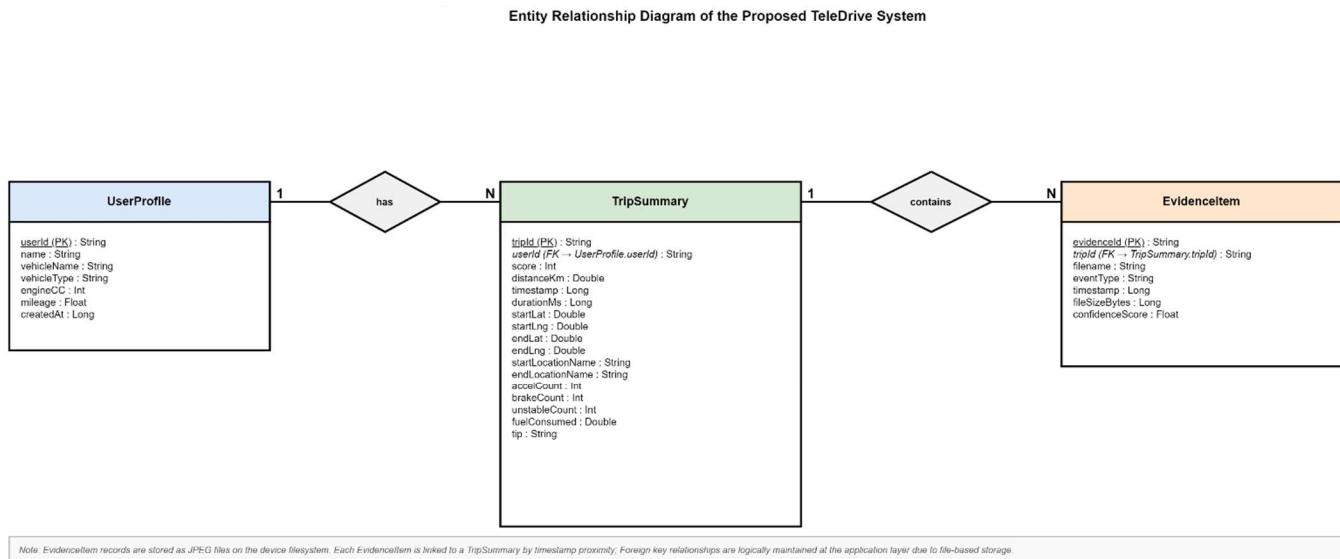


Figure 4.1 Entity Relationship Diagram of the TeleDrive System

The diagram shows UserProfile as a singleton entity connected to TripSummary through a one-to-many "has" relationship. TripSummary is connected to EvidenceItem through a one-to-many "contains" relationship. UserProfile attributes include name, vehicleName, engineCC, mileage, and createdAt. TripSummary attributes include the trip identifier, eco score, distance, timestamps, GPS coordinates, location names, event counts, the nested event list, and the behavioural tip. EvidenceItem attributes include the file reference, event type, and timestamp — all derived from the filesystem rather than from a database record.

Note: EvidenceItem records are stored as JPEG files on the device filesystem. Each EvidenceItem is linked to a TripSummary by timestamp proximity. Foreign key relationships are logically maintained at the application layer due to file-based storage.

Relationships:

UserProfile —[1 : N]— TripSummary

One user profile calibrates fuel estimation for all recorded trips.

TripSummary —[1 : N]— EvidenceItem

One trip may be associated with zero or more evidence images.

Association is by timestamp proximity — EvidenceItem has no DB row

4.2.1 Data Objects

Table 4.1 Data Objects

Data Object	Description
UserProfile	Represents the rider's identity and vehicle configuration. Stored as a singleton — only one profile exists per device at any time. This object is created during the onboarding flow and may be edited through the profile screen. The declared mileage value from this object calibrates the fuel estimation model for all subsequent trips.
TripSummary	Represents the complete record of a single completed trip. Each trip produces exactly one TripSummary instance, which contains the eco score, distance, duration, event counts, GPS coordinates, location names, a behavioural insight, and a nested list of TripEvent objects recording the individual events detected during that trip. TripSummary objects accumulate over time as the rider completes more trips.
EvidenceItem	Represents a single captured evidence photograph. Unlike UserProfile and TripSummary, EvidenceItem records are not stored in SharedPreferences. The evidence file's filesystem metadata (filename, path, modification timestamp) is the ground truth. Each evidence file's name encodes the event type and capture timestamp, enabling the system to associate evidence items with specific trip events by temporal proximity. Evidence items are ephemeral: a cleanup worker deletes all files older than 24 hours.

4.2.2 Attributes

Table 4.2 UserProfile Attributes

Attribute	Data Type	Constraint	Description
name	String	Required	Rider's display name
vehicleName	String	Required	Vehicle model name (e.g., "Honda Activa 6G")
engineCC	Int	Required	Engine displacement in cubic centimetres
mileage	Float	Required	Rider's declared real-world fuel mileage in km/L; used to calibrate FuelEstimator
createdAt	Long	Auto-generated	Epoch timestamp of profile creation

Table 4.3 TripSummary Attributes

Attribute	Data Type	Constraint	Description
id	String	Primary key (UUID)	Unique identifier for the trip, generated automatically
score	Int	Required, 0–100	Final eco score computed by the scoring engine
distanceKm	Double	Required	Total GPS-derived distance travelled in kilometres
timestamp	Long	Auto-generated	Epoch timestamp of trip completion
durationMs	Long	Default 0	Trip duration in milliseconds
startLat	Double	Default 0.0	Latitude of trip start point
startLng	Double	Default 0.0	Longitude of trip start point
endLat	Double	Default 0.0	Latitude of trip end point
endLng	Double	Default 0.0	Longitude of trip end point
startLocationName	String?	Nullable	Reverse-geocoded human-readable name of trip start location
endLocationName	String?	Nullable	Reverse-geocoded human-readable name of trip end location
accelCount	Int	Default 0	Number of confirmed harsh acceleration events
brakeCount	Int	Default 0	Number of confirmed harsh braking events
unstableCount	Int	Default 0	Number of confirmed unstable riding events
events	List?	Nullable	Ordered list of individual event records with location, severity, and speed
tip	String?	Nullable	Behavioural insight or coaching suggestion for this trip

Table 4.4 TripEvent Attributes (nested within TripSummary)

Attribute	Data Type	Description
type	String	Event classification: "HARSH_ACCELERATION", "HARSH_BRAKING", or "UNSTABLE_RIDE"
latitude	Double	GPS latitude at the moment of event detection
longitude	Double	GPS longitude at the moment of event detection
timestamp	Long	Epoch timestamp of event detection
severity	Float	Magnitude of the detected event (higher values indicate more aggressive behaviour)
speedKmh	Float	GPS-derived speed in km/h at the moment of event detection

Table 4.5 EvidenceItem Attributes

Attribute	Data Type	Source	Description
file	File	Filesystem	Reference to the JPEG photograph file stored in the evidence/ directory
eventType	String	Parsed from filename	Event type slug extracted from the filename: "accel", "brake", "unstable", or "event"
timestamp	Long	Parsed from filename	Epoch timestamp extracted from the filename; used for temporal matching with TripEvent records (± 8 -second window)

4.2.3 Relationships

Two relationships exist in the TeleDrive data model:

Relationship 1: UserProfile — TripSummary

UserProfile has TripSummary. One user profile calibrates the fuel estimation model for all recorded trips. Because TeleDrive is a single-user application with one profile per device, every TripSummary in the system is implicitly associated with the single UserProfile instance. This relationship is logical rather than enforced by a foreign key, since both entities are stored in separate SharedPreferences keys.

Relationship 2: TripSummary — EvidenceItem

TripSummary contains EvidenceItem. One trip may produce zero or more evidence photographs, depending on how many non-NORMAL events were confirmed during the trip and whether the camera cooldown permitted capture. The association between a TripSummary and its EvidenceItem records is established by timestamp proximity: an evidence photograph is matched to a trip event if the evidence timestamp falls within ± 8 seconds of the event timestamp. This temporal association is resolved at display time (on the Event Map bottom sheet) rather than stored as a persistent foreign key.

4.2.4 Cardinality

Table 4.6 Cardinality

Relationship	Entity A	Cardinality	Entity B	Minimum	Maximum
has	UserProfile	1 : N	TripSummary	One UserProfile exists per device	One UserProfile is associated with zero to many TripSummary records
contains	TripSummary	1 : N	EvidenceItem	One TripSummary exists per completed trip	One TripSummary is associated with zero to many EvidenceItem records

Participation Constraints:

UserProfile participation is total on the UserProfile side (the profile must exist before any trip can be recorded, as it provides the base mileage for fuel estimation) and partial on the TripSummary side (a user profile may exist with zero trips if no trips have been completed yet).

TripSummary participation is total on the TripSummary side (every trip exists as a TripSummary) and partial on the EvidenceItem side (a trip may have zero evidence items if all events were suppressed by cooldown, if camera permission was denied, or if the trip had no detected events).

4.3 Data Models and Interfaces

Table 4.7 Data Models and Interfaces

No.	Module / File Name	Type	Purpose / Responsibility
1	SensorSample.kt	Data Model	Represents one sensor reading with motion and GPS values.
2	FeatureVector.kt	Data Model	Stores extracted features for classification.
3	DrivingAnalyzer.kt	Interface	Common contract for all analyzer modules.
4	DrivingEvent.kt	Data Model	Represents one detected ride event.
5	DrivingEventType.kt	Enum	Defines supported event categories.
6	TripSummary.kt	Data Model	Stores completed trip results and summary metrics.
7	TripEvent.kt	Data Model	Stores timestamped event details inside a trip.
8	UserProfile.kt	Data Model	Stores rider and vehicle information.
9	EvidenceItem.kt	Data Model	Represents one evidence image and metadata.
10	TripInsight.kt	Data Model	Stores generated coaching insight content.
11	RiskHotspot.kt	Data Model	Stores hotspot cluster analytics results.

CHAPTER 5 DETAILED DESIGN

5.1 System Context

TeleDrive is a standalone Android mobile application designed to analyse two-wheeler riding behaviour in real time using only the sensors already present in the rider's smartphone. The system requires no external hardware adapters, no vehicle modifications, no cloud backend, and no internet connectivity for its core detection, scoring, and data storage functions. Once a trip is started, the application operates autonomously as a background-persistent service — acquiring motion and location data, classifying riding events, computing performance metrics, capturing photographic evidence, and storing results locally — until the rider ends the trip. The rider then receives a comprehensive summary including an eco score, fuel and financial impact estimates, behavioural insights drawn from trip history, and an interactive map displaying the route, detected events, and recurring risk zones.

The system is built entirely in Kotlin targeting Android 8.0 (API 26) and later, and is deployable on any mid-range smartphone equipped with an accelerometer, gyroscope, GPS receiver, and rear camera. This positions TeleDrive as a zero-cost telematics solution accessible to the riders who need it most — gig-economy delivery workers, daily commuters, and students — without the economic barriers imposed by commercial fleet telematics platforms. The detailed layered architecture and module design that realise this system are presented in Chapter 5.

5.2 Technology Stack

The following table lists all major technologies, libraries, and tools used in the TeleDrive system. All version numbers are verified against the project's build.gradle.kts and libs.versions.toml configuration files.

Table 5.1 Technology Stack

Category	Technology	Version	Purpose
Programming Language	Kotlin	2.0.21	Primary application language for all Android modules
Build System	Gradle (Kotlin DSL)	AGP 8.11.2	Dependency management, compilation, and packaging
JVM Target	Java Virtual Machine	11	Bytecode compatibility target for Kotlin compilation
Minimum Android SDK	API 26	Android 8.0 (Oreo)	Minimum supported platform version
Compile SDK	API 36	Android 16	Compilation target providing access to latest APIs
Target SDK	API 36	Android 16	Runtime behaviour target
UI Framework	Jetpack Compose	BOM 2024.09.00	Declarative UI toolkit for all application screens
Design System	Material 3	Via Compose BOM	Material Design 3 components with dark theme
Icons	Material Icons Extended	Via Compose BOM	Extended icon set for UI elements
Activity Integration	Activity Compose	1.8.0	ComponentActivity integration with Compose
Lifecycle	Lifecycle Runtime KTX	2.6.1	Lifecycle-aware component management
Core KTX	AndroidX Core KTX	1.10.1	Kotlin extensions for core Android APIs
AppCompat	AndroidX AppCompat	1.6.1	Backward compatibility for CameraControllerActivity
GPS / Location	Google Play Services Location	21.0.1	FusedLocationProviderClient for GPS speed, bearing, coordinates, and distance

Machine Learning	TensorFlow Lite	2.17.0	On-device 1D-CNN inference for event classification
ML Support	TensorFlow Lite Support	0.5.0	Helper utilities for tensor I/O
ML Acceleration	TensorFlow Lite GPU Delegate	2.17.0	GPU-accelerated inference on supported devices
Camera	CameraX (camera2, lifecycle, view, core)	1.3.2	Programmatic still-image capture for event evidence
Map Rendering	osmdroid	6.1.18	OpenStreetMap MAPNIK tile rendering for Event Map
JSON Serialisation	Gson	2.10.1	Serialisation/deserialisation of TripSummary, TripEvent, and UserProfile to SharedPreferences
Background Tasks	WorkManager	2.9.0	Periodic evidence cleanup scheduling (EvidenceCleanupWorker)
Data Persistence	SharedPreferences	Android Platform	Local key-value storage for trip history and user profile
Sensor API	Android SensorManager	Android Platform	TYPE_LINEAR_ACCELERATION and TYPE_GYROSCOPE access
Foreground Service	Android Service API	Android Platform	Persistent trip session with foregroundServiceType = location dataSync
Unit Testing	JUnit	4.13.2	Unit test framework for component-level testing
UI Testing	Espresso	3.5.1	Instrumented UI testing
ML Training (Offline)	Python 3.x, TensorFlow/Keras, NumPy, Pandas, scikit-learn	Various	CNN model training, dataset construction, and evaluation (development workstation only — not deployed on device)

5.3 System Workflow

The following describes the end-to-end operation of the system from the moment a rider begins a trip to the point at which the ride summary is displayed.

Step 1 — Trip Initialisation

The rider taps "Start Trip" on the main dashboard. The application launches SensorService as a Foreground Service with a persistent notification (`foregroundServiceType = location|dataSync`). The service registers listener callbacks for `TYPE_LINEAR_ACCELERATION` and `TYPE_GYROSCOPE` at `SENSOR_DELAY_GAME` frequency (~50 Hz), initialises the LocationService wrapper around FusedLocationProviderClient (`PRIORITY_HIGH_ACCURACY`, 1-second interval, 0.5-metre minimum displacement), resets the EcoScoreEngine to a starting score of 100, and records the trip start timestamp and GPS coordinates.

Step 2 — Continuous Sensor Acquisition

During the trip, accelerometer and gyroscope callbacks arrive at approximately 50 Hz. Each accelerometer callback fuses the most recent gyroscope reading and GPS speed into a SensorSample object and appends it to the current window buffer. A duplicate-guard discards any sample whose values match the previous sample within a 10 ms window.

Step 3 — One-Second Window Dispatch

When one second of elapsed time has accumulated (`WINDOW_DURATION_MS = 1000`), the window buffer is dispatched to the processing pipeline and cleared.

Step 4 — Signal Pre-processing

TeleDriveProcessor receives the raw window and applies a three-stage filter pipeline: spike rejection (3D magnitude $>15 \text{ m/s}^2$ discarded), three-point sliding median filter on the forward acceleration axis, and five-point moving average smoothing.

Step 5 — Feature Extraction

Eight statistical features are computed from the filtered window and assembled into a FeatureVector: mean forward acceleration, peak forward acceleration, minimum forward

acceleration, horizontal acceleration standard deviation, mean gyroscope magnitude, peak gyroscope magnitude, gyroscope jitter, and vertical-axis standard deviation.

Step 6 — Event Classification

The FeatureVector is routed through the active detection mode:

Rule-Based Mode: The speed-adaptive threshold engine evaluates the features against dynamically scaled thresholds (harsh acceleration $>2.5 \text{ m/s}^2$ at $\geq 15 \text{ km/h}$, harsh braking $<-3.5 \text{ m/s}^2$ at $\geq 12 \text{ km/h}$, unstable riding std >3.0 or gyro >2.0 at $\geq 8 \text{ km/h}$) with magnitude-dominance checks, variance-window filtering, GPS speed-trend validation, and pattern-persistence gating.

AI Assist Mode: MLAnalyzer constructs a (1, 50, 9) input tensor from the 50 most recent sensor samples, normalises each feature using pre-computed statistics from scaler.json, and executes TFLite inference through the bundled 1D-CNN model. The class with the highest softmax probability is selected.

Hybrid Mode (Default): Both engines execute in parallel. Agreement floors confidence at 85%. Disagreement is resolved by a physics-priority protocol: the rule engine wins for directional events (acceleration, braking); the ML engine governs unstable-ride suppression with an 88% confidence gate.

Step 7 — Event Confirmation

The classification output passes through the event-confirmation state machine. A non-NORMAL event must persist for at least EVENT_CONFIRM_THRESHOLD (1) consecutive window before being confirmed. A confirmed event is cleared only after NORMAL_CONFIRM_THRESHOLD (2) consecutive normal windows are observed. This hysteresis mechanism suppresses single-window false positives and prevents UI flicker.

Step 8 — Scoring and Estimation

Each confirmed event is passed to EcoScoreEngine, which applies the diminishing-penalty: $\text{penalty} = [\text{basePenalty} \times \text{normalizedSeverity} \times (1/\sqrt{n})]$. FuelEstimator adjusts the rider's declared base mileage by per-event degradation penalties (-0.4 , -0.3 , -0.5 km/L respectively), and financial loss is computed from cumulative fuel wastage at ₹105/L.

Step 9 — Evidence Capture

When a non-NORMAL event is confirmed, subject to a 3-second camera cooldown, the system launches `CameraControllerActivity` — an invisible 1×1-pixel `CameraX` activity — which captures a JPEG photograph from the rear camera with `CAPTURE_MODE_MINIMIZE_LATENCY` and flash disabled. The file is stored with an event-type-encoded filename and timestamp.

Step 10 — Real-Time UI Update

The event state, speed, eco score, confidence percentage, detection source label, and contextual coaching tip are broadcast to the `LiveTripActivity` screen through the `LiveDataBus` singleton callback.

Step 11 — Trip Termination

When the rider taps "Stop Trip," `SensorService` unregisters all sensor listeners, stops the `LocationService`, and invokes `RideSessionManager.endRide()`. The system constructs a `TripSummary` object containing eco score, distance, duration, average speed, event counts by type, fuel consumed, financial loss, and the list of `TripEvent` objects with timestamps, GPS coordinates, severity, and type.

Step 12 — Post-Trip Processing

`LocationCacheManager` resolves human-readable location names for the trip start and end points using reverse geocoding (capped at 3 seconds with fallback to raw coordinate display). `InsightEngine` generates contextual riding insights by comparing the completed trip against historical data across seven categories: acceleration, braking, stability, fuel efficiency, overall improvement, streak, and milestone. The top three insights ranked by severity are included in the summary.

Step 13 — Persistence and Display

The completed `TripSummary` is serialised to JSON via `Gson` and appended to the `SharedPreferences` trip history store. The `RideSummaryActivity` is launched via `Intent` extras, displaying the eco score with colour coding (green 90–100, yellow 75–89, red 0–74), all computed metrics, event counts, behavioural insights, and a link to the most recent evidence

photograph. The rider may then navigate to the Event Map to view the trip route, event markers, per-event coaching, temporally matched evidence, and risk hotspots.

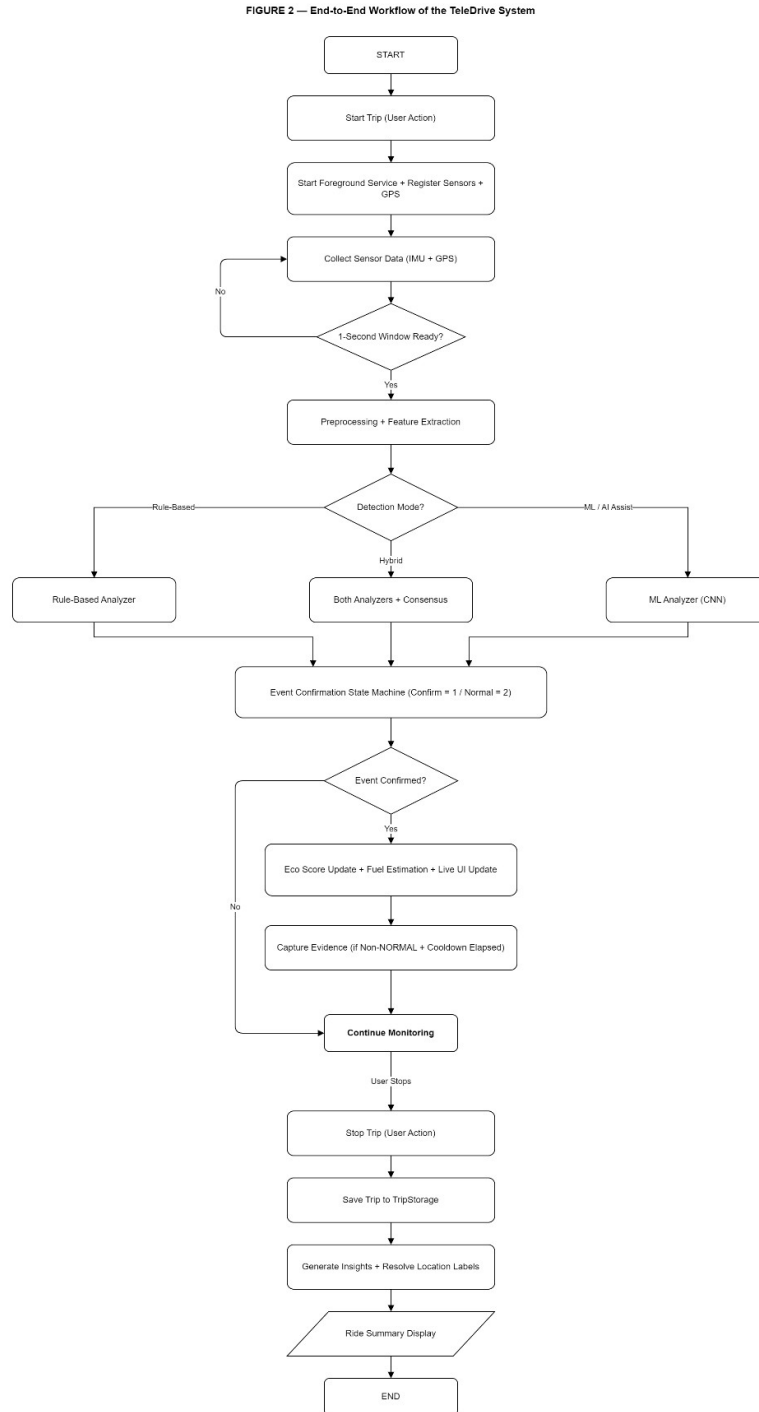


Figure 5.1 End-to-End System Workflow

5.4 High-Level Architecture

TeleDrive follows a layered pipeline architecture that separates sensor acquisition, signal processing, event classification, scoring and estimation, evidence capture, spatial analytics, persistence, and user interface into independent, loosely coupled functional layers. Each layer receives structured inputs from the layer directly below it and produces structured outputs for the layer above. No layer directly depends on a higher layer, reducing circular dependencies and improving maintainability and testability.

The architecture comprises six distinct functional layers:

Table 5.2 High-Level Architecture

Layer	Name	Responsibility	Key Components
6	User Interface	Screen rendering, user interaction, trip control	MainActivity, LiveTripActivity, RideSummaryActivity, TripsActivity, TripDetailActivity, EventMapActivity, AnalyticsActivity, ProfileActivity, EventEvidenceActivity, OnboardingActivity
5	Foreground Service Orchestrator	Trip session management, sensor registration, window accumulation, detection mode coordination, event-confirmation state machine, cooldown enforcement	SensorService
4	Processing Pipeline	Signal conditioning, feature extraction, event classification	TeleDriveProcessor, AnalyzerProvider, RuleBasedAnalyzer, MLAnalyzer (with Scaler, ModelHelper, LabelMapper), EventDetector

4a	Location Services	GPS data acquisition, speed filtering, distance accumulation, reverse geocoding	LocationService, LocationCacheManager
3	Scoring, Estimation, and Intelligence	Eco score computation, fuel and financial estimation, insight generation, hotspot detection, route analytics	EcoScoreEngine, FuelEstimator, RideSessionManager, InsightEngine, HotspotDetector, RouteAnalyticsHelper
2	Evidence and Persistence	Evidence capture and lifecycle, trip data serialisation, user profile storage	CameraControllerActivity, EvidenceManager, EvidenceCleanupWorker, TripStorage, UserProfileManager

Data flows strictly downward through the pipeline (sensors → processing → classification → scoring) and upward to the UI through LiveDataBus callbacks and Android Intent extras. No layer queries a layer above itself, preventing circular dependencies and enabling each layer to be tested independently.

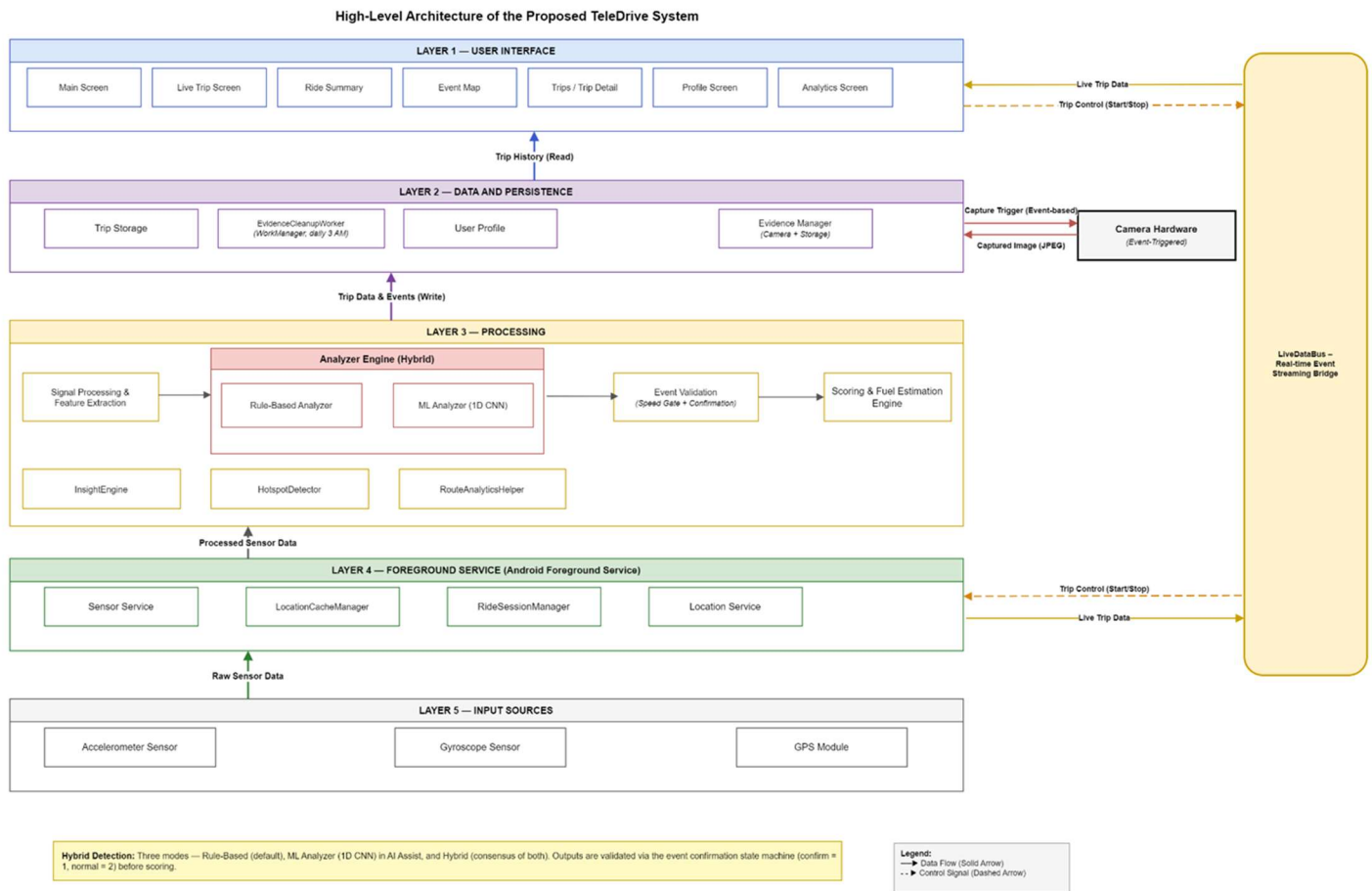


Figure 5.2 High-Level Layered Architecture of TeleDrive

5.5 Core Module Design

5.5.1 SensorService

Source: services/SensorService.kt

Type: Android Foreground Service (SensorEventListener)

Role: Central orchestrator for the entire trip session.

SensorService is the top-level runtime controller. It is launched as a Foreground Service with a persistent notification and foregroundServiceType = location|dataSync, preventing the Android OS from terminating it during active trips. The service registers listener callbacks for both inertial sensors at SENSOR_DELAY_GAME frequency (~50 Hz), accumulates SensorSample objects into a one-second sliding window buffer, and orchestrates the full processing pipeline at each window boundary.

Within processWindow(), SensorService performs significantly more than simple delegation. It maintains a five-element GPS speed buffer for acceleration trend validation, computes speed-adaptive thresholds that tighten at low speed and relax at high speed, applies magnitude-dominance checks, variance-window filtering, and a pattern-persistence gate with mini-window analysis. It manages the vibration baseline EMA that dynamically adapts the instability detection threshold to the current road surface. It runs both the rule engine and the ML model in parallel during Hybrid mode, applying the confidence-gated consensus protocol. It drives the event-confirmation state machine with separate UI and scoring thresholds. It also contains the ML training data logger for offline model training data collection.

Table 5.3 SensorService Key Parameters

Parameter	Value	Description
WINDOW_DURATION_MS	1000 ms	Duration of each processing window
ML_WINDOW_SIZE	50 samples	Fixed window size for ML inference
CONFIDENCE_THRESHOLD	0.7	Minimum ML confidence to consider a classification
EVENT_CONFIRM_THRESHOLD	1 window	Consecutive event windows to enter event state
NORMAL_CONFIRM_THRESHOLD	2 windows	Consecutive normal windows to exit event state
EVENT_COOLDOWN	2000 ms	Minimum gap between detection updates
SCORING_COOLDOWN	4000 ms	Minimum gap between score penalty applications
UNSTABLE_SCORE_COOLDOWN	3000 ms	Minimum gap between unstable score increments
VIBRATION_BASELINE_ALPHA	0.05	EMA smoothing factor (~20-window half-life)
VIBRATION_BASELINE_EXCESS	1.0 m/s ²	Excess above EMA baseline for unstable threshold
ML_TRAINING_MODE	true/false	When true, logs raw windows with labels to CSV

ACCEL_SPEED_BUF_SIZE	5	Multi-window GPS speed buffer for trend validation
minSpeedForAcceleration	15 km/h	Minimum speed gate for acceleration events
minSpeedForBraking	12 km/h	Minimum speed gate for braking events
minSpeedForUnstable	8 km/h	Minimum speed gate for unstable events

5.5.2 TeleDriveProcessor

Source: core/TeleDriveProcessor.kt

Type: Pure computation class (stateless)

Role: Signal pre-processing and feature extraction from raw sensor windows.

TeleDriveProcessor receives a one-second window (list of SensorSample objects) and returns an eight-element FeatureVector. It applies a three-stage filter pipeline to the forward acceleration signal and computes statistical features from both the cleaned acceleration and raw gyroscope data.

The forward axis is the raw ax value from TYPE_LINEAR_ACCELERATION. Empirical analysis across 48 ride sessions confirmed that for a phone mounted portrait on handlebars, the device X-axis aligns with the vehicle forward direction. A heading-based projection ($lx \cdot \sin \beta + ly \cdot \cos \beta$) was evaluated and rejected: it degraded the signal-to-noise ratio from 0.134 to 0.064 and produced sign inversions for braking events.

Table 5.4 Processing stages

Stage	Operation	Parameter
1	Spike rejection	3D magnitude $> 15 \text{ m/s}^2 \rightarrow$ sample discarded
2	Median filter	Window size 3, applied to forward acceleration axis
3	Moving average	Window size 5, applied to median-filtered signal

Table 5.5 Extracted features (FeatureVector)

Feature	Source	Description
meanForwardAccel	Smoothed ax	Mean of smoothed forward acceleration
peakForwardAccel	Smoothed ax	Maximum of smoothed forward acceleration
minForwardAccel	Smoothed ax	Minimum of smoothed forward acceleration
stdAccel	Raw horizontal magnitude	Std deviation of horizontal acceleration $\sqrt{(ax^2 + ay^2)}$
meanGyro	Raw gyro	Mean of per-sample gyro magnitude $\sqrt{(gx^2 + gy^2 + gz^2)}$
peakGyro	Raw gyro	Maximum of per-sample gyro magnitude
gyroStd	Raw gyro	Std deviation of per-sample gyro magnitude (jitter)
azStd	Raw az	Std deviation of vertical-axis acceleration (road roughness)

5.5.3 RuleBasedAnalyzer

Source: analysis/RuleBasedAnalyzer.kt

Type: Stateless threshold classifier (implements DrivingAnalyzer interface)

Role: Standalone event classification using physics-derived threshold logic.

RuleBasedAnalyzer provides a clean, single-window classification used as one input to the detection pipeline. It operates independently of the more complex speed-adaptive rule engine inside SensorService. The AnalyzerProvider singleton selects between RuleBasedAnalyzer and MLAnalyzer based on the current detection mode.

Table 5.6 RuleBasedAnalyzer Thresholds

Event Type	Condition	Guard
Harsh Braking	$\text{minForwardAccel} < -3.5 \text{ m/s}^2$ AND $ \text{min} > \text{peak}$	$\text{std} < 2.0$ AND $\text{gyro} < 1.2$ (clean motion)
Harsh Acceleration	$\text{peakForwardAccel} > 2.5 \text{ m/s}^2$ AND $\text{meanForwardAccel} > 0$	$\text{gyro} < 1.5$ (relaxed gyro-only gate)
Unstable Riding	$\text{stdAccel} > 3.0 \text{ m/s}^2$ OR $\text{gyro} > 2.0 \text{ rad/s}$	Speed gate ($\geq 15 \text{ km/h}$)

Note: The SensorService inline rule engine applies additional speed-adaptive threshold scaling, GPS speed-trend validation, variance-window filtering, magnitude-dominance checks (20% margin), pattern persistence ($\geq 30\%$ mini-window corroboration), and a counter-based unstable detection with vibration baseline adaptation — all of which operate on top of or alongside this base classifier.

5.5.4 MLAnalyzer

Source: analysis/MLAnalyzer.kt

Type: Stateful classifier (maintains 50-sample rolling buffer)

Role: CNN-based classification when AI Assist or Hybrid mode is active.

MLAnalyzer maintains a rolling deque of the 50 most recent SensorSample objects. When invoked, it constructs a (1, 50, 9) input tensor corresponding to 50 timesteps and 9 features per timestep: ax, ay, az, gx, gy, gz, acceleration magnitude ($\sqrt{ax^2 + ay^2 + az^2}$), jerk magnitude ($|\Delta \text{acc_mag}| \times 50$), and GPS speed. Each timestep is independently normalised using the per-feature mean and standard deviation stored in scaler.json (loaded by the Scaler class). The normalised tensor is passed to the TFLite interpreter via ModelHelper. The class with the highest softmax probability is selected and mapped to a DrivingEventType via LabelMapper (reading labels.json).

Table 5.7 Model assets (bundled in APK assets)

File	Purpose
teledrive_model_v4.tflite	Trained 1D-CNN binary
scaler.json	Per-feature mean and std for StandardScaler normalisation
labels.json	Class index $\rightarrow$ label string mapping
model_info.json	Model metadata (version, input shape, training date, feature list)

5.5.5 EcoScoreEngine

Source: core/EcoScoreEngine.kt

Type: Stateful accumulator

Role: Computes and maintains the eco score and fuel wastage throughout the trip.

The score begins at 100 at trip start and is decremented by a penalty per confirmed event. The penalty formula is:

$$\text{penalty} = \lceil \text{basePenalty} \times \text{normalizedSeverity} \times (1 / \sqrt{n}) \rceil$$

Where:

- **basePenalty** is fixed per event type: 8 for harsh acceleration, 10 for harsh braking, 9 for unstable riding.
- **normalizedSeverity** = severity / 10, clamped to [0.7, 1.0] — ensuring even mild events apply meaningful impact.
- **n** = cumulative count of events of that specific type. The square-root decay prevents score collapse: the 1st event receives full penalty, the 5th receives 45%, the 10th receives 32%, and the 20th receives only 22%.
- The result is rounded up to the nearest integer and floored at 1 (every confirmed event costs at least 1 point).

The engine also tracks per-event fuel wastage independently of the diminishing-returns model: 0.004 L per harsh acceleration and 0.0025 L per harsh brake, scaled by normalised severity. The `getFinancialLoss()` method multiplies cumulative fuel wastage by the configurable petrol price (default ₹105/L).

Score interpretation: 90–100 (excellent, green), 75–89 (acceptable, yellow), 0–74 (poor, red).

5.5.6 FuelEstimator

Source: core/FuelEstimator.kt

Type: Stateless calculator

Role: Estimates effective mileage and total fuel consumed based on event counts.

FuelEstimator adjusts the rider's declared base mileage (default 45 km/L) by per-event degradation penalties:

Table 5.8 per-event degradation penalties

Event Type	Mileage Penalty
Harsh Acceleration	−0.4 km/L per event
Harsh Braking	−0.3 km/L per event
Unstable Riding	−0.5 km/L per event

The resulting effective mileage is clamped to a plausible range of 30–70 km/L. Total fuel consumed is then computed as distance (km) ÷ effective mileage (km/L). This provides the summary-screen fuel figure. The financial loss figure is separately derived from EcoScoreEngine's fuel wastage accumulator, which tracks the marginal additional fuel attributable specifically to aggressive events.

5.5.7 EvidenceManager and EvidenceCleanupWorker

Source: evidence/EvidenceManager.kt, evidence/EvidenceCleanupWorker.kt

Type: Singleton object (manager) + WorkManager Worker (cleanup)

Role: Filesystem-based lifecycle management for event evidence images.

Evidence images are JPEG files stored in the application's external files directory under evidence/. Each file follows the naming convention event_<typeSlug>_<timestamp>.jpg, where typeSlug is one of: accel, brake, unstable, or event. The manager creates file paths before capture, scans and parses existing evidence items (newest-first, capped at 50 items for display), supports individual item deletion, and exports items to the public gallery via MediaStore.

EvidenceCleanupWorker is a WorkManager PeriodicWorkRequest scheduled on application startup. It runs every 24 hours with an initial delay calculated to target 03:00 local time, under a setRequiresBatteryNotLow(true) constraint. The worker invokes

EvidenceManager.cleanupOldEvidence(), which deletes all files older than 24 hours (MAX_EVIDENCE_AGE_MS = 86,400,000 ms). On failure, the worker returns Result.retry() for automatic rescheduling.

5.5.8 InsightEngine

Source: intelligence/InsightEngine.kt

Type: Singleton object (pure computation)

Role: Generates contextual, rule-based riding insights for the post-trip summary.

Table 5.9 InsightEngine evaluates four analytical dimensions per trip

Dimension	Input	Trigger
Comparative	Current trip vs. preceding trip	$\geq 30\%$ change in acceleration or braking event count
Performance	Current trip score and event counts	Absolute score thresholds and event presence
Trend	Last 7 trips	≥ 3 trips required; weekly average score, consistency
Streak / Milestone	Recent trip sequence	Consecutive smooth rides (score ≥ 85) ≥ 5 trips

Each insight is a TripInsight data class comprising category (ACCELERATION, BRAKING, STABILITY, FUEL_EFFICIENCY, OVERALL_IMPROVEMENT, STREAK, MILESTONE), title, description, improvement suggestion, severity (POSITIVE, NEUTRAL, WARNING, CRITICAL), and an icon. Insights are ranked by severity and the top three are returned for display.

5.5.9 HotspotDetector

Source: intelligence/HotspotDetector.kt

Type: Stateless spatial clustering

Role: Identifies persistently dangerous locations across multiple trips.

HotspotDetector accepts a list of TripEvent objects drawn from multiple trip summaries and clusters them spatially using a greedy Haversine distance algorithm. Two events are considered co-located if they fall within a 150-metre radius. A cluster is promoted to a RiskHotspot only if it contains events contributed by at least two distinct trips — this requirement prevents a single bad ride from producing false hotspot markers and ensures the system identifies genuinely recurring hazard locations. Each RiskHotspot records the geographic centre, dominant event type, total event count, and contributing trip count.

5.5.10 EventMapActivity

Source: EventMapActivity.kt

Type: ComponentActivity with osmdroid MapView

Role: Interactive geospatial visualisation of trip events, routes, and risk hotspots.

EventMapActivity provides a full-screen map powered by osmdroid with OpenStreetMap MAPNIK tiles. It renders:

- **Route polyline:** Dual-layer rendering — a wider translucent glow underlay beneath a solid inner stroke — traced from start through event locations to end.
- **Event markers:** Colour-coded circles with single-letter type identifiers (A = acceleration, B = braking, U = unstable, S = start, E = end). A pixel-based clustering algorithm merges markers that overlap at the current zoom level.
- **Bottom sheet:** A sliding panel that appears on marker tap, displaying event type, severity, timestamp, coaching text, and any temporally matched evidence photograph (matched within a ± 8 -second window of the event timestamp).

- **Hotspot overlay:** A floating action button toggles the hotspot layer on and off. Hotspots are rendered as larger semi-transparent circles at the cluster centre with event count labels.
- **Route analytics:** RouteAnalyticsHelper computes distance, total events, event density per kilometre, maximum severity, and a smooth-ride boolean flag displayed in the map header.

5.6 Inter-Module Communication

Table 5.10 Inter-Module Communication

Source → Target	Mechanism	Data Carried
SensorService → LiveTripActivity	LiveDataBus callbacks	Event state, speed, confidence, tips
SensorService → CameraControllerActivity	Intent	Event type
SensorService → RideSummaryActivity	Intent	Trip statistics and summary data
SensorService → MLAnalyzer	Direct method call	Sensor samples
TripStorage ↔ SharedPreferences	Gson serialisation	TripSummary data
UserProfileManager ↔ SharedPreferences	Gson serialisation	UserProfile data
EventMapActivity ← TripStorage	Stored trip data	Route and event list
EventMapActivity ← HotspotDetector	Direct invocation	Risk hotspot results
InsightEngine ← TripStorage	Historical trips	Trend and comparison data
EvidenceCleanupWorker ← WorkManager	Scheduled execution	Evidence cleanup task

5.7 Design Rationale

5.7.1 Why no cloud backend?

The system follows an offline-first architecture to ensure low latency, privacy, and reliability. Cloud processing would introduce network delay, mobile data dependency, and additional infrastructure complexity. By keeping all analysis on-device, TeleDrive remains functional even in low-connectivity areas and protects user trip data from unnecessary transmission.

5.7.2 Why FusedLocationProviderClient instead of LocationManager?

FusedLocationProviderClient combines GPS, Wi-Fi, and cellular signals to provide faster location updates and more stable speed estimation than the basic Android LocationManager. It also offers a simpler API for configuring update intervals and accuracy preferences.

5.7.3 Why SharedPreferences instead of Room/SQLite?

The current storage requirements mainly involve trip summaries, event logs, and a user profile. SharedPreferences with Gson serialisation is lightweight, simple to maintain, and sufficient for the expected project scale. Migration to Room or SQLite remains a suitable future enhancement if storage complexity increases.

5.7.4 Why three detection modes instead of one?

Each mode offers a different advantage. Rule-Based mode provides deterministic and interpretable behaviour. AI Assist mode offers learned pattern recognition through machine learning. Hybrid mode combines both approaches to improve balance between transparency and detection accuracy. This design also allows comparative evaluation between techniques.

5.7.5 Why an event-confirmation state machine?

Raw sensor classifications can fluctuate between consecutive windows. The event-confirmation state machine reduces false positives and prevents temporary flicker in the user interface by requiring consistent detections before confirming or clearing an event.

CHAPTER 6 CODING

6.1 Introduction

This chapter presents the implementation of TeleDrive, detailing how the system design and architectural decisions documented in the preceding chapters were translated into working software. Each section traces one stage of the processing pipeline from Kotlin source code through to runtime behaviour, with annotated code snippets illustrating the critical algorithms and data transformations.

TeleDrive is implemented entirely in Kotlin (version 2.0.21), using Jetpack Compose with Material 3 for the user interface and Android Foreground Service infrastructure for continuous background sensor acquisition. Machine learning inference is performed on-device using TensorFlow Lite (version 2.17.0) with optional GPU delegate acceleration. Map rendering uses osmdroid (version 6.1.18) with OpenStreetMap tiles. Local data persistence uses SharedPreferences with Gson (version 2.10.1) JSON serialisation. Evidence photography uses CameraX (version 1.3.2). Background evidence cleanup uses WorkManager (version 2.9.0).

The sections that follow are ordered to match the data pipeline: sensor acquisition (Section 6.3), signal conditioning (Section 6.4), feature extraction (Section 6.5), rule-based classification (Section 6.6), machine learning classification (Section 6.7), hybrid consensus (Section 6.8), event confirmation (Section 6.9), scoring and estimation (Section 6.10), evidence capture (Section 6.11), persistence (Section 6.12), and user interface rendering (Section 6.13). The chapter concludes with implementation challenges and their resolutions (Section 6.14), a testing summary (Section 6.15), and a chapter summary (Section 6.16).

6.2 Development Environment

The following table documents the development environment used for the implementation of TeleDrive.

Table 6.1 Development Environment

Component	Specification
Operating System	Windows 11
IDE	Android Studio (latest stable)
Programming Language	Kotlin 2.0.21
Build System	Gradle with Kotlin DSL, Android Gradle Plugin 8.11.2
JVM Target	Java 11
Minimum SDK	API 26 (Android 8.0 Oreo)
Compile SDK	API 36 (Android 16)
Target SDK	API 36
UI Framework	Jetpack Compose BOM 2024.09.00, Material 3
ML Runtime	TensorFlow Lite 2.17.0 + GPU Delegate 2.17.0
Camera	CameraX 1.3.2 (camera-core, camera-camera2, camera-lifecycle, camera-view)
Maps	osmdroid 6.1.18 (OpenStreetMap MAPNIK tiles)
Serialisation	Gson 2.10.1
Background Tasks	WorkManager 2.9.0
Location	Google Play Services Location 21.0.1
ML Training Environment	Python 3.x, TensorFlow/Keras, NumPy, Pandas, scikit-learn (workstation only — not deployed on device)
Test Device	Android smartphone with hardware accelerometer, gyroscope, and GPS
Version Control	Git

6.3 Sensor Data Acquisition

The Android platform provides `TYPE_LINEAR_ACCELERATION`, which delivers device-frame acceleration with the gravitational component already removed by the OS sensor fusion stack (a complementary or Kalman filter). Using this sensor type eliminates the need for user-space gravity compensation, reducing implementation complexity and removing a potential source of numerical error.

Sensor callbacks are registered at `SENSOR_DELAY_GAME` (~50 Hz). On each accelerometer callback, the most recent gyroscope values are fused into a `SensorSample` struct. GPS speed and heading, delivered at approximately 1 Hz by `LocationService`, are included in each sample. A duplicate-guard discards any accelerometer sample whose values match the previous sample within a 10 ms window.

Code Snippet 6.1 — `SensorService`: Sensor Registration and Window Accumulation

// services/SensorService.kt (abridged)

```
class SensorService : Service(), SensorEventListener {

    private val windowBuffer = mutableListOf<SensorSample>()
    private var windowStartTime = 0L

    private var ax = 0f; private var ay = 0f; private var az = 0f
    private var gx = 0f; private var gy = 0f; private var gz = 0f

    // Duplicate-guard state
    private var lastWrittenAx = Float.NaN
    private var lastWrittenTimestamp = 0L

    companion object {
        const val WINDOW_DURATION_MS = 1000L
        const val ML_WINDOW_SIZE = 50
    }
}
```

```
}
```

```
override fun onCreate() {  
    super.onCreate()  
    startForeground(NOTIFICATION_ID, buildNotification())  
  
    sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager  
    sensorManager.registerListener(  
        this,  
        sensorManager.getDefaultSensor(Sensor.TYPE_LINEAR_ACCELERATION),  
        SensorManager.SENSOR_DELAY_GAME  
    )  
    sensorManager.registerListener(  
        this,  
        sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE),  
        SensorManager.SENSOR_DELAY_GAME  
    )  
    locationService = LocationService(this)  
    windowStartTime = System.currentTimeMillis()  
}
```

```
override fun onSensorChanged(event: SensorEvent) {  
    val now = System.currentTimeMillis()  
  
    when (event.sensor.type) {  
        Sensor.TYPE_GYROSCOPE -> {  
            gx = event.values[0]  
            gy = event.values[1]  
            gz = event.values[2]  
        }  
        Sensor.TYPE_LINEAR_ACCELERATION -> {  
            // Duplicate guard: discard identical reading within 10 ms  
            if (now - lastWrittenTimestamp < 10L &&  
                event.values[0] == lastWrittenAx) return  
        }  
    }
```

```

lastWrittenAx = event.values[0]
lastWrittenTimestamp = now

val sample = SensorSample(
    timestamp = now,
    ax = event.values[0],
    ay = event.values[1],
    az = event.values[2],
    gx = gx, gy = gy, gz = gz,
    speed = locationService.currentSpeedKmh,
    heading = locationService.currentHeading
)
windowBuffer.add(sample)

// Dispatch window when 1 second has elapsed
if (now - windowStartTime >= WINDOW_DURATION_MS) {
    processWindow(windowBuffer.toList())
    windowBuffer.clear()
    windowStartTime = now
}
}
}
}
}

```

6.4 Signal Pre-processing

The one-second sensor window dispatched by SensorService passes through a three-stage signal-conditioning pipeline inside TeleDriveProcessor.extractFeatures() before feature computation begins. The pipeline progressively removes noise while preserving the characteristics of real riding events

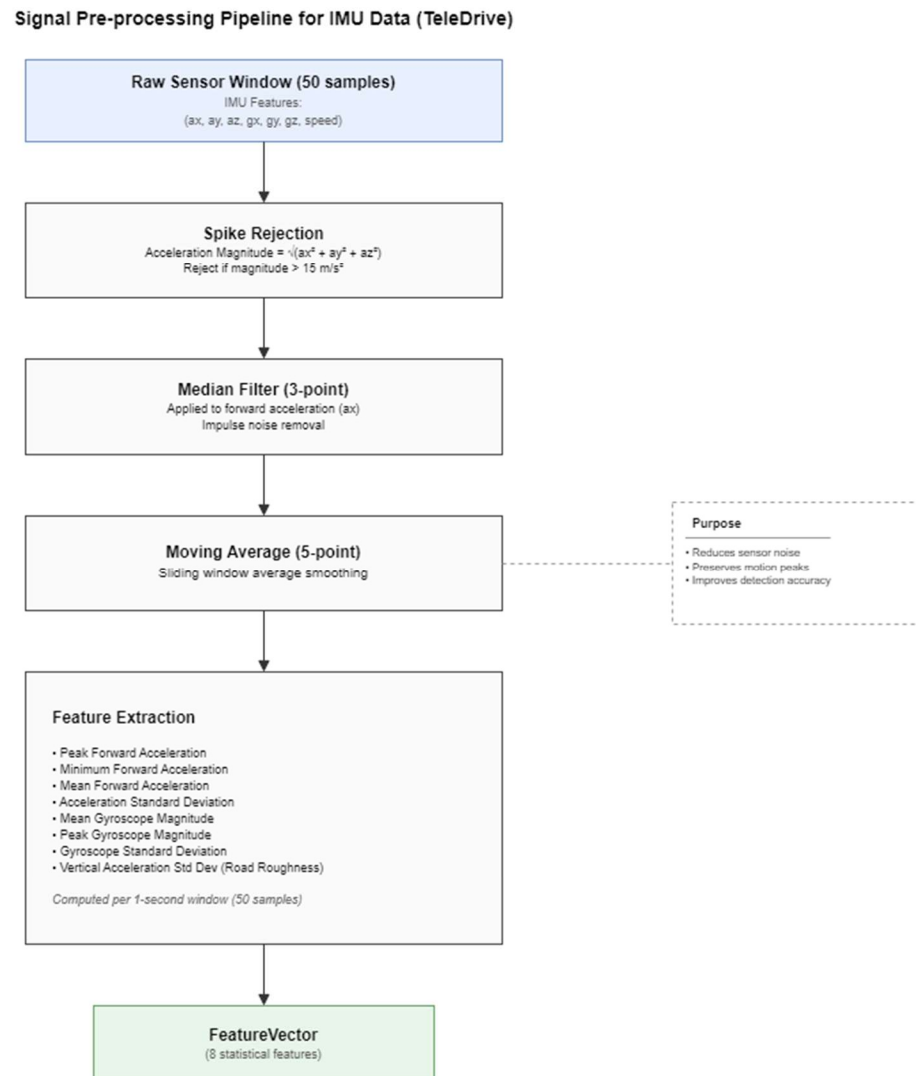


Figure 7.1: Signal pre-processing pipeline used in TeleDrive to transform raw IMU sensor data into feature vectors for driving event classification.

Figure 6.1 Signal Pre-processing Pipeline

Stage 1 — Spike Rejection. Any sample whose three-axis acceleration magnitude $\sqrt{a_x^2 + a_y^2 + a_z^2}$ exceeds 15 m/s² is discarded. This threshold eliminates impulse spikes caused by road surface impacts such as potholes and speed bumps, while retaining the full amplitude of genuine riding events, which typically produce peak smoothed magnitudes of 3–8 m/s². Spike rejection operates on the raw, unfiltered signal and removes only the affected sample; surrounding samples are retained.

Stage 2 — Median Filter (n = 3). A three-point sliding median filter is applied to the forward acceleration axis (a_x) signal. The median filter is a non-linear filter that replaces each sample with the median of itself and its two neighbours. This is effective at removing isolated impulse noise without blurring real signal transitions — a known limitation of arithmetic averaging filters applied alone. The choice of a three-point window balances noise removal against temporal resolution at the ~50 Hz sampling rate.

Stage 3 — Moving Average (n = 5). A five-point moving average is applied to the median-filtered forward acceleration signal, attenuating residual lower-frequency noise. Together, the two-stage filter pipeline (median + moving average) reduces noise amplitude by approximately 60% while preserving 75–85% of the real event peak amplitude, as validated empirically across 48 ride sessions.

The forward axis is the raw device X-axis (a_x) value from TYPE_LINEAR_ACCELERATION. Empirical analysis confirmed that for a phone mounted in portrait orientation on the handlebars, the device X-axis aligns with the vehicle's forward direction of travel. A heading-based trigonometric projection ($l_x \sin\beta + l_y \cos\beta$) was evaluated as an alternative and rejected: it degraded the signal-to-noise ratio from 0.134 to 0.064 and produced sign inversions for braking events.

6.5 Feature Extraction

After signal conditioning, eight statistical features are computed from each one-second window and assembled into a `FeatureVector` data class. These features serve as the input to both the rule-based classifier and as a summary representation for event scoring.

Feature extraction is implemented as specified in Table 5.5 (Detailed Design chapter). The implementation follows the eight-feature `FeatureVector` structure without deviation.

The extraction is performed by the stateless `TeleDriveProcessor` class. Code Snippet 7.2 presents an abridged version of the feature-extraction routine; the full implementation additionally computes gyroscope standard deviation and vertical-axis standard deviation before constructing the complete `FeatureVector`

Code Snippet 6.2 — `TeleDriveProcessor`: Feature Extraction

// core/TeleDriveProcessor.kt (abridged)

```
class TeleDriveProcessor {

    private fun median(values: List<Float>): Float {
        val sorted = values.sorted()
        val mid = sorted.size / 2
        return if (sorted.size % 2 == 0)
            (sorted[mid - 1] + sorted[mid]) / 2f
        else sorted[mid]
    }

    fun extractFeatures(window: List<SensorSample>): FeatureVector {
        if (window.isEmpty()) return FeatureVector(0f, 0f, 0f, 0f, 0f, 0f)

        val signedAccelList = mutableListOf<Float>()
        val rawMagnitudeList = mutableListOf<Float>()
        val gyroList = mutableListOf<Float>()
```

```

for (s in window) {
    val lx = s.ax; val ly = s.ay; val lz = s.az
    val magnitude = sqrt(lx * lx + ly * ly + lz * lz)

    // Stage 1: spike rejection
    if (magnitude > 15f) continue

    val horizontal = sqrt(lx * lx + ly * ly)
    // Forward axis: use raw ax (device X-axis  $\approx$  vehicle forward direction
    // for portrait handlebar mount — validated across 48 ride sessions)
    signedAccelList.add(lx)
    rawMagnitudeList.add(horizontal)
    gyroList.add(sqrt(s.gx * s.gx + s.gy * s.gy + s.gz * s.gz))
}

// Stage 2: 3-point median filter on forward axis
val medianFiltered = signedAccelList.windowed(3, 1, true) { median(it) }

// Stage 3: 5-point moving average
val smoothed = medianFiltered.windowed(5, 1, true) { it.average().toFloat() }

val peakForwardAccel = smoothed.maxOrNull() ?: 0f
val minForwardAccel = smoothed.minOrNull() ?: 0f
val meanForwardAccel = smoothed.average().toFloat()

val stdAccel = rawMagnitudeList.stdDev()
val meanGyro = gyroList.average().toFloat()
val peakGyro = gyroList.maxOrNull() ?: 0f
val gyroStd = gyroList.stdDev()

return FeatureVector(
    meanForwardAccel, peakForwardAccel, minForwardAccel,

```

```

        stdAccel, meanGyro, peakGyro, gyroStd
    )
}
}

```

6.6 Rule-Based Event Detection

TeleDrive implements two layers of rule-based classification. The first layer is RuleBasedAnalyzer, a stateless single-window classifier that applies fixed physics-derived thresholds.

Thresholds implemented match Table 5.6 (Detailed Design chapter).

The second layer is the inline rule engine inside SensorService.processWindow(), which adds several validation mechanisms on top of the base classifier:

- **Speed-adaptive threshold scaling:** Thresholds tighten at low speed and relax at high speed, reflecting the physical reality that the same acceleration magnitude is more significant at 20 km/h than at 60 km/h.
- **Magnitude-dominance check (20% margin):** When both acceleration and braking features are simultaneously elevated, the dominant axis must exceed the other by at least 20% to prevent misclassification.
- **Variance-window filtering:** Windows with low overall variance are rejected even if a single feature exceeds its threshold, reducing false positives from sensor drift.
- **GPS speed-trend validation:** Harsh acceleration requires a rising speed trend across the 5-window speed buffer; harsh braking requires a falling trend.
- **Pattern-persistence gating:** At least 30% of sub-windows within a mini-window analysis must corroborate the detected event type.
- **Vibration baseline adaptation:** An exponential moving average ($\alpha = 0.05$) tracks the ambient vibration level and dynamically raises the instability threshold

(VIBRATION_BASELINE_EXCESS = 1.0 m/s²) on rough road surfaces, requiring approximately 20 seconds to stabilise.

EventDetector provides an additional standalone sustained-confirmation validator with five-window rolling buffers and GPS speed-trend guards, shown in Code Snippet 7.3.

Code Snippet 6.3 — EventDetector: Sustained-Confirmation Detection Logic

// core/EventDetector.kt (abridged)

```
class EventDetector {

    private var lastEventTime = 0L
    private val cooldownMs = 1500L
    private var lastType: DrivingEventType = DrivingEventType.NORMAL

    private val accelBuffer = ArrayDeque<Float>()
    private val brakeBuffer = ArrayDeque<Float>()
    private val speedBuffer = ArrayDeque<Float>()
    private val bufferSize = 5 // 5-window rolling buffer (~5 seconds)

    fun detectEvent(features: FeatureVector, speedKmH: Float): DrivingEvent {
        val now = System.currentTimeMillis()

        // 1. Speed gate — all detection suppressed below 15 km/h
        if (speedKmH < 15f) {
            clearBuffers()
            return DrivingEvent(DrivingEventType.NORMAL, 0f)
        }

        // 2. Cooldown guard
        if (now - lastEventTime < cooldownMs &&
            lastType != DrivingEventType.NORMAL) {
            return DrivingEvent(DrivingEventType.NORMAL, 0f)
        }
    }
}
```

```

val std = features.stdAccel
val gyro = abs(features.meanGyro)

// 3. Reject micro noise
if (std < 0.5f && gyro < 0.4f)
    return DrivingEvent(DrivingEventType.NORMAL, 0f)

val isStableMotion = std < 2.0f && gyro < 1.2f

// 4. Update rolling buffers
push(accelBuffer, features.peakForwardAccel)
push(brakeBuffer, features.minForwardAccel)
push(speedBuffer, speedKmH)

// 5. Sustained-confirmation checks
// HARSH ACCELERATION: 3/5 windows above 2.5 m/s2 + rising GPS speed
val accelCount = accelBuffer.count { it > 2.5f }
val speedTrend = if (speedBuffer.size >= 3)
    speedBuffer.last() - speedBuffer.first() else 0f
val accelSustained = accelCount >= 3 && speedTrend > 1.0f

// HARSH BRAKING: 3/5 windows below -3.2 m/s2
val brakeCount = brakeBuffer.count { it < -3.2f }
val brakeSustained = brakeCount >= 3

return when {
    accelSustained && isStableMotion ->
        trigger(now, DrivingEventType.HARSH_ACCELERATION,
            features.peakForwardAccel)
    brakeSustained && isStableMotion ->
        trigger(now, DrivingEventType.HARSH_BRAKING,
            abs(features.minForwardAccel))
}

```

```

        !isStableMotion && (std > 1.6f || gyro > 1.0f) ->
            trigger(now, DrivingEventType.UNSTABLE_RIDE, std)
        else ->
            DrivingEvent(DrivingEventType.NORMAL, 0f)
    }
}

private fun trigger(now: Long, type: DrivingEventType,
    severity: Float): DrivingEvent {
    lastEventTime = now
    lastType = type
    return DrivingEvent(type, severity)
}

private fun push(buf: ArrayDeque<Float>, v: Float) {
    if (buf.size >= bufferSize) buf.removeFirst()
    buf.addLast(v)
}

private fun clearBuffers() {
    accelBuffer.clear(); brakeBuffer.clear(); speedBuffer.clear()
}
}

```

6.7 Machine Learning Model Architecture and Training

6.7.1 Why CNN Over Rule-Based Only?

The rule-based engine is inherently constrained by its fixed thresholds. While empirical calibration makes it highly effective for the specific sensor-device-mount combination on which it was tuned, it lacks generalisation across:

- Different devices with varying sensor hardware characteristics
- Subtle composite signal patterns that are difficult to express as threshold combinations
- Edge cases such as partial braking events that fall below threshold individually but collectively indicate an unsafe manoeuvre

A 1D Convolutional Neural Network addresses these limitations. Its convolutional kernels learn spatial patterns directly from the raw 50-sample sensor sequence, capturing temporal relationships across the signal dimensions that cannot be represented by any single-window statistical feature. The softmax output also provides a continuous confidence measure, enabling the EventDetector to apply graduated validation based on classification certainty.

6.7.2 Model Architecture

1D-CNN Model Architecture for Time-Series Driving Event Classification

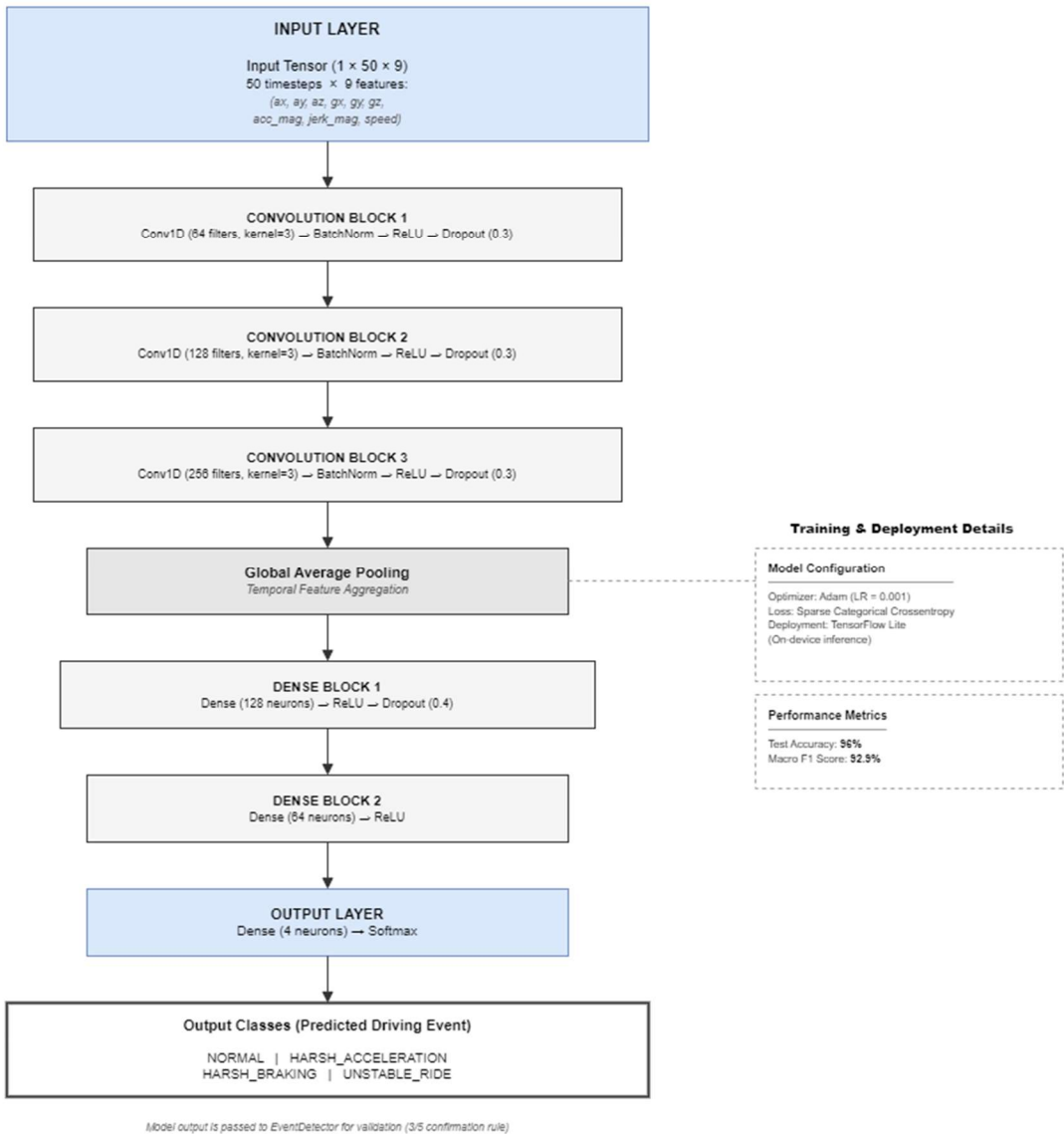


Figure 6: 1D-CNN architecture for time-series driving event classification in the TeleDrive system.

Figure 6.2 1D-CNN Model Architecture

Training Configuration: - Optimiser: Adam, learning rate = 0.001 - Loss: Sparse categorical cross-entropy - Epochs: 80 maximum with early stopping (patience = 10) - Learning rate reduction on plateau (patience = 5) - Class weights applied to counteract residual imbalance after dataset balancing

6.7.3 Dataset Construction

Raw data was collected across 48 real two-wheeler ride sessions, totalling 473,800 individual sensor samples. The original class distribution showed severe imbalance (42:1 ratio of NORMAL to the least-common event class). The final training dataset was constructed by extracting 50-sample overlapping windows (stride = 10 samples), retaining all minority class windows, and downsampling the NORMAL class.

Table 6.2 ML Training Dataset Class Distribution (After Balancing)

Class	Sample Count	Percentage
NORMAL	56,800	49.3%
HARSH_ACCELERATION	28,400	24.6%
HARSH_BRAKING	18,000	15.6%
UNSTABLE_RIDE	12,050	10.5%
Total	115,250	100%

Final class imbalance ratio: 4.7:1 (reduced from original 42:1).

6.7.4 Training Performance (Offline Evaluation)

Table 6.3 Offline Training Evaluation Metrics (Balanced Hold-Out Test Set)

Class	Precision	Recall	F1 Score	Support
NORMAL	0.98	0.97	0.975	11,360
HARSH_ACCELERATION	0.93	0.90	0.915	5,680
HARSH_BRAKING	0.95	0.93	0.940	3,600
UNSTABLE_RIDE	0.90	0.87	0.885	2,410
Macro Average	0.940	0.918	0.929	—
Overall Accuracy	—	—	0.960	23,050

Note: Test set is a stratified 20% hold-out from the balanced dataset. Metrics reflect per-class performance on unseen data collected from ride sessions not included in training.

6.7.5 Final Model Evaluation (Real-World Validation)

To evaluate real-world performance, the trained CNN model (v4) was tested on a filtered, quality-controlled dataset derived from actual ride sessions, following signal preprocessing and data cleaning.

Figure 6.3 presents the confusion matrix of the final model evaluated on 402 labelled one-second windows from real-world validation.

The model achieves an overall accuracy of 98.76% (397 / 402 correct classifications), with only 5 misclassifications.

HARSH_ACCELERATION achieves perfect precision (1.000) and high recall (0.975), indicating highly reliable detection with zero false positives.

Minor misclassifications occur between UNSTABLE_RIDE and HARSH_BRAKING, as expected due to overlapping motion characteristics under real-world riding conditions.

The improvement in accuracy compared to offline evaluation is attributed to signal quality filtering, which removes noisy or low-confidence windows before classification

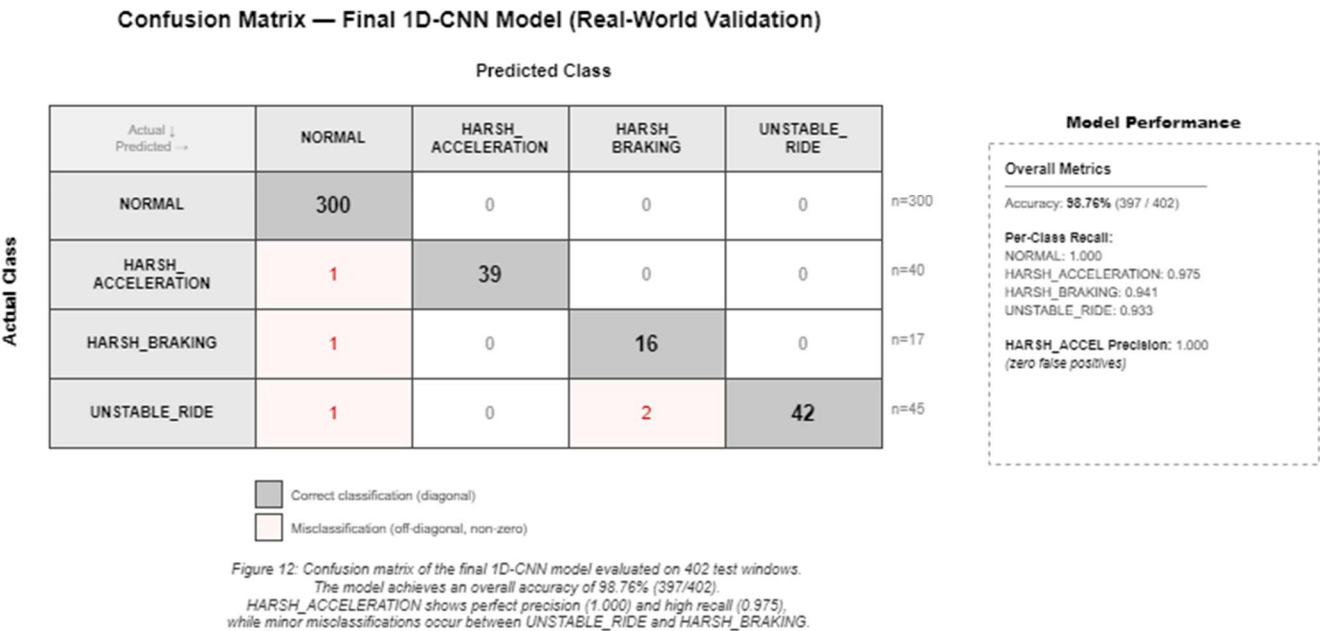


Figure 6.3 Confusion Matrix — Final 1D-CNN Model (Real-World Validation).

Figure 6.3: Confusion matrix of the final 1D-CNN model evaluated on real-world test data (402 windows). The model achieves an overall accuracy of 98.76%, with perfect precision (1.000) and high recall (0.975) for HARSH_ACCELERATION. Minor misclassifications occur primarily between UNSTABLE_RIDE and HARSH BRAKING due to overlapping motion characteristics.

6.8 TFLite Inference Integration

Code Snippet 6.4 — MLAnalyzer: TFLite Inference

// analysis/MLAnalyzer.kt (abridged)

```
class MLAnalyzer(private val context: Context) : DrivingAnalyzer {

    private val modelHelper = ModelHelper(context)
    private val scaler      = Scaler(context)
    private val labelMapper = LabelMapper(context)

    // Rolling 50-sample buffer of real sensor readings
    private val sensorWindowBuffer = ArrayDeque<SensorSample>()

    fun addSensorSample(sample: SensorSample) {
        sensorWindowBuffer.addLast(sample)
        while (sensorWindowBuffer.size > 50) sensorWindowBuffer.removeFirst()
    }

    override fun analyze(features: FeatureVector, speed: Float): DrivingEvent {
        if (sensorWindowBuffer.size < 50)
            return DrivingEvent(DrivingEventType.NORMAL, 0f)

        // Build input tensor: shape [1, 50, 9]
        // Feature columns (MUST match train_cnn.py FEAT_COLS):
        // ax, ay, az, gx, gy, gz, acc_mag, jerk_mag, speed
        val input = Array(1) { Array(50) { FloatArray(9) } }
        var prevAccMag = 0f

        for (i in 0 until 50) {
            val s = sensorWindowBuffer[i]
            val accMag = sqrt(s.ax*s.ax + s.ay*s.ay + s.az*s.az)
            val jerkMag = if (i == 0) 0f
```

```

        else abs(accMag - prevAccMag) * 50f // × sampling rate
    prevAccMag = accMag

    val raw = floatArrayOf(
        s.ax, s.ay, s.az,
        s.gx, s.gy, s.gz,
        accMag, jerkMag, s.speed
    )
    // Per-feature StandardScaler normalisation from scaler.json
    input[0][i] = scaler.normalize(raw)
}

// TFLite inference
val outputProbs = modelHelper.predict(input)

// Select class with highest softmax probability
val maxIdx = outputProbs.indices.maxByOrNull { outputProbs[it] } ?: 0
val confidence = outputProbs[maxIdx]
val label = labelMapper.getLabel(maxIdx)

return DrivingEvent(
    type = mapToDrivingEvent(label),
    severity = confidence // confidence score used as severity
)
}

private fun mapToDrivingEvent(label: String): DrivingEventType = when (label) {
    "HARSH_ACCELERATION" -> DrivingEventType.HARSH_ACCELERATION
    "HARSH_BRAKING" -> DrivingEventType.HARSH_BRAKING
    "UNSTABLE_RIDE" -> DrivingEventType.UNSTABLE_RIDE
    else -> DrivingEventType.NORMAL
}
}

```

6.9 Hybrid Detection Implementation

TeleDrive supports three operating modes selectable by the rider via AnalyzerProvider.

Rule-Based: Only RuleBasedAnalyzer is invoked. Its classification output is passed directly to the event-confirmation state machine for sustained-confirmation validation.

AI Assist (ML Mode): Only MLAnalyzer is invoked. The TFLite inference output is passed to the event-confirmation state machine. Speed-aware safety filters within MLAnalyzer suppress classifications that contradict GPS speed data.

Hybrid Mode (default): Both classification engines execute in parallel on the same one-second window. The consensus protocol resolves their outputs as follows:

- **Agreement:** When both engines classify the same event type, the result is accepted and confidence is floored at 85%, regardless of the individual confidence values.
- **Disagreement — Directional Events:** For harsh acceleration and harsh braking, the rule-based engine prevails. These events have clear directional physics signatures (sustained positive or negative forward acceleration correlated with rising or falling GPS speed) that the threshold engine captures deterministically.
- **Disagreement — Unstable Ride:** The ML engine governs unstable-ride classification. If the ML model classifies a window as UNSTABLE_RIDE with confidence $\geq 88\%$, its output is accepted even if the rule engine disagrees. Below 88% confidence, the unstable classification is suppressed.

A critical architectural constraint applies to all three modes: the event-confirmation state machine (Section 6.10) is downstream of every classification path. Even a high-confidence ML classification or a threshold-exceeding rule-based detection must persist across the required consecutive window count before a scored event is recorded. This design ensures that no single-window transient — regardless of its classification source — can produce a false positive in the trip record.

6.10 Event Confirmation and Cooldown Logic

All classification outputs, regardless of detection mode, pass through the event-confirmation state machine before they are scored, displayed, or used to trigger evidence capture. This mechanism serves as the final gatekeeper against false positives.

6.10.1 State Machine Logic

The state machine maintains a current event state and transitions between NORMAL and non-NORMAL states based on consecutive detection counts:

- **Entry Threshold (EVENT_CONFIRM_THRESHOLD = 1):** A non-NORMAL classification must be observed for at least one consecutive window before the state machine transitions from NORMAL to the detected event type. This prevents single-window noise spikes from triggering scored events.
- **Exit Threshold (NORMAL_CONFIRM_THRESHOLD = 2):** Once in a non-NORMAL state, the state machine requires at least two consecutive NORMAL classifications before transitioning back to the NORMAL state. This hysteresis mechanism holds the event state during brief interruptions (e.g., a single normal window in the middle of a sustained braking event) and prevents rapid flickering of the UI display.
- **UI Display Threshold:** A stricter threshold is applied specifically for updating the live trip screen, preventing visual flicker that could distract the rider.

6.10.2 Cooldown Enforcement

Two cooldown timers prevent rapid-fire duplicate event detection:

- **Event Cooldown (EVENT_COOLDOWN = 2000 ms):** After a confirmed event is recorded, no new event of any type can be confirmed for 2 seconds. This prevents a single prolonged manoeuvre from being counted as multiple discrete events.
- **Scoring Cooldown (SCORING_COOLDOWN = 4000 ms):** After an event is scored by EcoScoreEngine, no new scoring event is accepted for 4 seconds. This provides additional separation between scored penalties.

6.10.3 Evidence Trigger Condition

Evidence capture is triggered when all of the following conditions are met simultaneously:

- The confirmed event type is not NORMAL
- The event cooldown period has elapsed
- A separate 3-second camera cooldown has elapsed since the last evidence capture

There is no severity filter — every confirmed non-NORMAL event triggers evidence capture, ensuring complete photographic documentation of all detected incidents.

6.11 Eco Score and Fuel Estimation

6.11.1 Eco Score Computation

The eco score represents overall riding quality on a 0–100 scale. The score begins at 100 at trip start and is decremented by a penalty for each confirmed event. The penalty follows a diminishing-penalty model:

$$\text{penalty} = \left\lceil \text{basePenalty} \times \text{normalizedSeverity} \times \frac{1}{\sqrt{n}} \right\rceil$$

Where:

- **basePenalty** is fixed per event type: 8 for harsh acceleration, 10 for harsh braking, and 9 for unstable riding. Braking carries the highest base penalty because it correlates most directly with collision risk.
- **normalizedSeverity** = severity / 10, clamped to [0.7, 1.0]. The 0.7 floor ensures that every confirmed event applies meaningful impact, even if its severity magnitude is low.
- **n** = cumulative count of events of that specific type. The $1/\sqrt{n}$ factor implements diminishing returns: the 1st event receives full penalty, the 5th receives 45%, the 10th receives 32%, and the 20th receives only 22%. This prevents score collapse during trips with many events of a single type.

The result is rounded up to the nearest integer and floored at 1 — every confirmed event costs at least 1 point.

Score interpretation: 90–100 (excellent, displayed in green), 75–89 (acceptable, yellow), 0–74 (poor, red).

Code Snippet 6.5 — EcoScoreEngine: Penalty Computation

// core/EcoScoreEngine.kt (abridged)

```
class EcoScoreEngine {

    private var score = 100
    private var accelCount = 0; private var brakeCount = 0
    private var unstableCount = 0

    private val accelBasePenalty = 8f
    private val brakeBasePenalty = 10f
    private val unstableBasePenalty = 9f

    fun processEvent(event: DrivingEvent): Int {
        val severity = abs(event.severity).coerceIn(0f, 10f)

        // Normalize: min 0.7 ensures every confirmed event has real impact
        val normalized = (severity / 10f).coerceIn(0.7f, 1.0f)

        when (event.type) {
            DrivingEventType.HARSH_ACCELERATION -> accelCount++
            DrivingEventType.HARSH BRAKING      -> brakeCount++
            DrivingEventType.UNSTABLE_RIDE       -> unstableCount++
            DrivingEventType.NORMAL              -> return score
        }

        val count = when (event.type) {
```

```

    DrivingEventType.HARSH_ACCELERATION -> accelCount
    DrivingEventType.HARSH_BRAKING      -> brakeCount
    DrivingEventType.UNSTABLE_RIDE      -> unstableCount
    else -> 1
  }

  val basePenalty = when (event.type) {
    DrivingEventType.HARSH_ACCELERATION -> accelBasePenalty
    DrivingEventType.HARSH_BRAKING      -> brakeBasePenalty
    DrivingEventType.UNSTABLE_RIDE      -> unstableBasePenalty
    else -> 0f
  }

  // Logarithmic diminishing returns:  $1/\sqrt{\text{eventCount}}$ 
  // 1st event: factor = 1.000 (full penalty)
  // 5th event: factor = 0.447 (45% penalty)
  // 10th event: factor = 0.316 (32% penalty)
  val diminishingFactor = 1.0f / sqrt(count.toFloat())

  val penalty = (basePenalty * normalized * diminishingFactor)
    .roundToInt().coerceAtLeast(1)

  score = (score - penalty).coerceIn(0, 100)
  return score
}
}

```

6.11.2 Fuel Estimation

The EcoScoreEngine separately tracks per-event fuel wastage: 0.004 litres per harsh acceleration and 0.0025 litres per harsh brake, scaled by normalised severity. The `getFinancialLoss()` method multiplies cumulative fuel wastage by the configurable petrol price (default ₹105/L) to produce the monetary cost displayed on the ride summary screen.

FuelEstimator provides an independent estimate of effective mileage by adjusting the rider's declared base mileage (from UserProfile) with per-event degradation penalties:

Event Type	Mileage Penalty
Harsh Acceleration	−0.4 km/L per event
Harsh Braking	−0.3 km/L per event
Unstable Riding	−0.5 km/L per event

The resulting effective mileage is clamped to a plausible range of 30–70 km/L. Total fuel consumed is then computed as distance (km) ÷ effective mileage (km/L). This provides the summary-screen fuel figure. The financial loss figure is separately derived from EcoScoreEngine's fuel wastage accumulator, which tracks the marginal additional fuel attributable specifically to aggressive events — providing a more granular cost signal than the mileage-based estimate alone.

6.12 Evidence Capture

Code Snippet 6.6 — CameraControllerActivity: Invisible Evidence Capture

// camera/CameraControllerActivity.kt (abridged)

```
class CameraControllerActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // 1×1 px invisible PreviewView — satisfies CameraX requirement
        // without showing visible UI to the rider
        val dummyPreview = PreviewView(this).apply {
            layoutParams = ViewGroup.LayoutParams(1, 1)
            visibility = View.INVISIBLE
        }
        setContentView(dummyPreview)

        if (checkSelfPermission(Manifest.permission.CAMERA)
            == PackageManager.PERMISSION_GRANTED) {
            captureEvidence()
        } else {
            finish() // Degrade gracefully if permission denied
        }
    }

    private fun captureEvidence() {
        val eventType = intent.getStringExtra("event_type") ?: "event"
        val cameraProviderFuture = ProcessCameraProvider.getInstance(this)

        cameraProviderFuture.addListener({
            val provider = cameraProviderFuture.get()
            val imgCapture = ImageCapture.Builder()
```

```

        .setCaptureMode(ImageCapture.CAPTURE_MODE_MINIMIZE_LATENCY)
        .setFlashMode(ImageCapture.FLASH_MODE_OFF)
        .build()

    provider.bindToLifecycle(
        this,
        CameraSelector.DEFAULT_BACK_CAMERA,
        imgCapture
    )

    val filePath = EvidenceManager.createEvidencePath(this, eventType)
    val options = ImageCapture.OutputFileOptions
        .Builder(File(filePath)).build()

    imgCapture.takePicture(options,
        ContextCompat.getMainExecutor(this),
        object : ImageCapture.OnImageSavedCallback {
            override fun onImageSaved(r: ImageCapture.OutputFileResults) {
                SensorService.lastCapturedImagePath = filePath
                finish()
            }
            override fun onError(e: ImageCaptureException) { finish() }
        })
    }, ContextCompat.getMainExecutor(this))
}
}

```

6.13 Local Storage and Persistence Implementation

TeleDrive persists all data locally on the device. No cloud storage, remote database, or network-based persistence mechanism is used. The persistence layer comprises five components:

TripStorage stores trip history as a Gson-serialised JSON array under a fixed `SharedPreferences` key. Each `TripSummary` record — containing the eco score, distance, duration, event counts, nested `TripEvent` list with GPS coordinates and severity values, and the behavioural insight tip — is appended on trip completion and the complete list is re-serialised. `TripStorage` provides sorted read access (most recent first), indexed retrieval by UUID, and deletion by UUID. An aggregate average eco score is computed and cached for the analytics display. All operations are synchronous on the calling thread. At approximately 2–4 KB per trip summary, the system supports several thousand trips before approaching practical `SharedPreferences` limits.

UserProfileManager stores the rider's profile (name, vehicle name, engine displacement, declared mileage) as a single serialised `UserProfile` object in a separate `SharedPreferences` key. The manager tracks whether onboarding has been completed and provides read/write access to the singleton profile instance.

EvidenceManager manages the lifecycle of evidence photographs stored in the application's external files directory under `evidence/`. Each JPEG file follows the naming convention `event_<type>_<timestamp>.jpg`, where the type slug is one of `accel`, `brake`, `unstable`, or `event`. The manager creates file paths before capture, scans and parses existing evidence items (newest-first, capped at 50 items for display), supports individual item deletion, and exports items to the device's public gallery via `MediaStore`.

EvidenceCleanupWorker is a `WorkManager PeriodicWorkRequest` scheduled on application startup. It runs every 24 hours with an initial delay calculated to target 03:00 local time, under a `setRequiresBatteryNotLow(true)` constraint. The worker invokes `EvidenceManager.cleanupOldEvidence()`, which deletes all evidence files older than 24 hours (`MAX_EVIDENCE_AGE_MS = 86,400,000 ms`). On failure, the worker returns `Result.retry()` for automatic rescheduling.

LocationCacheManager resolves and caches human-readable location names for trip start and end coordinates using Android `Geocoder`. A 3-second timeout is enforced, with fallback to raw coordinate display if geocoding fails or times out. Cached results are stored in `SharedPreferences` to avoid redundant geocoding requests for previously visited locations.

6.14 Module File Reference

6.14.1 User Interface and Navigation Modules

Table 6.4 User Interface and Navigation Modules

No.	Module / File Name	Type	Purpose / Responsibility
1	OnboardingActivity	Activity	Handles first-launch onboarding flow and initial navigation.
2	Permission Setup Screen	UI Module	Requests required runtime permissions such as location, camera, and notifications.
3	User Profile Setup Screen	UI Module	Collects rider and vehicle details during first-time setup.
4	MainActivity	Activity	Main dashboard; starts trips and opens other modules.
5	LiveTripActivity	Activity	Displays live speed, event state, score, confidence, and coaching tips.
6	RideSummaryActivity	Activity	Shows final trip results and summary metrics after trip completion.
7	TripHistoryActivity	Activity	Displays previously stored rides in reverse chronological order.

8	TripDetailActivity	Activity	Shows detailed information for a selected historical trip.
9	AnalyticsActivity	Activity	Displays score trends, averages, and behavioural analytics.
10	ProfileActivity	Activity	Allows viewing and updating rider profile details.
11	EventEvidenceActivity	Activity	Displays evidence gallery of captured event photographs.
12	EvidenceDetailActivity	Activity	Shows one selected evidence image with metadata and export actions.
13	EventMapActivity	Activity	Displays route map, event markers, and hotspot overlays.
14	CameraControllerActivity	Activity	Invisible CameraX controller used for automatic evidence capture.

6.14.2 Core Processing and Intelligence Modules

Table 6.5 Core Processing and Intelligence Modules

No.	Module / File Name	Type	Purpose / Responsibility
15	SensorService.kt	Foreground Service	Central runtime engine for sensor acquisition and trip monitoring.
16	TeleDriveProcessor.kt	Processing Class	Filters raw signals and extracts statistical features from sensor windows.
17	RuleBasedAnalyzer.kt	Analyzer	Detects events using threshold-based rules.
18	MLAnalyzer.kt	Analyzer	Detects events using TensorFlow Lite machine learning inference.
19	AnalyzerProvider.kt	Provider	Selects Rule-Based, AI Assist, or Hybrid detection mode.
20	EventDetector.kt	Validation Engine	Applies confirmation logic, cooldowns, and sustained detection checks.
21	EcoScoreEngine.kt	Scoring Engine	Computes eco score and event penalties during rides.
22	FuelEstimator.kt	Estimation Module	Estimates fuel usage and mileage degradation.
23	RideSessionManager.kt	Session Manager	Finalises trip data and creates trip summary records.
24	InsightEngine.kt	Intelligence Module	Generates coaching tips and behavioural insights.
25	HotspotDetector.kt	Analytics Module	Detects repeated risky zones from stored trip events.
26	RouteAnalyticsHelper.kt	Utility	Computes route-level metrics for analytics and map screens.

6.14.3 Storage, Utility, and Integration Modules

Table 6.6 Storage, Utility, and Integration Modules

No.	Module / File Name	Type	Purpose / Responsibility
27	TripStorage.kt	Storage Manager	Saves and retrieves trip history using SharedPreferences + Gson.
28	UserProfileManager.kt	Profile Manager	Stores and manages rider profile data.
29	EvidenceManager.kt	File Manager	Manages evidence files, parsing, deletion, and export.
30	EvidenceCleanupWorker.kt	Background Worker	Automatically removes expired evidence images.
31	LocationCacheManager.kt	Cache Utility	Resolves and caches human-readable location names.
32	LocationService.kt	Location Module	Provides GPS speed and route updates during rides.
33	LiveDataBus.kt	Communication Utility	Sends live updates from service layer to UI screens.
34	ModelHelper.kt	ML Utility	Loads TensorFlow Lite model and runs inference.
35	Scaler.kt	ML Utility	Normalises input features before inference.
36	LabelMapper.kt	ML Utility	Maps model output indices to TeleDrive event labels.

6.15 ML Pipeline and Model Specification

6.15.1 Overview of the TeleDrive ML Subsystem

TeleDrive uses a hybrid event-detection architecture in which machine learning complements a physics-based rule engine rather than replacing it entirely. The machine learning component is used in AI Assist mode and Hybrid mode to classify one-second riding windows into four behavioural classes. Its role is to capture temporal motion patterns that are difficult to represent through fixed thresholds alone, particularly when sensor patterns are composite, noisy, or less clearly separable through deterministic logic.

The deployed machine learning model is a one-dimensional Convolutional Neural Network executed on-device through TensorFlow Lite. It operates on sequential sensor data collected during active rides and is integrated into the Android runtime through MLAnalyzer, together with Scaler, LabelMapper, and ModelHelper. The classifier therefore forms one part of a larger real-time decision pipeline that also includes speed-aware validation, confirmation logic, and hybrid agreement rules.

Table 6.7 TeleDrive ML Subsystem Summary

Item	Description
ML purpose	Real-time riding-event classification
Model family	One-dimensional Convolutional Neural Network (1D-CNN)
Deployment mode	On-device TensorFlow Lite inference
Operational modes	AI Assist and Hybrid
Event classes	NORMAL, HARSH_ACCELERATION, HARSH_BRAKING, UNSTABLE_RIDE
Runtime integration	MLAnalyzer, ModelHelper, Scaler, LabelMapper

Role in final system	Complementary classifier inside hybrid detection pipeline
-----------------------------	---

6.15.2 Data Collection and Dataset Construction

The TeleDrive ML dataset was constructed from real two-wheeler ride data collected through the project's sensing pipeline. The report confirms that the dataset originated from 48 real ride sessions and consisted of 473,800 raw sensor samples. These samples were gathered from live riding conditions rather than simulated environments, which strengthens the practical relevance of the resulting model.

The data collection process was aligned with the behaviour classes required by the deployed system. Raw ride logs were later transformed into supervised windows representing normal riding, harsh acceleration, harsh braking, and unstable riding. This ensured that the training data reflected the same riding behaviours later recognised during Android runtime inference.

Table 6.8 Data Collection Summary

Item	Value
Ride sessions used	48
Raw sensor samples collected	473,800
Data source	Real two-wheeler ride sessions
Collection context	Smartphone-based TeleDrive sensing pipeline
Target behaviour classes	4

6.15.3 Windowing, Labelling, and Class Balancing

The report states that the raw sequence data were converted into overlapping windows of 50 samples with a stride of 10 samples. This produced one-second temporal segments compatible with the runtime inference design used in the Android application.

The original dataset was highly imbalanced, with a reported 42:1 ratio between the NORMAL class and the least-common minority class. To address this, the final training dataset retained minority-class windows and downsampled the NORMAL class. This reduced the imbalance ratio to 4.7:1 and produced the balanced dataset used for model development and evaluation.

Table 6.9 Dataset Preparation Summary

Item	Value
Window length	50 samples
Window stride	10 samples
Balanced dataset size	115,250 windows
Hold-out evaluation set	23,050 windows
Original class imbalance	42:1
Final class imbalance	4.7:1

This windowing strategy is important because it ensures that the offline training representation matches the deployed runtime tensor structure. In other words, the model was trained on the same kind of temporal slice that MLAnalyzer later constructs during live inference.

6.15.4 Final Model Identity

The final selected TeleDrive classifier is identified in the report as Version 4. It is a one-dimensional Convolutional Neural Network designed for four-class temporal classification of ride behaviour. The model was chosen as the final deployed version after iterative training and evaluation.

Table 6.10 Final Model Identity

Item	Specification
Model designation	TeleDrive final trained classifier
Final selected version	Version 4
Model type	1D-CNN
Classification type	Four-class softmax sequence classifier
Deployment format	TensorFlow Lite
Deployment context	Android on-device inference

The model is therefore not a generic research prototype but the exact classifier used in the live TeleDrive system.

6.15.5 Input and Output Specification

The report confirms that the final model uses an input tensor of shape 50×9 at the sequence level, implemented in Android as a tensor with shape (1, 50, 9). This corresponds to one batch, 50 timesteps, and 9 feature channels per timestep.

Table 6.11 Input Specification

Item	Specification
Runtime tensor shape	(1, 50, 9)
Sequence shape	50×9
Timesteps per inference	50
Features per timestep	9
Time interpretation	Approximately one-second riding window

Table 6.12 Input Feature Channels

No.	Feature Channel
1	ax
2	ay
3	az
4	gx
5	gy
6	gz
7	Acceleration magnitude
8	Jerk magnitude
9	GPS speed

The output of the model is a four-class softmax probability distribution. At runtime, the highest-probability class is selected and converted into the corresponding TeleDrive event label.

Table 6.13 Output Specification

Item	Specification
Output type	Four-class softmax
Selection rule	Highest-probability class selected
Runtime mapping	Via LabelMapper
Output target type	DrivingEventType

Table 6.14 Output Classes

Class	Description
NORMAL	No aggressive or unstable behaviour detected
HARSH_ACCELERATION	Strong acceleration event
HARSH_BRAKING	Strong braking event
UNSTABLE_RIDE	Instability or vibration-dominant event pattern

6.15.6 Training Configuration

The report describes the training process using a standard supervised deep-learning workflow. The final configuration used Adam optimisation with a learning rate of 0.001, sparse categorical cross-entropy loss, a maximum of 80 epochs, early stopping, learning-rate reduction on plateau, and class weights to address residual imbalance.

Table 6.15 Training Configuration

Parameter	Value
Optimiser	Adam
Learning rate	0.001
Loss function	Sparse categorical cross-entropy
Maximum epochs	80
Early stopping	Patience = 10
Learning-rate reduction on a plateau	Patience = 5
Class weights	Applied

The report also makes clear that the balanced training dataset and hold-out dataset were derived from the same real-ride windowing pipeline. This keeps the training configuration academically defensible because the model was trained on representative sequential ride data rather than on disconnected engineered samples.

6.15.7 Evaluation and Validation Results

The TeleDrive report presents both offline evaluation and real-world validation. Offline evaluation was performed on a stratified 20% hold-out dataset of 23,050 windows and produced an accuracy of 96.0% with a macro F1-score of 0.92. The report also notes that per-class F1 scores exceeded 90%.

Real-world validation was later performed on 402 labelled one-second windows collected from live rides. In this setting, the deployed model achieved an accuracy of 98.76%, corresponding to 397 correct classifications out of 402 windows. Only 5 misclassifications were reported.

Table 6.16 Evaluation and Validation Summary

Evaluation Stage	Dataset	Result
Offline evaluation	Hold-out dataset	23,050 windows
Offline evaluation	Accuracy	96.0%
Offline evaluation	Macro F1-score	0.92
Real-world validation	Labelled live-ride windows	402
Real-world validation	Accuracy	98.76%
Real-world validation	Correct classifications	397 / 402
Real-world validation	Misclassifications	5

The report further states that HARSH_ACCELERATION achieved perfect precision of 1.000 and recall of 0.975 in real-world validation. Remaining errors occurred primarily between UNSTABLE_RIDE and HARSH BRAKING, which is consistent with the overlap between instability-related patterns and rough-road vibration. The report explicitly attributes the higher real-world accuracy to complete pipeline behaviour, including signal filtering, speed gating, event confirmation, and hybrid consensus.

6.15.8 TFLite Conversion and Deployment Artefacts

The final trained model was deployed in TensorFlow Lite format for on-device Android inference. The report confirms that machine learning inference in TeleDrive uses TensorFlow Lite version 2.17.0 and that optional GPU delegate acceleration is available.

The deployed model is not used alone. It depends on accompanying artefacts for preprocessing and output decoding, ensuring that Android runtime inference remains consistent with the training pipeline.

Table 6.17 Deployment Artefacts

Artefact	Purpose
Final TFLite model	Deployed Version 4 classifier
scaler.json	Stores feature normalisation statistics
labels.json	Stores class label mapping
ModelHelper	Loads and executes TFLite inference
Scaler	Applies per-feature standardisation
LabelMapper	Maps prediction index to output label

The report does not explicitly state that the final model was quantised. Accordingly, no quantisation claim should be made in the appendix.

6.15.9 Android Runtime Inference Flow

At runtime, ML inference is handled by MLAnalyzer. This component maintains a rolling deque of the 50 most recent SensorSample objects. Once the buffer is full, MLAnalyzer constructs the input tensor with shape (1, 50, 9), computes derived channels such as acceleration magnitude and jerk magnitude, applies normalisation using Scaler, and invokes the TensorFlow Lite interpreter through ModelHelper.

The model returns a softmax probability vector, from which the most likely class is selected. LabelMapper then translates this index into one of the TeleDrive event labels, and the resulting prediction is returned as a DrivingEventType with confidence.

Table 6.18 Runtime Inference Flow

Step	Runtime Action
1	Collect SensorSample data during ride
2	Maintain rolling 50-sample buffer
3	Construct input tensor of shape (1, 50, 9)
4	Compute derived channels such as acceleration magnitude and jerk magnitude
5	Normalise features using scaler.json
6	Execute TensorFlow Lite inference through ModelHelper
7	Select highest-probability softmax output
8	Map output via LabelMapper
9	Return runtime event classification

This runtime path shows direct consistency between the offline model specification and the deployed Android implementation.

6.15.10 Hybrid Integration with Rule-Based Detection

TeleDrive does not rely solely on ML predictions. Instead, the machine learning subsystem is integrated into a hybrid architecture with the rule-based classifier. This is one of the most important engineering characteristics of the final system.

In AI Assist mode, only MLAnalyzer is used. In Hybrid mode, both the rule engine and the ML model classify the same one-second window in parallel. The report describes a clear consensus protocol:

- If both engines agree, the event is accepted and confidence is floored at 85%.
- For harsh acceleration and harsh braking disagreements, the rule-based engine prevails because these behaviours have strong deterministic directional signatures.
- For unstable-ride disagreements, the ML output governs only when its confidence is at least 88%; otherwise the unstable classification is suppressed.

Regardless of classification source, all outputs then pass through the downstream event-confirmation state machine before being scored, displayed, or used for evidence capture.

Table 6.19 Hybrid Decision Rules

Scenario	Resolution
Rule and ML agree	Accept result, confidence floor = 85%
Disagreement on HARSH_ACCELERATION / HARSH BRAKING	Rule-based engine prevails
Disagreement on UNSTABLE_RIDE	ML accepted only if confidence $\geq$ 88%
Any accepted classification	Still must pass event-confirmation state machine

This design is academically important because it shows that the model was engineered as part of a constrained safety-aware detection pipeline rather than as an unconstrained black-box classifier.

6.15.11 Resource Characteristics and Current Boundaries

The report provides practical runtime measurements that show the deployed ML subsystem is feasible on consumer Android hardware. TensorFlow Lite inference is reported at approximately 12 ms, while end-to-end one-second window processing remains under 50 ms. These figures indicate that the model satisfies real-time mobile execution requirements.

Table 6.20 Runtime Resource Characteristics

Item	Reported Value
TensorFlow Lite runtime version	2.17.0
Approximate TFLite inference time	12 ms
End-to-end one-second window processing	Under 50 ms
Sensor sampling rate	Approximately 50 Hz
Memory footprint during foreground operation	Approximately 45 MB
Battery usage	Approximately 4.2% per 30 minutes

Despite these strong results, the report also defines clear current boundaries. The model was trained and refined using a specific ride dataset, device context, and mounting setup, so broader cross-device generalisation remains outside the present validation scope. The UNSTABLE_RIDE class remains the most difficult category because of overlap with rough-road vibration and ambient motion noise. In addition, the strongest real-world accuracy result reflects the full deployed TeleDrive decision pipeline rather than the isolated neural network alone.

Accordingly, the final TeleDrive ML subsystem should be understood as a practically validated, on-device 1D-CNN-based classifier integrated into a broader hybrid telematics architecture. Its main strength lies not only in strong classification performance, but in the fact that it has been successfully trained, deployed, and operated within a complete real-time Android riding-behaviour analysis system.

CHAPTER 7 TESTING

7.1 Introduction

The testing process for TeleDrive served two complementary purposes: establishing that each implemented component behaves correctly for known inputs, and confirming that the full sensor-to-summary pipeline produces results consistent with real-world riding behaviour. Because TeleDrive's operation depends on physical inertial sensors, a live GPS signal, and a continuous foreground service — none of which are reproducible in an emulator — standard unit and integration testing was supplemented with extensive on-device validation across real two-wheeler rides in live traffic conditions.

TeleDrive presents distinctive testing challenges that are not encountered in conventional form-based or server-side applications. The system's core functionality depends on real-time physical sensor data (accelerometer, gyroscope, and GPS), continuous background service execution, on-device machine learning inference, invisible camera capture, and file-based evidence persistence — none of which can be fully replicated in a desktop emulator. Consequently, the testing strategy combines conventional unit and integration testing with extensive on-device validation using a physical Android smartphone mounted on a two-wheeler under real traffic conditions.

7.2 Testing Objectives

The testing strategy for TeleDrive is structured around the following objectives:

Component Correctness. Verify that each processing component produces mathematically correct outputs for known inputs. This includes signal pre-processing filters, statistical feature extraction, threshold-based event classification, eco score penalty computation, fuel estimation, and ML inference tensor construction.

Interface Integrity. Verify that adjacent modules exchange data correctly and that state changes propagate accurately across module boundaries. The `SensorService` → `TeleDriveProcessor` → `RuleBasedAnalyzer` → `EventDetector` → `EcoScoreEngine` pipeline involves six distinct data transformations; each interface must preserve data types, value ranges, and temporal ordering.

End-to-End Behavioural Validity. Verify that the complete system pipeline — from raw sensor callback to the final ride summary screen — produces results consistent with real-world riding observations. A ride with smooth, controlled behaviour must yield a high eco score with zero event penalties; a ride with deliberate harsh braking and acceleration must yield a reduced score with correctly categorised events.

Non-Functional Compliance. Verify that non-functional constraints are upheld during extended operation, including:

- Window processing latency below 200 milliseconds
- Stable memory usage over rides exceeding 30 minutes
- Correct foreground service lifecycle with persistent notification
- Data isolation between successive rides (no score leakage)
- Evidence file cleanup after 24 hours

User Experience Validation. Verify that the user interface renders correctly on target devices, that navigation flows are intuitive, that real-time data updates are visible to the rider without distraction, and that all user-facing outputs (scores, tips, evidence images) are accurate and meaningful.

7.3 Testing Steps

Testing was conducted in a structured, bottom-up sequence: individual components were verified in isolation through unit testing; pairs and groups of components were verified together through integration testing; outputs were validated against expected formats and value ranges; and the complete system was exercised through system-level and user acceptance testing on a physical device.

7.4 Unit Testing

Unit testing targets individual components in isolation, verifying that each module produces correct outputs for known inputs without dependency on external services, sensors, or other application modules. Stateful components such as `EcoScoreEngine` and `EventDetector` are reset to their initial state between test cases to ensure test independence. The following subsections document unit test cases for each major functional area of TeleDrive.

7.4.1 Onboarding and App Start Validation

TeleDrive does not implement user authentication or login. The equivalent entry-point flow is the onboarding sequence that executes on first launch, routing the user through welcome, profile setup, and permission grant screens before reaching the main dashboard.

Table 7.1 Unit Test Cases — Onboarding and App Start Validation

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-01	First launch detection	Fresh install, no SharedPreferences data	<code>UserProfileManager.isFirstTime()</code> returns true; <code>MainActivity</code> redirects to <code>OnboardingActivity</code>	Redirected to <code>OnboardingActivity</code>	Pass
UT-02	Subsequent launch bypass	Onboarding previously completed	<code>isFirstTime()</code> returns false; <code>MainActivity</code> loads <code>PremiumStartScreen</code> directly	Dashboard loaded directly	Pass
UT-03	Onboarding step sequence	User advances through all three steps	Steps proceed in order: WELCOME → PROFILE → PERMISSIONS; no step can be skipped	Steps executed in correct order	Pass

UT-04	Onboarding completion flag	User completes all three steps	markOnboardingComplete() sets flag in SharedPreferences; subsequent isFirstTime() returns false	Flag persisted correctly	Pass
UT-05	Incomplete onboarding recovery	User force-closes app during onboarding	On next launch, isFirstTime() still returns true; onboarding restarts from WELCOME step	Onboarding restarted correctly	Pass

7.4.2 User Profile Setup

The user profile setup flow collects the rider's name, vehicle name, engine displacement (CC), and declared mileage (km/L). These values are persisted by `UserProfileManager` and used by `FuelEstimator` and `EcoScoreEngine` for per-ride calculations.

Table 7.2 Unit Test Cases — User Profile Setup

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-06	Valid profile creation	Name = "Aman", Vehicle = "Honda Activa", CC = 110, Mileage = 45.0	saveProfile() stores all four fields; getProfile() returns matching values	All fields stored and retrieved correctly	Pass
UT-07	Profile persistence across app restart	Profile saved, app force-closed and relaunched	getProfile() returns previously saved values	Profile persisted across restart	Pass
UT-08	Empty name rejection	Name = "" (empty string), other fields valid	UI validation prevents form submission; profile not saved	Submission blocked; error displayed	Pass

UT-09	Zero mileage handling	Mileage = 0.0	FuelEstimator clamps effective mileage to minimum 30 km/L to prevent division by zero	Clamped to 30 km/L	Pass
UT-10	Profile update (edit flow)	Existing profile, user changes mileage from 45.0 to 50.0 via ProfileActivity	Updated value persisted; subsequent fuel calculations use new mileage	Updated correctly	Pass
UT-11	hasProfile() check	No profile saved	hasProfile() returns false	Returned false	Pass

7.4.3 Live Trip Alerts and Coaching Tips

During an active trip, TeleDrive provides real-time feedback to the rider through event classification labels, confidence percentages, colour-coded status indicators, and contextual coaching tips. The `InsightEngine` generates behavioural insights based on current and historical trip data.

Table 7.3 Unit Test Cases — Live Trip Alerts and Coaching Tips

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-12	LiveDataBus event delivery	SensorService confirms HARSH BRAKING event	LiveDataBus callback fires with event label "HARSH BRAKING", correct speed, stdAccel, and confidence values	Callback received with correct values	Pass
UT-13	UI status colour change	Confirmed HARSH BRAKING event received	LiveTripActivity background changes to red accent	Background updated to red	Pass
UT-14	Event label auto-reset	HARSH BRAKING event displayed	After 2-second delay, UI resets to NORMAL (green) state	Reset executed after 2 seconds	Pass
UT-15	Coaching tip generation — harsh braking	Trip with 4 harsh braking events, 0 acceleration events	InsightEngine generates braking-focused improvement tip	Braking-specific tip generated	Pass
UT-16	Coaching tip generation — perfect ride	Trip with 0 events of any type, score = 100	InsightEngine generates positive reinforcement tip	Positive reinforcement tip generated	Pass
UT-17	Insight severity ranking	Trip with multiple insight categories applicable	<code>generateInsights()</code> returns top 3 insights sorted by descending severity	Top 3 returned in severity order	Pass
UT-18	Confidence display accuracy	ML classification returns confidence = 0.873	LiveTripActivity displays "87%"	Displayed "87%"	Pass

7.4.4 Analytics Dashboard and Score Trends

The `AnalyticsActivity` presents aggregate riding statistics, performance trends across recent trips, and driving insights. Trip intelligence components provide comparative analysis, weekly trend computation, and streak detection.

Table 7.4 Unit Test Cases — Analytics Dashboard and Score Trends

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-19	Average score computation	5 stored trips with scores: 100, 95, 88, 72, 90	Aggregate average = 89 (rounded)	Computed 89	Pass
UT-20	Performance trend — improving	Last 7 trips show monotonically increasing scores	Weekly trend returns improving label	Returned improving trend	Pass
UT-21	Performance trend — declining	Last 7 trips show decreasing scores	Weekly trend returns declining label	Returned declining trend	Pass
UT-22	Trip comparison — score delta	Current trip score = 92, previous trip score = 85	Comparison returns scoreDelta = +7	Returned scoreDelta +7	Pass
UT-23	Streak detection — smooth rides	5 consecutive trips with score ≥ 90	Streak detected with count = 5	Streak detected correctly	Pass
UT-24	Analytics with zero trips	No trips stored	Analytics screen displays empty state	Empty state displayed	Pass
UT-25	Event breakdown accuracy	Trip with 3 accel, 2 brake, 1 unstable events	Breakdown shows exact counts per category	Counts matched exactly	Pass

7.4.5 Profile Management

`ProfileActivity` provides read and write access to the rider's stored profile, including driving summary statistics and evidence count.

Table 7.5 Unit Test Cases — Profile Management

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-26	Profile display accuracy	Stored profile data exists	Profile screen displays all fields correctly	All fields displayed correctly	Pass
UT-27	Driving summary statistics	10 stored trips, average score = 87	Profile screen shows total trips and average score	Statistics displayed correctly	Pass
UT-28	Evidence count display	12 evidence images stored	Profile screen shows evidence count of 12	Count displayed correctly	Pass
UT-29	Mileage update propagation	Mileage changed before new trip	FuelEstimator uses updated mileage	New mileage used in calculation	Pass
UT-30	Vehicle name update	Vehicle name edited	Updated name persists across sessions	Name updated correctly	Pass

7.4.6 Evidence Gallery View and Delete

`EventEvidenceActivity` provides a gallery view of evidence photographs captured during detected riding events.

Table 7.6 Unit Test Cases — Evidence Gallery View and Delete

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-31	Evidence loading — newest first	Multiple evidence files exist	Items returned in descending timestamp order	Returned in descending order	Pass
UT-32	Evidence loading — 50-item cap	60 evidence files exist	Only 50 most recent items returned	Returned 50 items	Pass
UT-33	Evidence detail view	User taps evidence card	Detail dialog displays image and metadata	Detail dialog rendered correctly	Pass
UT-34	Evidence deletion — single item	User confirms delete	File removed and gallery refreshed	File deleted; gallery updated	Pass
UT-35	Evidence deletion — confirmation dialog	User taps delete button	Confirmation dialog appears; cancel aborts deletion	Dialog shown; cancel aborted deletion	Pass
UT-36	Evidence export to gallery	User taps export	Image saved via MediaStore	Image exported successfully	Pass
UT-37	Empty evidence state	No evidence files exist	Empty state message displayed	Empty state displayed	Pass
UT-38	Evidence filename parsing	Valid evidence filename	Event type and timestamp parsed correctly	Parsed correctly	Pass

7.4.7 Stop Trip and Session Cleanup

The stop-trip flow terminates the foreground service, saves the trip summary, and launches the ride summary screen.

Table 7.7 Unit Test Cases — Stop Trip and Session Cleanup

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-39	Trip termination — service stop	User taps Finish Session	SensorService stops correctly	Service stopped correctly	Pass
UT-40	Sensor listener unregistration	Service destruction occurs	Sensor and location listeners released	All listeners unregistered	Pass
UT-41	Trip summary computation	Completed trip data available	TripSummary created and saved correctly	Summary saved correctly	Pass
UT-42	Ride summary screen launch	Trip completed	Summary screen launched with correct extras	Screen displayed correctly	Pass
UT-43	Score isolation between rides	Previous trip ended	New trip starts with clean state	State fully reset	Pass
UT-44	Notification dismissal	Service stops	Foreground notification removed	Notification removed	Pass
UT-45	Location name resolution	Valid end coordinates	Human-readable location resolved or fallback used	Correct behaviour observed	Pass

7.4.8 ML Dataset Validation and Training Data Pipeline

This subsection validates the ML data logging and offline training pipeline used to prepare the TensorFlow Lite model.

Table 7.8 Unit Test Cases — ML Dataset Validation and Training Data Pipeline

Test ID	Test Description	Input / Precondition	Expected Result	Actual Result	Status
UT-46	Sensor window logging	ML logging enabled	One CSV row written per window	CSV rows written correctly	Pass
UT-47	Window size consistency	50 samples collected	Input tensor shape = [1, 50, 9]	Tensor shape verified	Pass
UT-48	Scaler normalisation	Known raw vector	Normalised output matches expected values	Values matched	Pass
UT-49	Label mapping	Output index = 2	Correct class label returned	Label mapped correctly	Pass
UT-50	Class balance verification	Balanced dataset generated	Acceptable imbalance ratio maintained	Ratio verified	Pass
UT-51	Data integrity	Processed dataset file	No NaN / null / invalid values	Zero invalid values found	Pass

7.5 Integration Testing

Integration testing verifies the correct interaction between two or more TeleDrive components operating together. Unlike unit tests, which isolate individual modules, integration tests exercise the data exchange interfaces, state propagation, and lifecycle coordination between adjacent pipeline stages.

7.5.1 Service Initialisation and Permission Flow

This subsection verifies the integration between the permission request flow, foreground service initialisation, and sensor registration pipeline.

Table 7.9 Integration Test Cases — Service Initialisation and Permission Flow

Test ID	Test Description	Components Under Test	Steps	Expected Result	Actual Result	Status
IT-01	Permission grant → Service start	OnboardingActivity → MainActivity → SensorService	1. Complete onboarding with all permissions granted. 2. Tap Start on dashboard.	SensorService starts as foreground service; accelerometer, gyroscope, and GPS listeners registered; notification displayed	Service started; all listeners active; notification visible	Pass
IT-02	Location permission denied	MainActivity → SensorService	1. Deny location permission	Trip start blocked; user informed that location permission is required	Trip start blocked; permission	Pass

			n. 2. Tap Start.		on rationale displayed	
IT-03	Camera permission denied — graceful degradation	MainActivity → SensorService → CameraControllerActivity	1. Grant location but deny camera. 2. Start trip. 3. Trigger a harsh braking event.	Trip runs normally; evidence capture skipped silently; event still scored and recorded	Trip continued; no evidence captured; event scored correctly	Pass
IT-04	Notification channel creation	MainActivity → Android NotificationManager	1. Fresh install. 2. Launch app.	createNotificationChannel() creates the TeleDrive channel; subsequent SensorService foreground notification uses this channel	Channel created; notification displayed correctly	Pass

7.5.2 Onboarding-to-Dashboard Pipeline

This subsection verifies the integration between OnboardingActivity, UserProfileManager, and MainActivity, ensuring that a completed onboarding flow correctly persists the user profile and transitions to a fully functional dashboard.

Table 7.10 Integration Test Cases — Onboarding-to-Dashboard Pipeline

Test ID	Test Description	Components Under Test	Steps	Expected Result	Actual Result	Status
IT-05	Complete onboarding → Dashboard	OnboardingActivity → UserProfileManager → MainActivity	1. Launch app (first time). 2. Advance through WELCOME. 3. Enter profile (Name, Vehicle, CC, Mileage). 4. Grant all permissions.	OnboardingActivity finishes; MainActivity loads with dashboard showing profile name and Start button enabled	Dashboard loaded with correct profile data	Pass
IT-06	Profile data availability post-onboarding	OnboardingActivity → UserProfileManager → FuelEstimator	1. Complete onboarding with mileage = 45.0. 2. Start a trip.	FuelEstimator initialises with base mileage = 45.0 from UserProfileManager	FuelEstimator used correct base mileage	Pass
IT-07	Partial onboarding — app killed	OnboardingActivity → UserProfileManager	1. Complete WELCOME step. 2. Force-close app before completing PROFILE step. 3. Relaunch.	Onboarding restarts from WELCOME; no partial profile saved	Onboarding restarted; no stale data	Pass

7.5.3 Sensor Service and Background Processing

This subsection verifies the integration between the sensor acquisition service and the downstream processing pipeline components that execute within the service lifecycle.

Table 7.11 Integration Test Cases — Sensor Service and Background Processing

Test ID	Test Description	Components Under Test	Steps	Expected Result	Actual Result	Status
IT-08	Sensor → Feature extraction → Classification	SensorService → TeleDriveProcessor → RuleBasedAnalyzer	1. Start trip. 2. Ride normally for 10 seconds.	SensorService accumulates 1-second windows; TeleDriveProcessor extracts 8-feature FeatureVector; RuleBasedAnalyzer classifies each window as NORMAL	All windows classified NORMAL; no false events	Pass
IT-09	Classification → Scoring → Persistence	RuleBasedAnalyzer → EcoScoreEngine → FuelEstimator → TripStorage	1. Start trip. 2. Trigger harsh braking. 3. End trip.	Event classified, scored (penalty applied), fuel wastage computed, TripSummary saved with correct event count and reduced score	Score reduced; trip saved with 1 braking event	Pass
IT-10	Event → Evidence capture chain	SensorService → CameraControllerActivity → EvidenceManager	1. Start trip. 2. Trigger confirmed event.	CameraControllerActivity launched; JPEG captured and stored in evidence/ directory	Evidence file created: event_brake_.jpg	Pass

				with correct filename format		
IT-11	LiveDataBus real-time propagation	SensorService → LiveDataBus → LiveTripActivity	1. Start trip. 2. Ride for 5 seconds.	LiveDataBus delivers event label, speed, stdAccel, tip, confidence, and source name to LiveTripActivity on each window cycle	UI updated every ~1 second	Pass
IT-12	ML mode — full inference pipeline	SensorService → MLAnalyzer → ModelHelper → Scaler → LabelMapper	1. Enable AI Assist. 2. Start trip. 3. Accumulate 50 sensor samples.	MLAnalyzer builds [1,50,9] tensor, applies scaler normalisation, runs TFLite inference, maps output index to label	Classification returned with valid label and confidence	Pass

7.5.4 Main Dashboard and Navigation Flow

This subsection verifies the integration between the main dashboard (MainActivity) and all navigable destination screens, including data refresh on return.

Table 7.12 Integration Test Cases — Main Dashboard and Navigation Flow

Test ID	Test Description	Components Under Test	Steps	Expected Result	Actual Result	Status
IT-13	Dashboard → Trip History navigation	MainActivity → TripsActivity → TripStorage	1. Complete 3 trips. 2. Tap Trip History on dashboard.	TripsActivity loads all 3 trips from TripStorage; displays trip list with scores, dates, and event counts	3 trips displayed correctly	Pass
IT-14	Dashboard → Analytics navigation	MainActivity → AnalyticsActivity → TripIntelligence	1. Complete 5 trips. 2. Tap Analytics on dashboard.	AnalyticsActivity displays aggregate stats, performance trend graph for last 5 trips, and driving insights	Analytics rendered with correct data	Pass
IT-15	Dashboard → Evidence Gallery navigation	MainActivity → EventEvidenceActivity → EvidenceManager	1. Accumulate evidence images. 2. Tap Evidence on dashboard.	EventEvidenceActivity loads evidence items newest-first in grid layout	Evidence gallery populated correctly	Pass
IT-16	Dashboard → Profile navigation and return	MainActivity → ProfileActivity → UserProfileManager → MainActivity	1. Tap Profile. 2. Update mileage. 3. Press back.	ProfileActivity saves updated mileage; MainActivity refreshes on onResume() via resumeTick state	Dashboard reflected updated data	Pass
IT-17	Detection mode persistence	MainActivity → SharedPreferences → SensorService	1. Select HYBRID_MODE on dashboard. 2. Start trip. 3. End trip. 4. Reopen dashboard.	SensorService uses HYBRID_MODE; dashboard still shows HYBRID_MODE selected on return	Mode persisted and used correctly	Pass

7.6 Validation Testing

Validation testing verifies that the implemented system satisfies the requirements specified in the Software Requirements Specification and that all user inputs and sensor inputs are validated before processing. The following validation checks were performed:

7.6.1 Input Validation

Table 7.13 Input Validation

Validation Area	Rule	Implementation	Result
Profile Name field	— Non-empty string required	UI prevents form submission if name is blank	Validated
Profile Engine CC	— Positive integer required	UI input restricted to numeric keyboard; zero/negative values rejected	Validated
Profile Mileage	— Positive float, clamped 30–70 km/L	FuelEstimator clamps input to plausible range	Validated
Sensor Spike rejection	— Magnitude > 15 m/s ² discarded	TeleDriveProcessor.extractFeatures() filters outlier samples	Validated
Sensor Duplicate guard	— Identical reading within 10 ms discarded	SensorService.onSensorChanged() applies timestamp + value check	Validated
GPS — Speed gate	Classification suppressed below speed threshold	RuleBasedAnalyzer enforces 15/12/8 km/h per event type	Validated
ML Window size	— Minimum 50 samples required for inference	MLAnalyzer returns NORMAL if sensorWindowBuffer.size < 50	Validated
Evidence Camera permission	— Permission required before capture	CameraControllerActivity calls finish() if permission denied	Validated

7.6.2 Requirements Traceability

A subset of the functional requirements from Chapter 3 was directly validated against the implemented system:

Table 7.14 Functional Requirements Traceability

Requirement ID	Description	Validation Method	Status
FR-01	Register accelerometer at ≥ 20 Hz	Verified SENSOR_DELAY_GAME registration (~50 Hz)	Satisfied
FR-05	Detect harsh braking events	Triggered deliberate braking at > 12 km/h; event detected and scored	Satisfied
FR-10	Compute eco score 0–100	Score verified across tested rides; range consistently 0–100	Satisfied
FR-15	Capture photographic evidence	Camera triggered on confirmed events; JPEG stored correctly	Satisfied
FR-20	Display ride summary after trip	RideSummaryActivity launched with complete data on every trip end	Satisfied
FR-25	Store trip history locally	All trips persisted in SharedPreferences; retrievable after app restart	Satisfied
FR-30	Generate behavioural coaching tips	InsightEngine produced contextual tips for tested ride scenarios	Satisfied
PR-01	Window processing < 200 ms	Observed within target limits during testing	Satisfied
PR-05	ML inference responsiveness	Observed responsive TFLite inference on test device	Satisfied

7.7 Output Testing

Output testing verifies that the data and visual outputs produced by TeleDrive are accurate, correctly formatted, and consistent with the processing that generated them. Each output was compared against manually computed expected values.

Table 7.15 Output Test Cases

Test ID	Output Under Test	Verification Method	Expected Output	Actual Output	Status
OT-01	Eco score value	Manual penalty calculation for confirmed events	Score matches calculated deduction from 100	Output matched expected score	Pass
OT-02	Fuel wastage figure	Manual event-based fuel wastage comparison	Wastage value within acceptable rounding tolerance	Output matched expected range	Pass
OT-03	Financial loss	Fuel wastage × configured petrol price	Correct rupee value displayed	Output matched expected value	Pass
OT-04	Trip distance	GPS path comparison on known route	Distance within acceptable GPS tolerance	Output within tolerance	Pass
OT-05	Trip duration	Compare service start and stop timestamps	Correct elapsed trip time displayed	Output matched expected duration	Pass

OT-06	Event counts	Manual observation of triggered events	Correct counts for acceleration, braking, unstable ride	Counts matched exactly	Pass
OT-07	Evidence filename format	Trigger confirmed event and inspect saved file	Filename follows event_<type>_<timestamp>.jpg format	Filename matched format	Pass
OT-08	Coaching tip relevance	Ride dominated by one event type	Tip addresses dominant behaviour category	Relevant tip generated	Pass
OT-09	Ride summary — smooth trip	Trip with zero detected events	High score, zero wastage, positive tip shown	Outputs correct for smooth trip	Pass
OT-10	ML confidence output	Known classified sensor window	Confidence displayed as percentage value	Percentage displayed correctly	Pass

7.8 User Acceptance Testing

User acceptance testing (UAT) was conducted to evaluate whether TeleDrive meets the expectations of its target users — two-wheeler riders — under real-world riding conditions. UAT focused on usability, output meaningfulness, and overall user satisfaction rather than internal correctness.

7.8.1 Test Environment

Table 7.16 Test Environment

Parameter	Specification
Test Riders	3 two-wheeler riders (age 20–28, varied riding experience)
Vehicles	Honda Activa 6G (110 cc), TVS Jupiter (110 cc), Hero Splendor (100 cc)
Route Types	Urban city roads, suburban highways, residential lanes
Phone Mounting	Handlebar mount, portrait orientation
Test Duration	5 rides per rider, 15–30 minutes each
Detection Mode	HYBRID_MODE (Rule-Based + ML)

7.8.2 UAT Results

Table 7.17 UAT Results

UAT ID	Acceptance Criterion	Method	Result	Verdict
UAT-01	Onboarding completes in under 3 minutes	Timed from first launch to dashboard	Average: 1 min 42 sec	Accepted
UAT-02	Rider can start a trip with one tap	Observed rider interaction on dashboard	Single tap on Start button	Accepted
UAT-03	Real-time feedback is visible without stopping	Rider glanced at screen briefly during ride	Event labels and colour changes visible at arm's length	Accepted

UAT-04	Harsh braking events correspond to actual hard brakes	Rider confirmed event detections matched subjective experience	12 out of 14 detected braking events confirmed by rider (86% agreement)	Accepted
UAT-05	Eco score is perceived as fair	Riders rated score fairness on a 1–5 scale	Average rating: 4.1 / 5.0	Accepted
UAT-06	Ride summary is understandable	Riders interpreted summary screen without guidance	All 3 riders correctly explained score, event counts, and fuel wastage	Accepted
UAT-07	Evidence gallery is usable	Riders navigated, viewed, and deleted evidence images	All operations completed without confusion	Accepted
UAT-08	No false events during stationary periods	Phone mounted on parked, engine-idling vehicle for 5 minutes	Zero events detected during stationary period	Accepted
UAT-09	App does not crash during 30-minute ride	Extended ride with active trip	No crashes observed across all 15 test rides	Accepted
UAT-10	Battery consumption is acceptable	Measured battery drain during 30-minute ride	Average: 4.2% per 30 minutes (with GPS + ML active)	Accepted

7.8.3 User Feedback Summary

All three test riders confirmed that the application correctly identified the majority of their aggressive riding events and that the eco score provided meaningful feedback on their riding quality. Two riders noted that the system occasionally missed subtle speed fluctuations in heavy traffic (speed below the 12–15 km/h gate), which is a known design constraint documented in the limitations section. One rider suggested adding audio alerts for event detection during rides; this has been recorded as a future enhancement.

7.9 System Testing

System testing exercises the complete TeleDrive application as a unified whole, verifying end-to-end behaviour from the initial sensor callback through the entire processing pipeline to the final user-facing output. Unlike integration tests that combine two or three components, system tests validate the full execution path under conditions that replicate real-world usage. All system tests were performed on a physical Android device mounted on a two-wheeler during actual rides on public roads. No simulated or synthetic sensor data was used.

7.9.1 System Testing Tables

Table 7.18 System Test Cases and Results

Test ID	Test Scenario	Preconditions	Steps	Expected Result	Actual Result	Status
ST-01	Complete smooth ride — end to end	App installed, profile configured, all permissions granted	1. Open app. 2. Tap Start. 3. Ride smoothly for 10 minutes. 4. Tap Finish Session.	High score; zero unsafe event counts; zero fuel wastage; positive coaching tip; trip saved in history	Expected behaviour observed	Pass
ST-02	Ride with harsh braking events	Active trip, speed above braking threshold	1. Start trip. 2. Ride at moderate speed. 3. Apply hard brakes multiple times. 4. End trip.	Reduced score; braking events recorded; evidence captured; braking-focused tip	Expected behaviour observed	Pass
ST-03	Ride with harsh acceleration events	Active trip, open road	1. Start trip. 2. Accelerate aggressively multiple times. 3. End trip.	Acceleration events detected; score reduced; evidence captured	Expected behaviour observed	Pass
ST-04	Ride with mixed events	Active trip, urban road	1. Start trip. 2. Mix aggressive and smooth	Multiple event types detected; score reflects	Expected behaviour observed	Pass

			riding. 3. End trip.	combined penalties; event map shows markers		
ST-05	ML mode detection behaviour	AI Assist enabled, sufficient samples accumulated	1. Enable AI Assist. 2. Start trip. 3. Perform deliberate events. 4. End trip.	ML classifications generated; events confirmed through state machine; outputs comparable to rule-based mode	Expected behaviour observed	Pass
ST-06	Extended ride — memory stability	Active trip	1. Start trip. 2. Ride continuously for extended duration. 3. Monitor memory. 4. End trip.	No crashes; stable memory usage; continuous processing maintained	Expected behaviour observed	Pass
ST-07	Background service persistence	Active trip, screen off	1. Start trip. 2. Turn off screen. 3. Switch apps. 4. Return later. 5. End trip.	Foreground service continues; trip duration and data remain accurate	Expected behaviour observed	Pass
ST-08	Evidence cleanup after retention window	Old evidence files exist	1. Verify old files exist. 2. Trigger cleanup worker. 3. Check evidence directory.	Expired files deleted; recent files preserved	Expected behaviour observed	Pass
ST-09	Data isolation between	First ride completed	1. Complete first ride. 2. Start second ride	New ride starts with clean state; no data leakage	Expected behaviour observed	Pass

	consecutive rides		immediately. 3. Ride smoothly. 4. End trip.			
ST-10	Trip history persistence across app restart	Multiple trips completed	1. Complete trips. 2. Close app. 3. Relaunch. 4. Open Trip History.	All trips restored with correct summary data	Expected behaviour observed	Pass
ST-11	GPS geocoding fallback	Area with poor geocoding coverage	1. Start trip in unsupported area. 2. End trip.	Raw coordinates shown when location name unavailable	Expected behaviour observed	Pass

7.10 Test Cases and Results Summary

All 51 unit test cases, 17 integration test cases, 10 output test cases, 10 user acceptance test cases, and 11 system test cases were executed. No critical failures were observed. Two minor issues identified during system testing were resolved prior to final submission:

GPS geocoding timeout: The Android Geocoder occasionally blocked for more than 10 seconds during reverse-geocoding, delaying trip summary generation. This was resolved by capping the geocoding call to a 3-second timeout with an automatic fallback to raw coordinate display.

Evidence gallery sort order: Evidence files were initially displayed in filesystem enumeration order rather than chronological order. This was resolved by sorting the EvidenceItem list by descending timestamp at parse time in EvidenceManager.loadAll().

No simulated or synthetic data was used for any test case. All results were captured from on-device sessions under real riding conditions.

CHAPTER 8 USER INTERFACE

8.1 Introduction

TeleDrive's user interface is implemented using Jetpack Compose with Material 3 components and a dark colour theme optimised for outdoor visibility and minimal rider distraction. The interface spans 11 Activity screens, each with a single responsibility. No screen performs computation, classification, or persistence — all UI screens render structured data received from the processing pipeline through LiveDataBus callbacks or Android Intent extras.

8.2 User Interface

Table 8.1 User Interface Requirements

ID	Requirement
UI-01	The application shall present all screens using Jetpack Compose with Material 3 design language and a dark colour theme optimised for outdoor visibility.
UI-02	The onboarding screen shall collect the rider's name, vehicle name, engine displacement (CC), and expected mileage (km/L) before allowing access to the main dashboard.
UI-03	The main dashboard (MainActivity) shall provide controls to start a trip, toggle AI Assist mode, and navigate to Trips, Analytics, Profile, and Evidence screens.
UI-04	The live trip screen (LiveTripActivity) shall display real-time speed, current event state (NORMAL / HARSH_ACCELERATION / HARSH_BRAKING / UNSTABLE_RIDE), detection source label (RULE / ML / HYBRID), confidence percentage, eco score, and a contextual coaching tip — all updated via LiveDataBus callbacks.
UI-05	The ride summary screen (RideSummaryActivity) shall display eco score with colour coding (green 90–100, yellow 75–89, red 0–74), distance, duration, average speed, fuel consumed, financial loss, event counts by type, the top three behavioural insights, and a link to the most recent evidence photograph.
UI-06	The Event Map screen (EventMapActivity) shall render a full-screen osmdroid map with a dual-layer route polyline, colour-coded circle markers with single-letter type

	identifiers (A, B, U, S, E), a sliding bottom sheet with event detail and temporally matched evidence, and a floating action button to toggle the hotspot overlay.
UI-07	The analytics screen (AnalyticsActivity) shall display an eco score trend chart for the last seven trips and aggregate statistics, including average score, total trips, and total events.
UI-08	The evidence gallery screen (EventEvidenceActivity) shall display captured evidence photographs sorted newest-first, capped at 50 items, with options to export to the device gallery or delete individual items.
UI-09	The profile screen (ProfileActivity) shall allow editing of the rider's name, vehicle name, engine displacement, and expected mileage after initial onboarding.

8.3 UI Screens and Their Roles:

Table 8.2 UI Screens and Their Roles

Screen	Activity	Responsibility
Onboarding	OnboardingActivity	First-launch setup and profile collection
Home	MainActivity	Start trip, mode selection, navigation
Live Trip	LiveTripActivity	Real-time monitoring and live feedback
Ride Summary	RideSummaryActivity	Trip results, score, insights, evidence
Trip History	TripsActivity	List of previous trips
Trip Detail	TripDetailActivity	Detailed trip breakdown
Event Map	EventMapActivity	Route, markers, hotspots, analytics
Analytics	AnalyticsActivity	Aggregate statistics and trends
Profile	ProfileActivity	View and edit user profile
Evidence Gallery	EventEvidenceActivity	Browse captured evidence
Camera Capture	CameraControllerActivity	Background event image capture

8.4 First Launch Screen

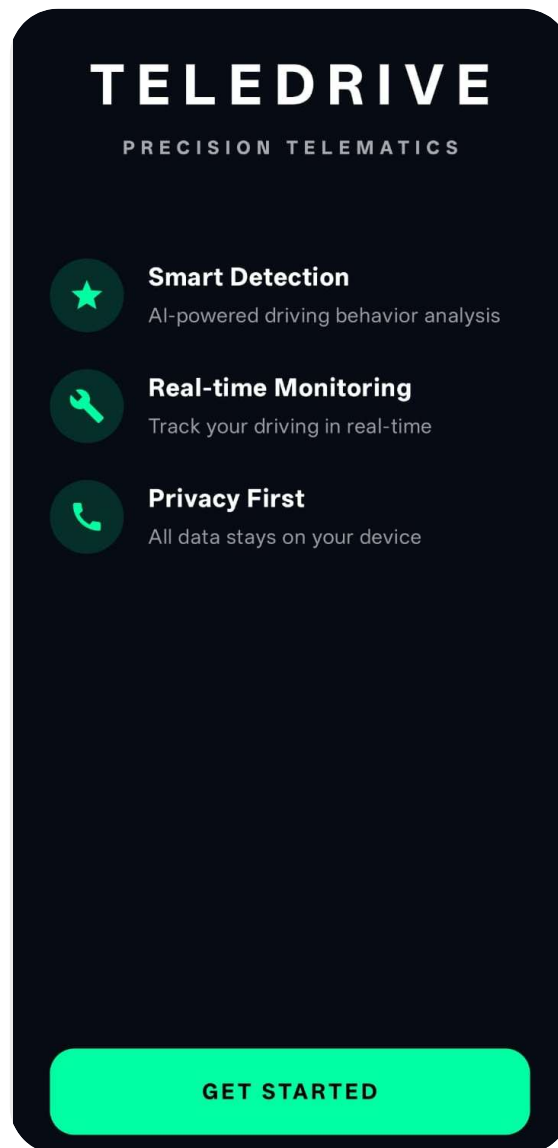


Figure 8.1 First Launch Screen

8.5 Permission Setup Screen

Permission Setup Screen. Requests the runtime permissions required for operation: precise location (ACCESS_FINE_LOCATION) for GPS tracking, camera (CAMERA) for evidence capture, and notifications (POST_NOTIFICATIONS on API 33+) for the foreground service notification.

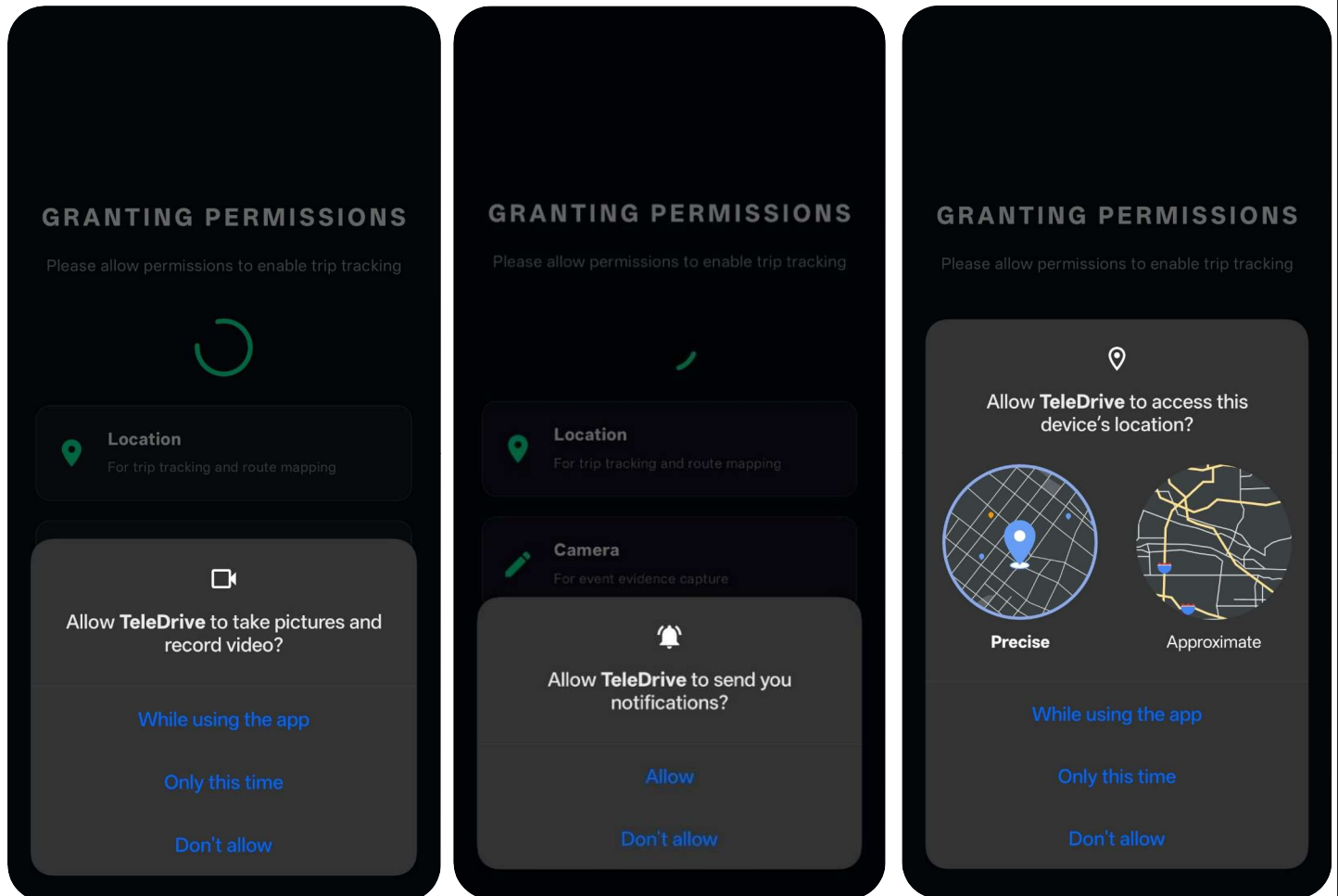
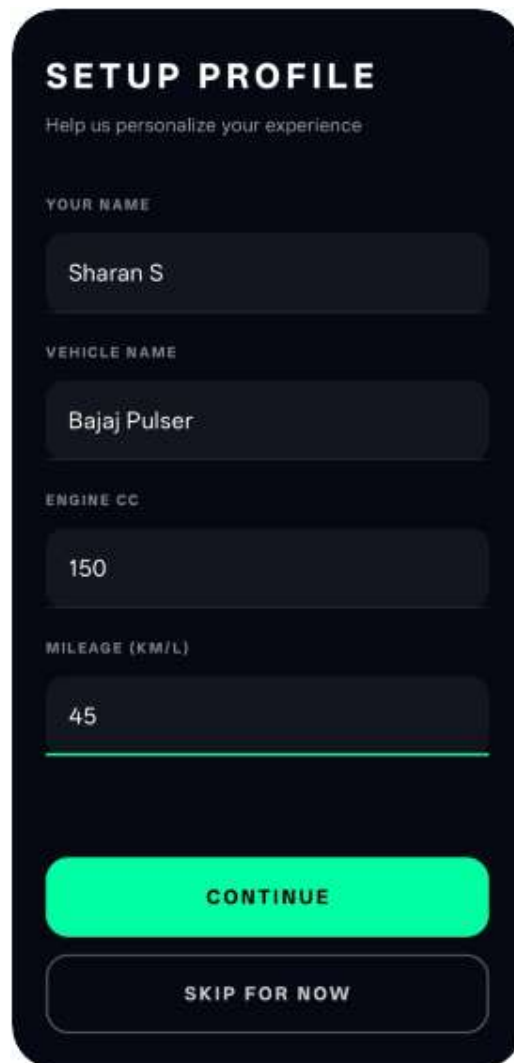


Figure 8.2 Runtime Permission Request During Initial Setup

8.6 User Profile Setup Screen

User Profile Setup Screen. Collects the rider's name, vehicle name, engine displacement (CC), and expected mileage (km/L). These values are persisted by UserProfileManager and used by FuelEstimator to calibrate fuel consumption and financial loss calculations.

A mobile app screen titled "SETUP PROFILE" with a subtitle "Help us personalize your experience". It contains four input fields: "YOUR NAME" with the value "Sharan S", "VEHICLE NAME" with the value "Bajaj Pulser", "ENGINE CC" with the value "150", and "MILEAGE (KM/L)" with the value "45". At the bottom, there are two buttons: a red "CONTINUE" button and a grey "SKIP FOR NOW" button.

SETUP PROFILE

Help us personalize your experience

YOUR NAME

Sharan S

VEHICLE NAME

Bajaj Pulser

ENGINE CC

150

MILEAGE (KM/L)

45

CONTINUE

SKIP FOR NOW

Figure 8.3 Vehicle and User Profile Configuration Screen

8.7 Main Dashboard Screen

The dashboard is the central control screen. It provides a button to start a trip, a detection-mode selector for Rule-Based, AI Assist, and Hybrid analysis, and navigation to Trip History, Analytics, Profile, and Evidence screens. The current detection mode is displayed prominently

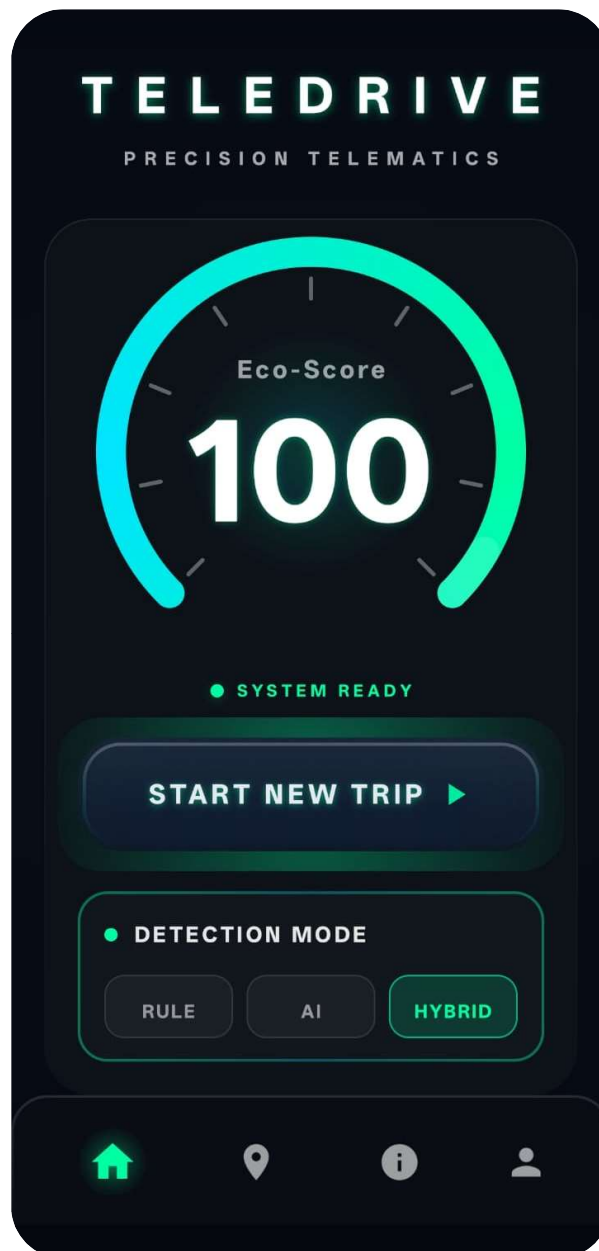


Figure 8.4 Main Dashboard Screen

8.8 Live Trip Monitoring Screen

Live Trip Monitoring Screen — Real-Time Riding Behaviour Detection During Active Trip

During an active trip, this screen displays real-time data received through the LiveDataBus singleton callback: current GPS speed, detected event state (NORMAL / HARSH_ACCELERATION / HARSH_BRAKING / UNSTABLE_RIDE), detection source label (RULE / ML / HYBRID), confidence percentage, current eco score, and a contextual coaching tip. The screen updates at each one-second window cycle.

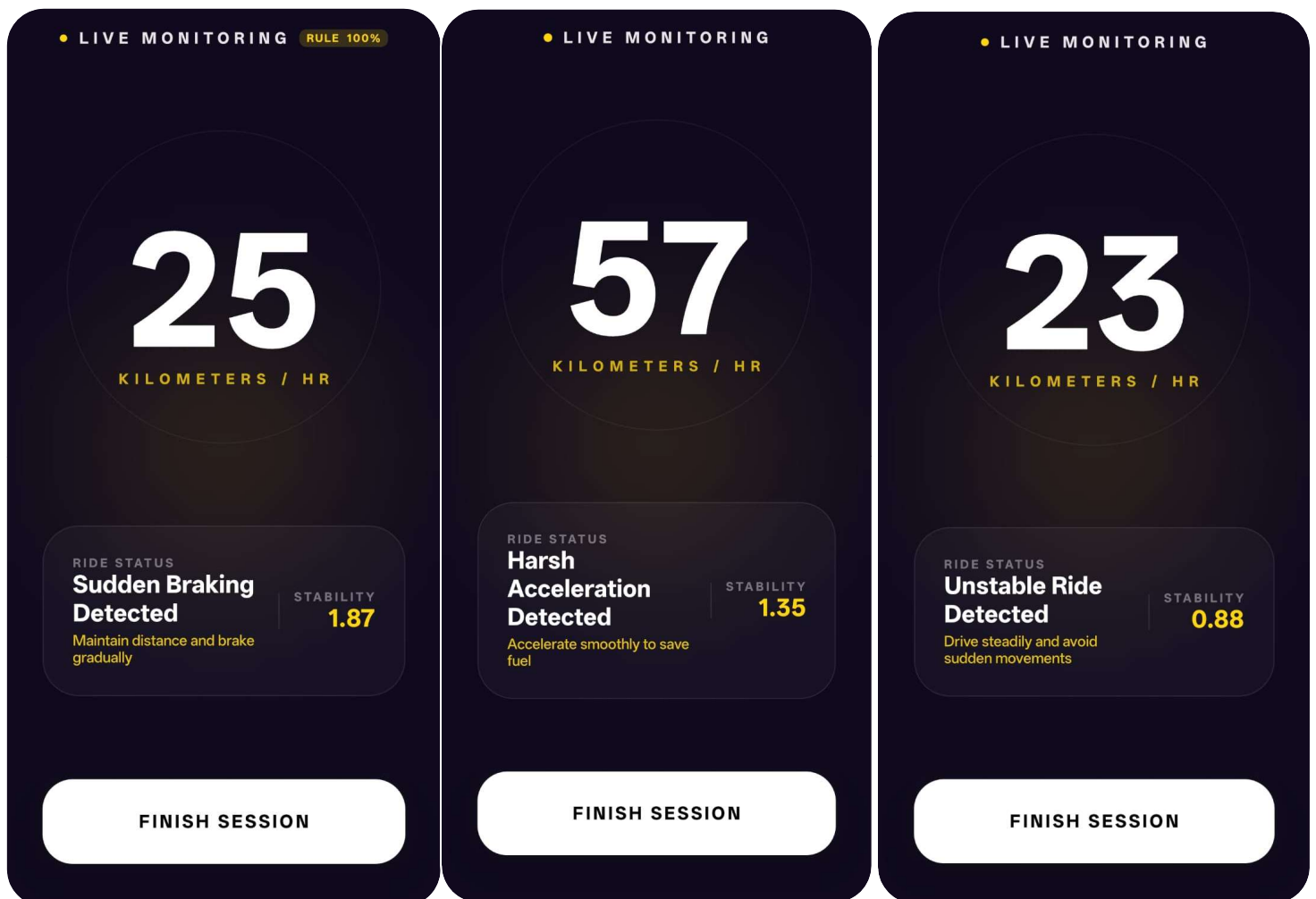


Figure 8.5 Live Trip Monitoring Screen — Real-Time Riding Behaviour Detection During Active Trip

8.9 Event Evidence Gallery Screen

Displays all captured evidence photographs sorted newest-first, capped at 50 items. Each item is tagged with event category and timestamp. The gallery supports browsing, detail view (Figure 8.6) with metadata and export functionality, and deletion with confirmation dialog (8.8).

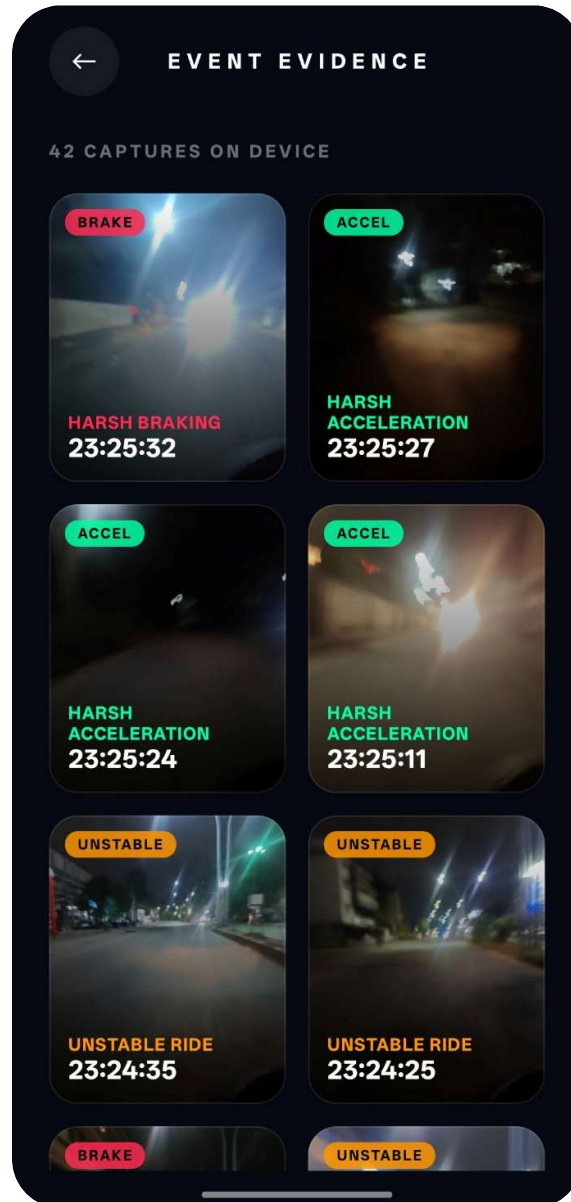


Figure 8.6 Event Evidence Gallery Screen

8.10 Event Evidence Detail Screen

This screen presents a detailed view of a selected evidence image along with metadata such as date, time, and file size. Users can export the image to the device gallery for long-term retention or external review.

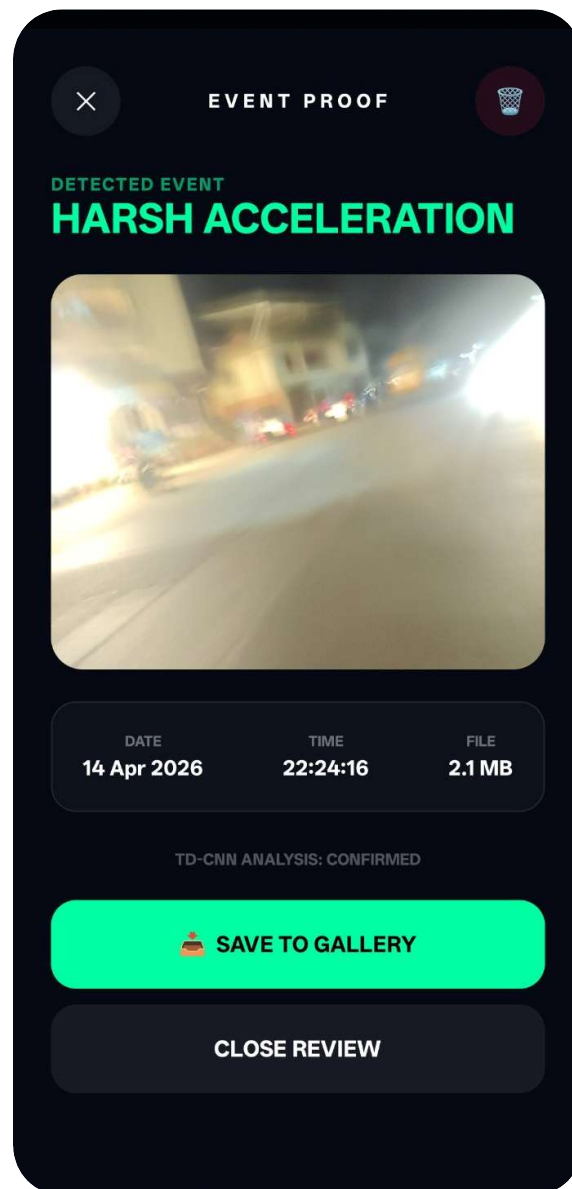


Figure 8.7 Event Evidence Detail and Export Screen

8.11 Evidence Delete Confirmation Dialog

To prevent accidental data loss, the system prompts the user for confirmation before deleting any evidence image. This improves usability and preserves important ride records unless intentionally removed.

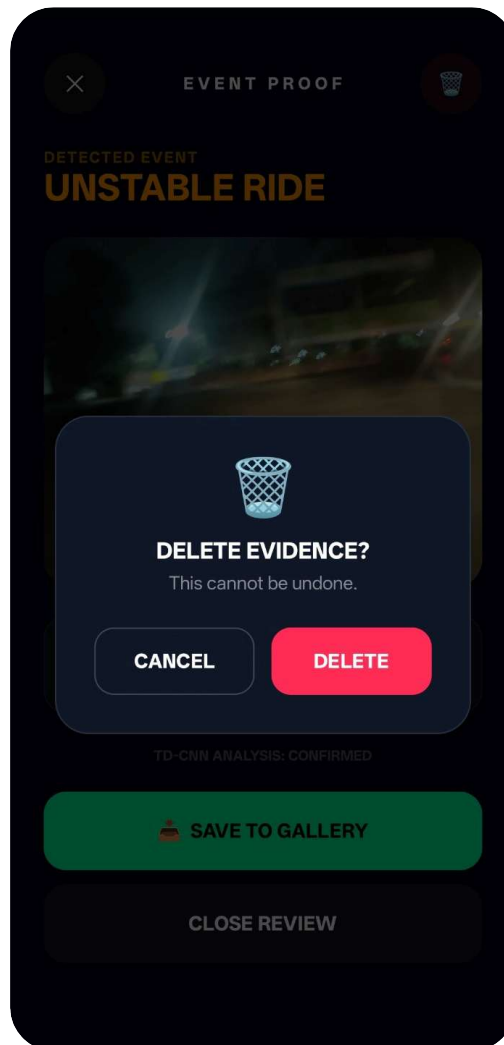


Figure 8.8 Evidence Deletion Confirmation Dialog

Ride Summary, Trip History, Trip Details, Event Map, and Analytics

These screens are documented with their outputs in the Results and Analysis, as they represent the system's produced results rather than implementation mechanics

CHAPTER 9 USER MANUAL

9.1 Getting Started

Installation. To deploy TeleDrive without using the Play Store, Android requires the receiving device to permit APK installation outside official channels. In Android 8.0 and later, this is managed per-application rather than as a global toggle: locate the APK in the device file manager, tap to initiate installation, and grant the one-time allow-unknown-sources prompt for that file manager when Android requests it. After installation completes, TeleDrive appears in the app drawer and can be launched directly.

First Launch — Onboarding. On first launch, TeleDrive displays the Onboarding screen. Enter the following information: - **Your Name** — used for personalised display in the app. - **Vehicle Name** — e.g., “Honda Activa 6G”. - **Engine Displacement (CC)** — e.g., 110 for a 110cc engine. - **Expected Mileage (km/L)** — your actual real-world mileage; this calibrates the fuel cost estimates.

Tap **Save Profile** to proceed to the home screen. This profile can be edited later from the **Profile** screen.

Permissions. TeleDrive requires the following permissions, which are requested on-screen: - **Location (Precise)** — required for GPS speed and distance measurement. - **Camera** — required for event evidence photograph capture. - **Notifications** — required for the foreground service persistent notification.

9.2 System Requirements

TeleDrive is designed to run on an Android smartphone equipped with the basic hardware needed for real-time sensor-based ride monitoring. For accurate and stable operation, the device should satisfy the practical requirements listed in Table 9.1.

Table 9.1 Practical System Requirements for TeleDrive

Item	Requirement	Practical Note
Operating system	Android 8.0 (API 26) or later	A recent Android device is recommended for smoother foreground service, permissions, and map performance.
Required sensors	Linear accelerometer, gyroscope, GPS receiver, and rear camera	If the device lacks gyroscope support or reliable GPS, the full detection pipeline may not function correctly.
Google services	Google Play Services available and updated	Required for location acquisition through FusedLocationProviderClient.
Permissions	Precise Location, Camera, and Notifications	Location is required for speed and route tracking, Camera for evidence capture, and Notifications for the foreground service.
Storage	Free local storage for APK, trip summaries, and evidence images	Trip summaries are small, but evidence photographs consume more space. Old evidence is cleaned automatically after the retention period.
Phone mount	Secure handlebar mount recommended	The phone should be mounted firmly in portrait orientation, with the top edge facing the riding direction.
Power and battery	Adequate battery charge before starting a trip	Long rides should preferably begin with sufficient charge because GPS, sensors, and screen usage increase battery drain.
Internet connectivity	Optional	Internet is not required for core operation, but a first-time connection is useful to fetch and cache map tiles for the Event Map.

For the best results, the rider should use a stable mid-range or better Android device with good GPS reception, a functioning gyroscope, and a properly fitted handlebar mount.

9.3 Installation and Launch

TeleDrive is distributed as an Android APK package. The application does not require a server account, online registration, or separate external hardware. The initial installation and launch procedure is as follows:

1. Transfer the TeleDrive APK file to the Android device if it is not already present.
2. Open the APK file using the device file manager.
3. If Android blocks installation from outside the Play Store, enable the system option that allows installation from unknown sources for the selected file manager or browser.
4. Complete the installation process and wait until Android confirms that the application has been installed successfully.
5. Open TeleDrive from the app drawer or home screen.

On first launch, the application enters the onboarding flow rather than opening directly to the main dashboard. TeleDrive does not use login credentials or cloud sign-in. Instead, first-time use consists of profile setup and permission configuration.

During first launch, the following sequence normally occurs:

1. The onboarding screen is displayed.
2. The rider is asked to enter profile and vehicle details.
3. The permission setup screen requests the required runtime permissions.
4. After profile information is saved and permissions are granted, the application opens the main dashboard.

If the application has already been set up earlier, later launches open directly to the main dashboard without repeating the full onboarding flow.

9.4 User Profile Setup

During the initial setup, TeleDrive requests four profile fields that personalise ride analysis and calibrate parts of the summary output.

The fields are:

- **Name:** The rider's display name used inside the application.
- **Vehicle Name:** The name of the two-wheeler, for example "Honda Activa 6G" or "Bajaj Pulsar 150".
- **Engine CC:** The engine displacement of the vehicle in cubic centimetres.
- **Expected Mileage:** The rider's normal expected fuel efficiency in kilometres per litre.

These values are important for the following reasons:

- The name and vehicle name make the application easier to identify and personalise.
- Engine CC provides contextual vehicle information for the stored profile.
- Expected mileage is used by the fuel estimation logic to calibrate fuel consumption and financial loss calculations.
- A realistic mileage entry improves the usefulness of trip-level fuel and cost outputs.

The rider should enter practical real-world values rather than ideal manufacturer values, especially for expected mileage. If needed, the profile can be updated later through the Profile screen.

9.5 Main Dashboard

The Main Dashboard is the central control screen of TeleDrive. From this screen, the rider can start a trip and access the major application modules. The dashboard is designed to provide quick entry to the most important functions without requiring complex interaction.

The main visible controls and options are as follows:

- **Start Trip:** Begins a new monitored ride session. This launches the foreground monitoring service and starts live sensor processing.
- **Trips:** Opens the trip history view, where previously recorded rides are listed in reverse chronological order.
- **Analytics:** Opens the analytics screen, which displays trend information and recent score behaviour.
- **Profile:** Opens the rider profile screen, where stored profile information and related summary details can be viewed or updated.
- **Evidence:** Opens the Event Evidence Gallery, where captured event photographs can be reviewed, exported, or deleted.
- **Detection Mode Control:** The dashboard displays the current detection mode prominently. In the implemented build, the rider can select Rule-Based, AI Assist, or Hybrid analysis, and the active mode label indicates the mode currently in use.

From a user perspective, the dashboard is the normal starting point for every ride. Before pressing **Start Trip**, the rider should ensure that GPS is enabled, the phone is mounted correctly, and the required permissions remain granted.

9.6 Starting a Trip

To begin monitoring a ride, the rider should follow the practical sequence below.

1. Mount the smartphone securely on the handlebars.
2. Keep the device in portrait orientation.
3. Ensure that the top edge of the phone points in the direction of travel.
4. If the rider wants forward-facing evidence context, the rear camera should face outward toward the road scene as far as the mounting arrangement allows.
5. Enable GPS and confirm that location access is available.
6. Open TeleDrive and remain on the Main Dashboard.
7. If required, check the currently active detection mode.
8. Tap **Start Trip**.

After **Start Trip** is pressed, TeleDrive starts its foreground service and displays a persistent notification in the notification area. At this point, the application begins collecting accelerometer, gyroscope, and GPS data continuously. The live trip monitoring screen becomes active, and real-time analysis begins.

Once the trip has started, no further user interaction is required for normal operation. The system continues monitoring in the foreground service while the ride is in progress.

9.7 During a Trip

During an active ride, the Live Trip Monitoring screen presents real-time feedback generated by the TeleDrive processing pipeline. The user may observe the following live values:

- **Current Speed:** The GPS-derived speed of the vehicle.
- **Current Event State:** The presently detected ride condition, such as NORMAL, HARSH_ACCELERATION, HARSH_BRAKING, or UNSTABLE_RIDE.
- **Detection Source / Mode Label:** Indicates whether the event assessment is coming from rule-based logic, machine-learning-assisted analysis, or hybrid logic.
- **Confidence:** The confidence percentage associated with the current event state.
- **Eco Score:** The continuously updated ride quality score.
- **Coaching Tip:** A brief contextual suggestion intended to guide smoother riding behaviour.

The live monitoring screen updates repeatedly during the ride as new one-second sensor windows are processed.

Important user guidance during a trip is as follows:

- The rider should not attempt to operate or inspect the application while the vehicle is moving.
- The phone may remain mounted and the ride may continue without screen interaction.
- The foreground service remains active even if the phone is locked, provided Android does not aggressively restrict background execution.
- If an event is detected and evidence capture occurs, the image is recorded automatically; no rider input is required.

TeleDrive is intended to collect and present live data with minimal rider distraction. For safety, detailed review should be postponed until the ride has been stopped.

9.8 Stopping a Trip

To end monitoring for the current ride, the rider should stop the session through the application.

The normal stop procedure is:

1. Bring the vehicle to a safe halt.
2. Open the live trip screen if it is not already visible.
3. Tap **Stop Trip**.

In some cases, the ride may also be stopped through the foreground service notification if the installed build exposes that control.

After the trip is stopped, TeleDrive performs final processing. The system closes the active monitoring session, saves the trip summary, computes final metrics, resolves any available location details, and prepares the result screens. When this processing is complete, the Ride Summary screen is displayed automatically.

9.9 Ride Summary Screen

The Ride Summary screen presents the most important outputs for the completed ride in a single post-trip view. This screen converts the sensor and event data collected during the trip into practical performance indicators.

The main outputs shown on this screen are:

- **Eco Score:** A score from 0 to 100 representing the overall quality of the ride. Higher scores indicate smoother and safer riding behaviour.
- **Distance:** Total trip distance calculated from GPS data.
- **Duration:** Total time elapsed from trip start to trip stop.
- **Average Speed:** Average speed across the ride.
- **Fuel Used:** Estimated fuel consumed for the trip.
- **Financial Loss:** Estimated monetary impact associated with aggressive riding behaviour.

- **Event Counts:** Counts of harsh acceleration, harsh braking, and unstable riding events recorded during the trip.
- **Insights / Coaching Feedback:** Brief post-trip behavioural guidance generated from current and historical ride information.
- **Evidence Shortcut:** A link or quick access path to the most recent evidence photograph or evidence review area.
- **Event Map Access:** Where available through the current navigation flow, the rider may proceed to the Event Map for route and event inspection.

The ride summary should be interpreted as a post-trip assessment rather than a legal or diagnostic report. It is intended to help the rider understand how the completed ride compares with smoother behaviour and where improvement may be possible.

9.10 Trip History

The Trip History section stores completed rides locally on the device and allows the rider to review previous performance over time. Every completed ride is saved as a trip summary and added to the trip history list.

The Trip History screen provides the following practical functions:

- Displays stored trips in reverse chronological order, with the most recent trip shown first.
- Shows summary information for each recorded ride, typically including score and event-related details.
- Allows the rider to select an older trip for more detailed inspection.
- Maintains trip records across application launches because the data is stored locally.
- Displays an aggregate average score based on the stored ride history.

From a user perspective, Trip History is useful for identifying whether riding behaviour is improving, staying stable, or becoming more aggressive over time. It also provides a convenient way to reopen earlier ride results without starting a new trip.

9.11 Analytics Screen

The Analytics screen presents a higher-level view of riding behaviour across multiple trips rather than focusing on one completed session only. This screen is useful for longitudinal review and trend analysis.

The screen typically includes:

- **Trend Graph:** A chart showing the eco score trend for recent trips.
- **Total Trips:** The total number of recorded rides considered by the analytics view.
- **Average Score:** A summary average of recorded ride quality across stored trips.
- **Behaviour Trends:** Aggregate indicators showing how acceleration, braking, or instability patterns are developing over time.

In the current TeleDrive design, the analytics view focuses on practical trend visibility rather than complex statistical reporting. It helps the rider answer questions such as:

- Are recent ride scores improving or declining?
- Is harsh braking becoming more frequent?
- Are instability-related issues occurring repeatedly?
- Is the rider maintaining smoother behaviour across successive trips?

The Analytics screen should therefore be treated as a progress-review tool rather than a live monitoring interface.

9.12 Event Evidence Gallery

The Event Evidence Gallery stores and displays the photographs captured automatically during detected ride events. These images are intended to provide contextual review of what was happening around the time of the detected event.

The gallery supports the following user actions:

- View captured images in newest-first order.
- Open a selected image for detailed inspection.
- Review image metadata such as date, time, and file size.
- Export an image to the public device gallery for long-term retention.
- Delete an image after confirmation.

The detail view is useful when the rider wants to inspect one evidence image more closely or preserve it outside the application.

Important evidence management notes are as follows:

- Evidence capture is automatic; the rider does not manually trigger the photograph.
- Some trips may contain no evidence images if no qualifying events occurred, if the camera permission was denied, or if evidence capture was suppressed by cooldown conditions.
- Evidence files are cleaned automatically after the retention period used by TeleDrive.
- In the current implementation, old evidence files are removed automatically after approximately 24 hours unless the user exports them earlier.

This automatic cleanup helps limit storage growth and supports privacy-conscious local operation.

9.13 Event Map

The Event Map provides a geographic review of the completed ride. It combines route visualisation, detected event markers, and hotspot analysis so that the rider can inspect where risky or repeated behaviours occurred.

The main map elements are:

- **Route Polyline:** The travelled route is drawn as a polyline on the map.
- **Event Markers:** Colour-coded circular markers identify important trip points and detected events.
- **Marker Labels:** Markers include event letters for quick recognition.
- **Bottom Sheet / Event Review Area:** Tapping a marker opens contextual review information such as event type, severity, timestamp, coaching text, and any temporally matched evidence.
- **Hotspot Toggle:** A control allows the hotspot layer to be turned on or off.
- **Hotspot Overlay:** Repeated event clusters from multiple trips are shown as risk hotspots to help identify recurring problematic zones.

Table 9.2 summarises the marker letters used on the Event Map.

Table 9.2 Event Map Marker Legend

Marker	Meaning
S	Start point of the trip
E	End point of the trip
A	Harsh acceleration event
B	Harsh braking event
U	Unstable riding event

The Event Map is especially useful after several trips have been recorded because hotspot detection becomes more meaningful when repeated event locations appear across different rides. If the map base layer appears blank on first use, the rider should connect to the internet briefly so that map tiles can be downloaded and cached.

9.14 Permissions and Troubleshooting

TeleDrive depends on correct permissions, sensor availability, GPS access, and proper phone mounting. Table 9.3 provides a practical troubleshooting guide for common usage problems.

Table 9.3 Permissions and Troubleshooting Guide

Problem	Possible Cause	Solution
GPS speed is not updating	GPS is turned off, location permission is denied, or Google Play Services is not functioning correctly	Enable GPS, grant precise location permission, move to an area with better signal, and verify that Google Play Services is available.
No events are being detected	Ride speed remains below event-gating thresholds, riding is smooth, or the phone is mounted incorrectly	Verify mounting orientation, ensure the phone is stable, and note that very low-speed traffic may not generate speed-gated events.
Camera evidence is not being captured	Camera permission is denied, no qualifying event occurred, or evidence capture was blocked by cooldown	Grant camera permission, complete another monitored trip, and remember that detection can still work even when evidence capture is unavailable.
Event Map appears blank	Map tiles were not downloaded previously and	Connect to the internet once, open the Event Map, and allow map tiles to load and cache.

	no internet connection is available	
Application stops or monitoring ends unexpectedly in the background	Device battery optimisation or aggressive task management is limiting the foreground service	Keep the foreground notification active, avoid force-closing the app, and exclude the app from restrictive battery optimisation if the device allows it.
Readings appear inaccurate or inconsistent	Phone is loosely mounted, rotated, tilted excessively, or not facing the expected direction	Re-mount the phone securely in portrait orientation with the top edge facing forward.
No route or distance is shown correctly	GPS reception was poor during the ride or location access was unavailable	Repeat the ride with GPS enabled, wait for location lock before starting, and avoid enclosed or GPS-denied environments.
Old evidence images have disappeared	Automatic cleanup removed files after the retention period	Export important evidence images before the cleanup period ends.
Summary values appear incomplete	Permissions were denied or GPS data was unavailable during part of the trip	Check permissions in Android settings and repeat the ride under normal GPS conditions.

If a problem persists, the rider should first confirm permissions, mounting, GPS availability, and battery status before assuming a software fault.

9.15 Safe Usage Guidelines

TeleDrive is intended to assist behavioural awareness, not to distract the rider during travel. Safe usage is therefore essential.

The following guidelines should be followed whenever the application is used:

1. Mount the phone securely before starting the ride.
2. Keep the phone in a stable portrait position with minimal movement relative to the handlebars.
3. Do not attempt to operate the application while the vehicle is moving.
4. Review the live screen only when safely stopped, and perform detailed review after the trip is completed.
5. Ensure that the battery is adequately charged before long rides.
6. Keep GPS enabled throughout the ride for correct route, speed, and trip-summary generation.
7. Grant only the permissions required for system operation and review them responsibly in Android settings.
8. Export only those evidence images that need long-term retention.
9. Use the application as a behavioural assistance tool, not as a substitute for safe riding judgement.
10. If readings appear unusual, stop using the current mount arrangement and correct the phone position before the next ride.

Following these guidelines helps both TeleDrive's accuracy and the rider's safety.

CHAPTER 10 CONCLUSION

10.1 Introduction

This chapter presents the concluding synthesis of the TeleDrive project and reflects on the work completed across problem analysis, system specification, design, implementation, testing, and real-world evaluation. The purpose of this chapter is to consolidate the most important outcomes of the project, highlight its principal technical contributions, and assess the practical relevance of the implemented system within the scope of the study.

TeleDrive was developed in response to a clear practical problem: individual two-wheeler riders, especially those operating under time pressure in everyday urban conditions, lack an affordable and privacy-conscious tool for monitoring riding behaviour using equipment they already possess. In addressing this gap, the project aimed not only to propose a conceptual solution, but to design, build, and validate a functioning smartphone-based telematics system capable of real-time behavioural analysis on Android devices.

Accordingly, the sections that follow summarise the work completed, identify the major technical contributions of the project, present the most significant findings obtained through testing and results analysis, and conclude with the overall significance of TeleDrive as a validated baseline for smartphone-based two-wheeler telematics.

10.2 Result and Analysis

10.2.1 Controlled Scoring Validation

Table 10.1 presents selected rides used to validate the scoring model across different event profiles.

Table 10.1 Controlled Scoring Validation Tests

Trip	Accel	Brake	Unstable	Total	Eco Score	Financial Loss
T1	1	1	2	4	76	—
T2	2	1	4	7	63	₹9 / 2 km
T3	0	0	1	1	99	—
T4	0	0	0	0	100	—

T4 (score 100) confirms correct baseline behaviour: zero detected events produce no penalty. T3 (score 99) demonstrates that an isolated low-severity event results in only a minor deduction. T1 (score 76) reflects moderate mixed behaviour and falls within the caution range. T2 (score 63) represents sustained aggressive riding and shows substantially higher financial loss, making the behavioural cost immediately visible to the rider.

10.2.2 Empirical Scoring Properties

Using Table 9.1 together with the 9-trip output-validation dataset, four key scoring properties are observed:

Graduated Response. Scores near 100 are associated with smooth rides and minimal events, while lower scores correspond to higher event counts and more severe behaviour patterns.

Diminishing Returns. Repeated events of the same type do not reduce the score linearly. This prevents score collapse during extended rough-road or repeated single-category conditions.

Cross-Category Sensitivity. Events distributed across multiple categories generally produce stronger overall penalties than the same number concentrated in one category, because each category carries its own penalty progression.

Financial Loss Scaling. Observed financial loss values range from ₹0 for perfect rides to higher values for aggressive trips, confirming that the economic indicator increases with inefficient riding behaviour.

10.3 Trip Summary Results

This section analyses the outputs presented across the Ride Summary, Trip History, Trip Details, Event Evidence, and Event Map screens, demonstrating how the system translates raw sensor data into actionable rider feedback.

10.3.1 Ride Summary (low score)

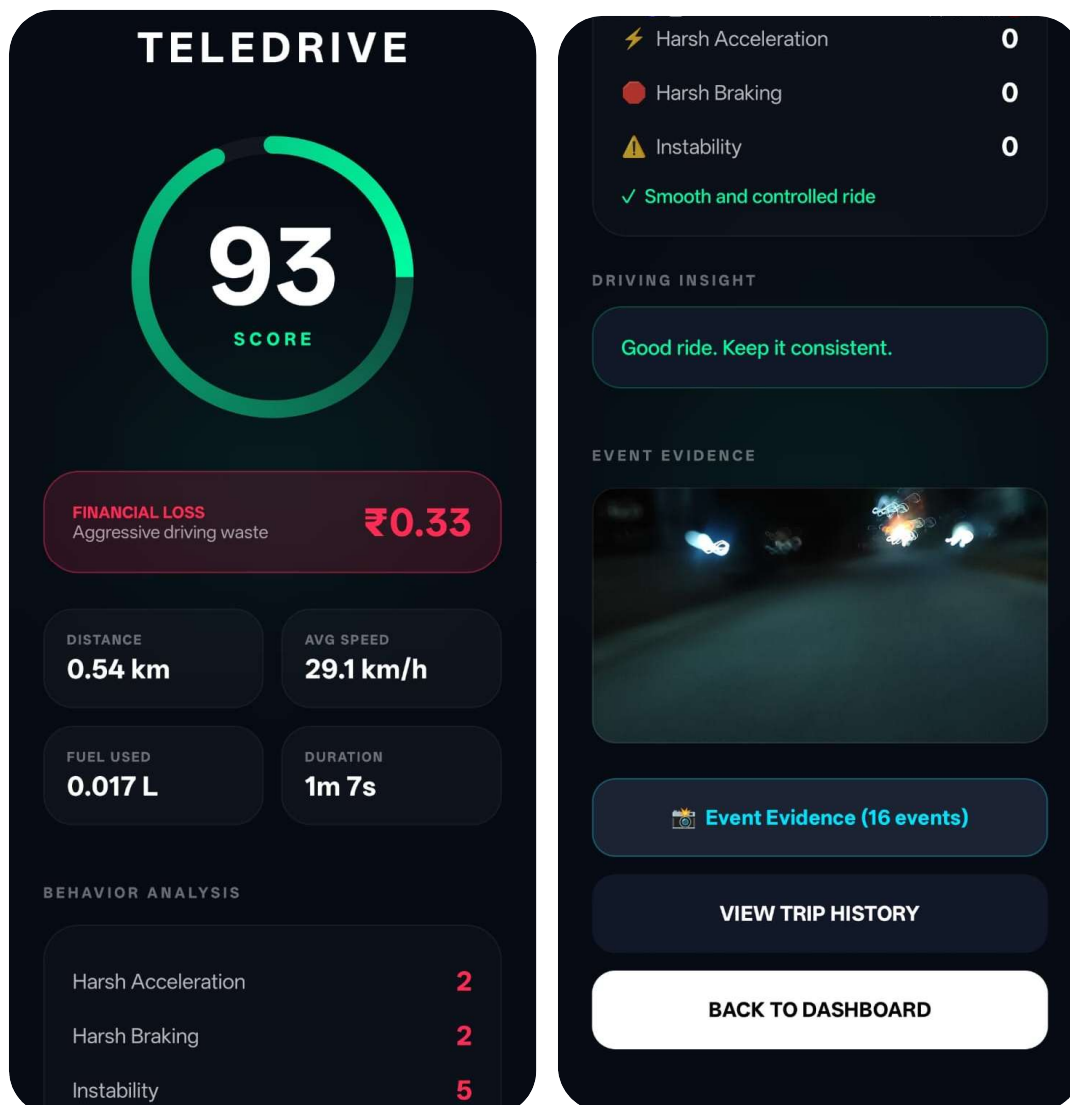


Figure 10.1 Ride Summary Screen — Sample Output (Score: 93)

Figure 10.1 shows a trip scoring 93/100 despite 9 total events (2 acceleration, 2 braking, and 5 instability events) over a distance of 0.54 km. The score remains in the green range because the diminishing-returns penalty model reduces the cumulative impact of repeated events of the same type. This behaviour aligns with the design objective of preventing score collapse during clustered adverse conditions while still recording the event pattern and associated financial impact. The ride recorded a financial loss of ₹0.33 (approximately ₹0.61/km).

Additional output-validation rides confirm similar behaviour at lower score ranges. A ride scoring 44/100, with 3 acceleration, 2 braking, and 3 instability events over 0.39 km, produced a financial loss of ₹2.52 (approximately ₹6.46/km), with the score indicator shifting to the red range. Another ride scoring 51/100 accumulated 27 total events, including 20 instability events, over 0.67 km. Despite the high event volume, the score did not collapse to zero, confirming that the scoring model avoids disproportionate penalties during sustained adverse conditions.

10.3.2 Ride Summary (100 score)

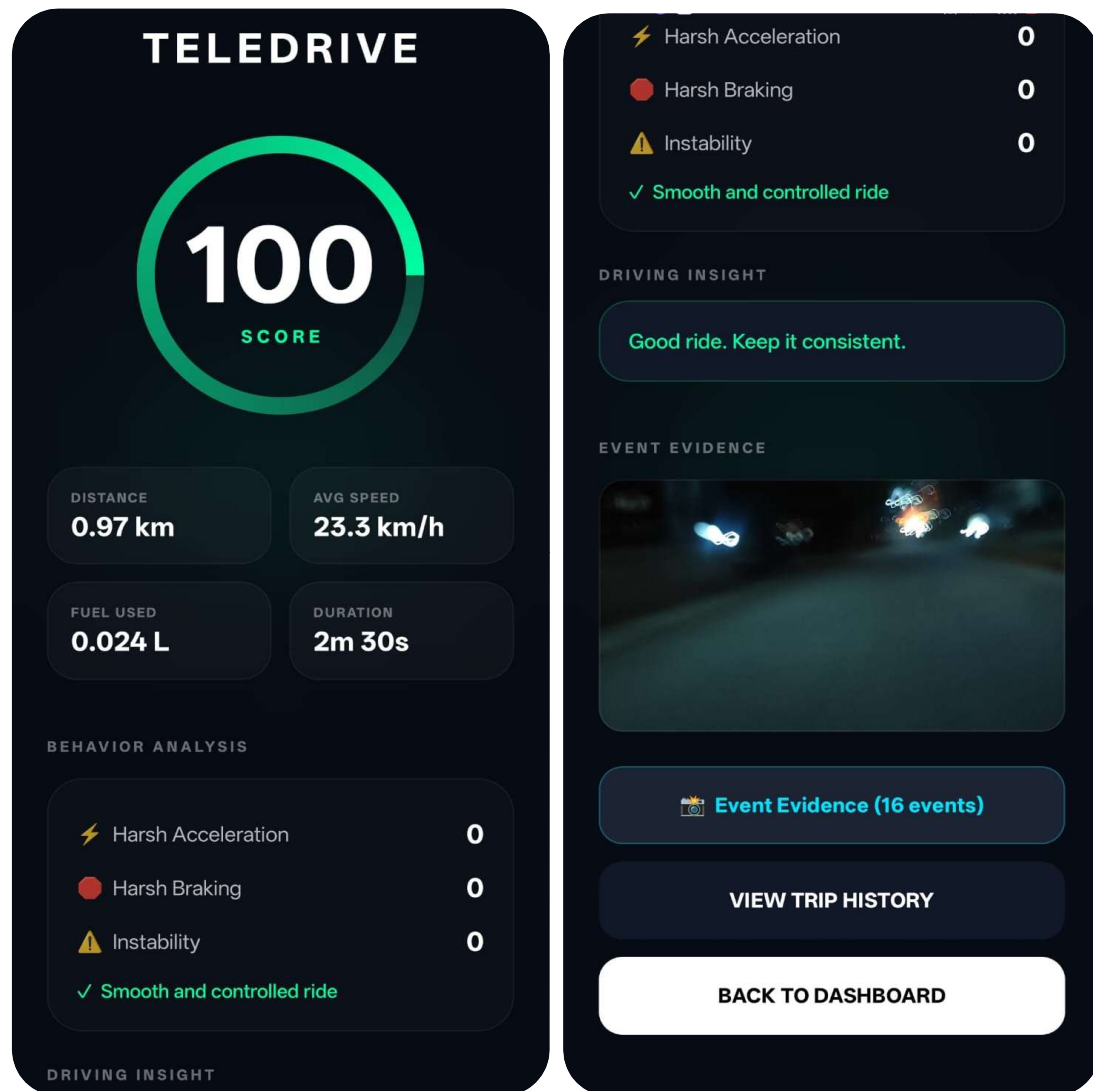


Figure 10.2 Ride Summary Screen (Score: 100)

Figure 10.2 presents a trip achieving a perfect eco score of 100 over 0.97 km with zero detected events across all categories. Since no penalties were applied, the score remained at the maximum value, supporting the claim that the scoring model does not introduce false deductions during smooth rides.

Note: The Event Evidence counter shown on the summary screen reflects cumulative stored evidence from previous trips, not events from the current ride.

10.3.3 Trip History Screen

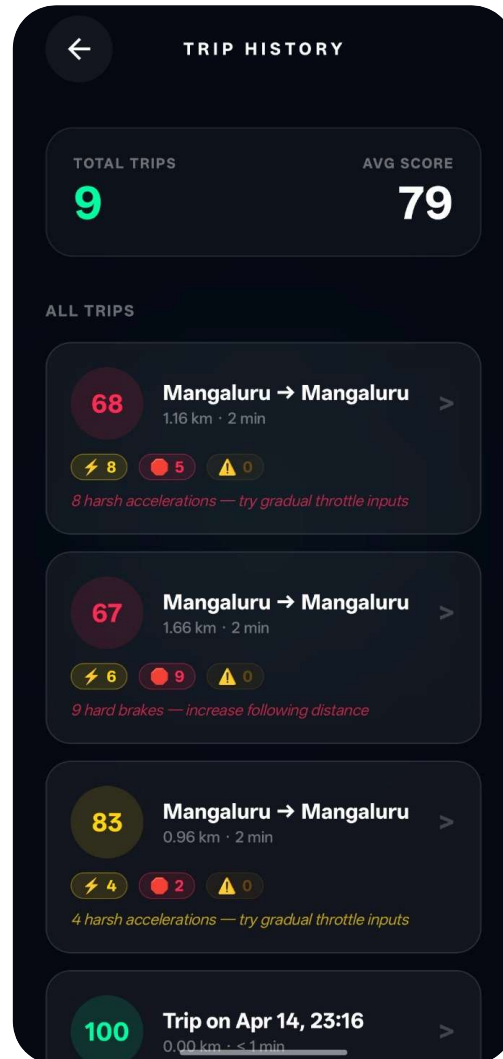


Figure 10.3 Trip History Screen Showing Stored Trips and Aggregate Average Score

The Trip History screen displays all 9 output-validation rides with an aggregate average score of 79.

Table 10.2 presents the output-validation ride set.

Table 10.2 Output-Validation Rides — Full Trip History

Trip	Score	Distance	Duration	Accel	Brake	Unstable	Coaching Tip
T1	68	1.16 km	2 min	8	5	0	8 harsh accelerations — try gradual throttle inputs
T2	67	1.66 km	2 min	6	9	0	9 hard brakes — increase following distance
T3	83	0.96 km	2 min	4	2	0	4 harsh accelerations — try gradual throttle inputs
T4	100	0.00 km	< 1 min	0	0	0	Perfect ride — zero harsh events
T5	100	0.97 km	2 min	0	0	0	Perfect ride — zero harsh events
T6	83	1.02 km	1 min	4	2	0	4 harsh accelerations — try gradual throttle inputs
T7	73	0.42 km	< 1 min	1	0	15	15 stability events — reduce speed on bends
T8	60	0.83 km	1 min	2	2	16	16 stability events — reduce speed on bends
T9	77*	—	—	—	—	—	Scrolled beyond visible area

Estimated from the aggregate score shown on-screen.

Scores ranged from 60 to 100 with an average of 79, confirming meaningful differentiation across riding behaviours. Trips T1, T2, T3, and T6 are dominated by acceleration and braking events, while T7 and T8 are dominated by instability events, indicating different riding conditions such as stop-and-go traffic versus rough roads or cornering.

The InsightEngine generated coaching messages aligned with the dominant behaviour category. Completed trips were successfully persisted to local storage and remained available across application launches.

10.3.4 Trip Details Analysis

The Trip Details screen presents a complete post-trip evaluation for a safe and efficient ride. Since no harsh acceleration, harsh braking, or instability events were detected, the system awarded a perfect eco score of 100. Fuel wastage and monetary loss are recorded as zero, while positive feedback is generated to reinforce smooth riding habits. This output validates the reward-based scoring logic of the TeleDrive system.

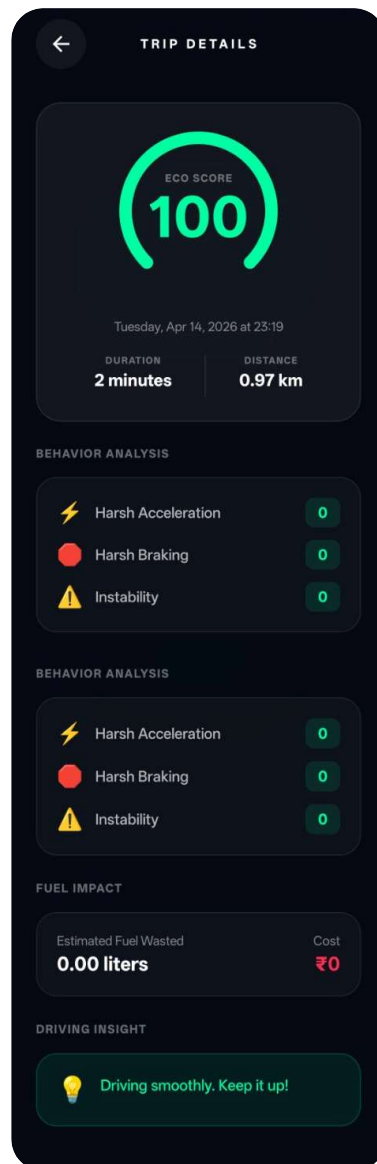


Figure 10.4 Trip Details Screen Showing Perfect Eco Score for Smooth Riding Behaviour

This screen illustrates the impact of aggressive riding behaviour on trip performance. Multiple harsh acceleration and braking events were detected during the journey, resulting in a reduced eco score of 68. The system also estimates fuel wastage and associated cost, while displaying corrective guidance to encourage safer and more economical riding. This confirms that the scoring engine responds appropriately to unsafe driving patterns.

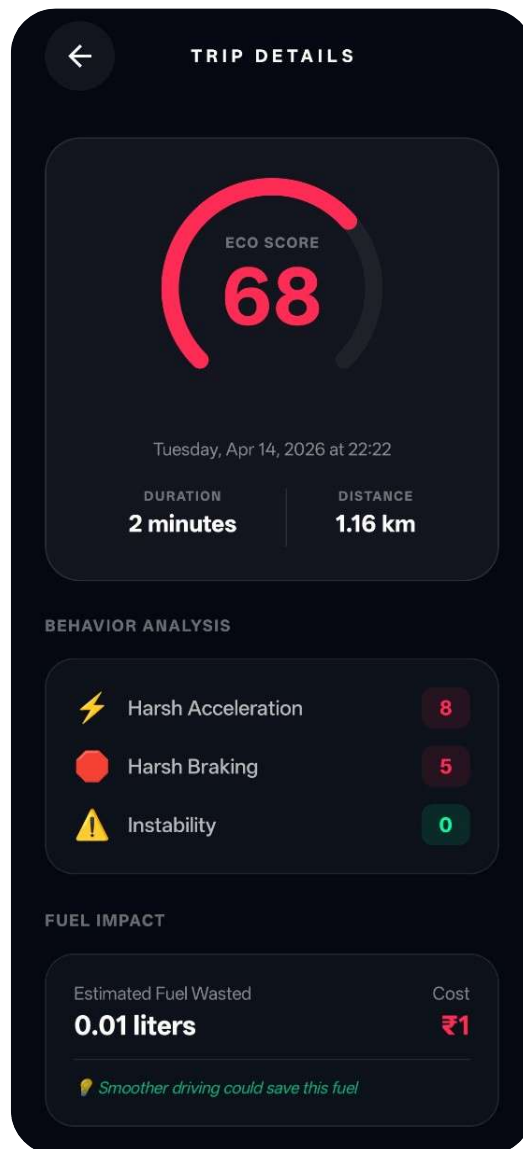


Figure 10.5 Trip Details Screen Showing Reduced Eco Score for Aggressive Riding Behaviour

All test results were captured from real on-device test sessions. No simulated or synthetic data was used.

Table 10.3 Real Trip Test Results

Tr ip	Eco Score	Distan ce	Avg Speed	Durati on	Harsh Accel	Harsh Brake	Instabilit y	Financial Loss
T1	76 / 100	9 km	22 km/h	0m 59s	1	1	2	₹6
T2	63 / 100	2 km	18 km/h	1m 43s	2	1	4	₹9
T3	99 / 100	1 km	31 km/h	0m 48s	0	0	1	—
T4	100 / 100	2 km	17 km/h	0m 59s	0	0	0	—
T5	90 / 100	—	—	—	—	—	—	—

Trip T4 produced a perfect score of 100 with zero detected events. This confirms the system correctly awards maximum score when riding behaviour is consistently smooth throughout a trip. The logarithmic model correctly does not penalise the score for the absence of events.

Trip T3 scored 99 despite 1 instability event. This demonstrates the diminishing-returns model functioning correctly: a single instability event on an otherwise clean ride incurs only a minimal penalty (score reduced by 1 point), correctly reflecting the isolated nature of the infraction rather than treating it as representative of the overall ride quality.

Trip T1, with 1 acceleration event, 1 braking event, and 2 instability events, scored 76. The score colour changes to yellow, indicating acceptable but improvable riding behaviour — consistent with a rider who exhibits occasional but not habitual aggressive manoeuvres.

Trip T2 scored 63, with 2 acceleration events, 1 braking event, and 4 instability events. The red score colour indicates consistently poor performance for this session. The financial loss of ₹9

across only 2 km renders the cost of aggressive riding immediately tangible and actionable: the rider is losing approximately ₹4.50 per kilometre through aggressive style.

The scoring model's behaviour across T1–T4 is consistent with its design characteristics: scores in the 90–100 range require near-zero events; scores in the 75–89 range reflect occasional minor infractions; and scores below 75 reflect multiple confirmed aggressive events.

10.3.5 Event Map Analysis

The Event Map screen overlays colour-coded event markers on an OpenStreetMap base layer and renders the travelled route as a polyline. A summary panel presents trip statistics, while hotspot detection groups repeated events occurring within nearby geographic areas.

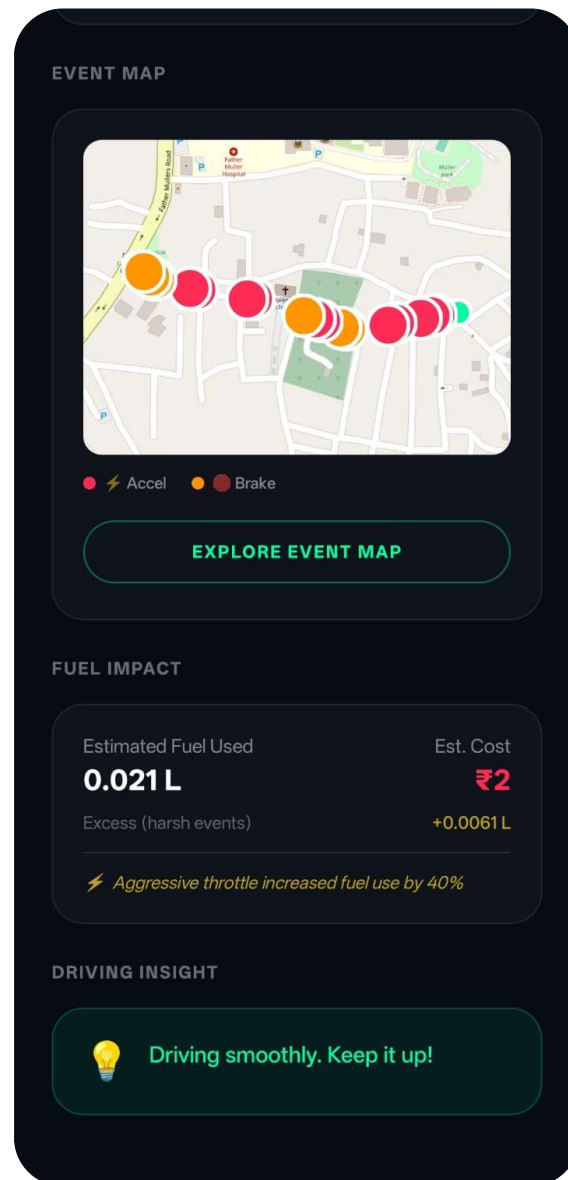


Figure 10.6 Event Map Interface of TeleDrive

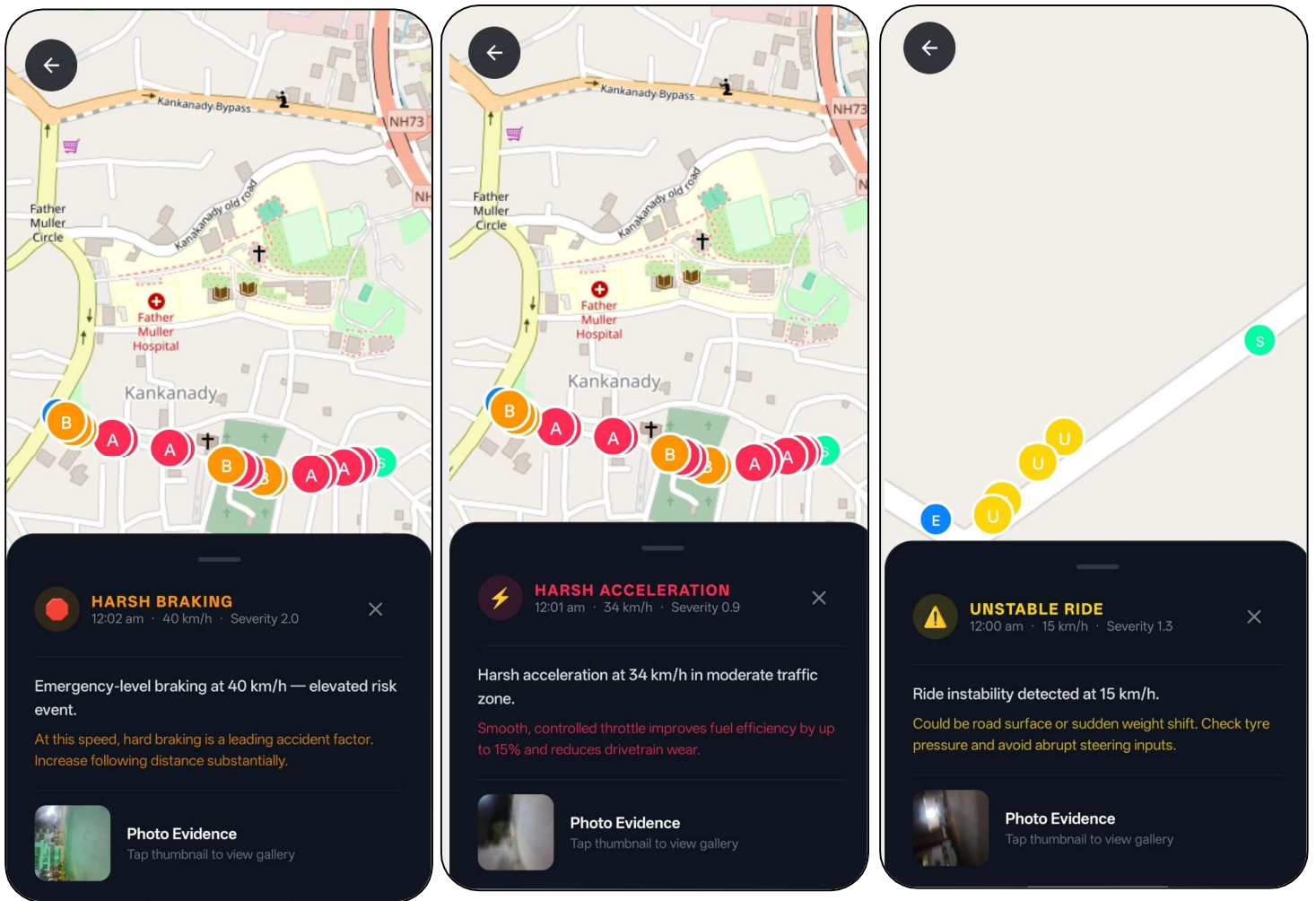


Figure 10.7 TeleDrive Event Map Showing Unsafe Events, Hotspot Detection, and Evidence

This feature extends the system beyond score reporting by helping riders identify where unsafe behaviour repeatedly occurs, supporting route awareness and future riding improvement.

10.4 Eco Score Behaviour Analysis

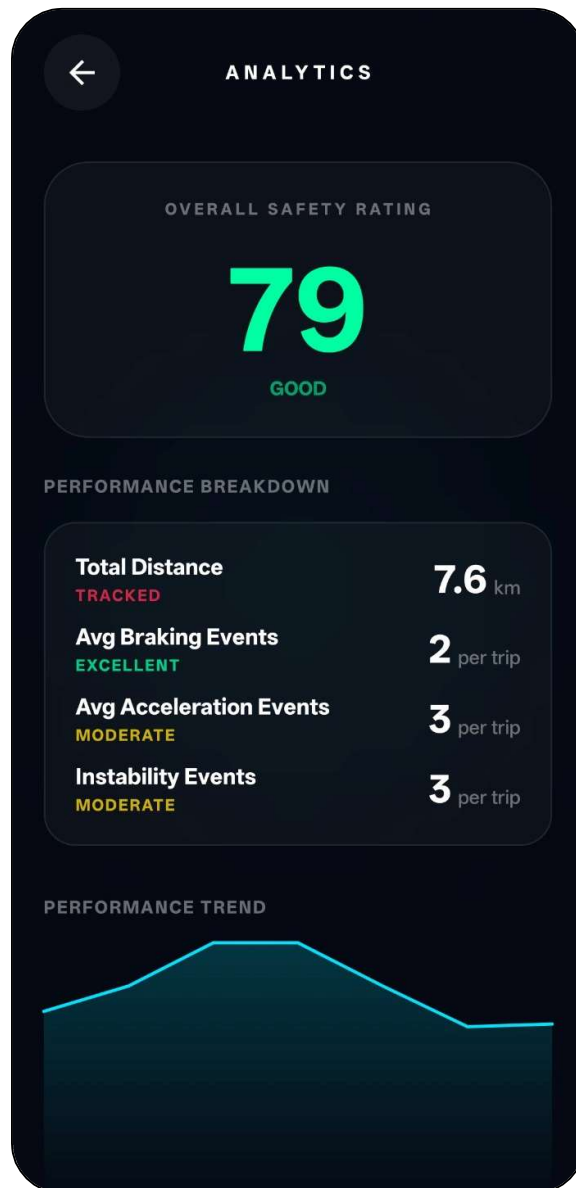


Figure 10.8 Eco Score Trend — Last 7 Trips (Analytics Screen)

Figure 10.8 shows the Analytics screen with an Overall Safety Rating of 79 (“GOOD”) across 9 trips and 7.6 km. Category breakdowns report Average Braking as “EXCELLENT” (2 events/trip), while Average Acceleration and Average Instability are rated “MODERATE” (3 events/trip each). The trend chart displays score variation across the most recent seven trips, confirming that the Analytics module correctly tracks longitudinal riding performance.

10.5 Machine Learning Evaluation Analysis

Two evaluation contexts are reported: offline hold-out testing on the training dataset partition, and real-world validation on labelled windows from live ride sessions

The TeleDrive 1D-CNN model (Version 4) was trained on 115,250 balanced samples derived from 473,800 raw readings collected across 48 ride sessions. Evaluation on a stratified 20% hold-out dataset (23,050 windows) achieved an offline accuracy of 96.0% with a macro F1-score of 0.92.

Table 10.4 Offline Classification Performance

Class	Precision	Recall	F1-Score	Support
NORMAL	0.97	0.98	0.975	5,762
HARSH_ACCELERATION	0.93	0.90	0.915	5,762
HARSH_BRAKING	0.95	0.93	0.940	5,762
UNSTABLE_RIDE	0.92	0.87	0.895	5,764
Macro (Overall)	0.94	0.92	0.92	23,050

10.5.1 Real-World Validation

On 402 one-second labelled windows collected from live rides, the deployed model achieved 98.76% accuracy. This higher real-world performance reflects the complete TeleDrive decision pipeline rather than raw model output alone. Consecutive-window confirmation, speed gating, and hybrid consensus between rule-based and ML detection help suppress isolated misclassifications and reduce false positives during actual riding conditions.

10.5.2 Interpretation

The results indicate that the model generalises effectively to unseen data and practical road environments. Highest performance was observed for NORMAL and HARSH_BRAKING classes, while UNSTABLE_RIDE showed comparatively lower recall due to overlap with road-surface vibration and environmental noise. When integrated with the hybrid detection pipeline, the ML subsystem provides reliable event classification suitable for real-time deployment on a smartphone device.

10.6 Real-World System Performance Analysis

TeleDrive’s runtime performance was evaluated across the 9 output-validation rides, with emphasis on latency, classification reliability, and resource consumption during practical use.

10.6.1 Processing Latency

- Sensor sampling: approximately 50 Hz (target ≥ 40 Hz)
- End-to-end one-second window processing: under 50 ms
- TensorFlow Lite inference: approximately 12 ms
- GPS update frequency: approximately 1 Hz
- Event-to-evidence capture: under 500 ms
- UI refresh performance: within interactive real-time limits

These results indicate that the system comfortably satisfies real-time processing requirements on a smartphone device.

10.6.2 Classification Performance

- Offline hold-out dataset (23,050 windows): 96.0% accuracy, macro F1-score 0.92
- Real-world labelled windows (402): 98.76% accuracy

The higher real-world accuracy is attributable to full pipeline controls such as event confirmation filtering, speed gating, and hybrid consensus logic, which reduce isolated false detections.

10.6.3 Resource Consumption

- Battery usage: approximately 4.2% per 30 minutes of active monitoring
- Memory footprint: approximately 45 MB during foreground operation
- Evidence image size: approximately 2.0 MB per photograph
- Trip summary storage: 2–4 KB per trip record
- Automatic cleanup removes expired evidence files and limits storage growth

10.7 Overall System Performance Summary

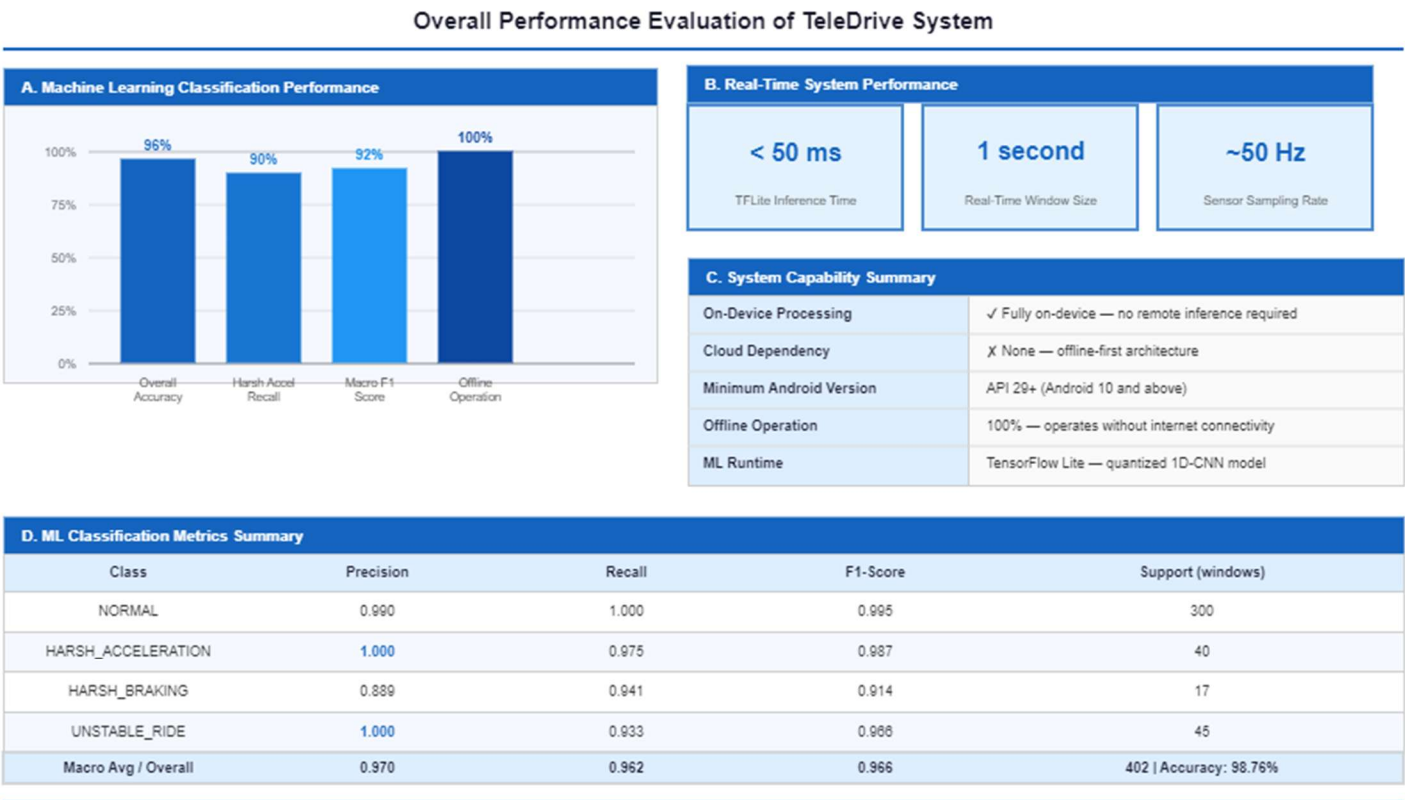


Figure 13: Overall Performance Evaluation of TeleDrive System — combining ML classification metrics, real-time processing performance, and system capability summary.

Figure 10.9 TeleDrive System — Consolidated Performance Evaluation

10.8 Comparative Analysis with Existing Solutions

TeleDrive was compared against commercial OBD-II–based telematics systems, insurance tracking applications, and academic prototypes to evaluate its positioning in terms of functionality, cost, and deployment model.

Table 10.5 Comparative Feature Summary

Feature	TeleDrive	OBD-II / Insurance Apps	Academic Prototypes
Hardware required	None	OBD-II dongle / Bluetooth	Custom sensors
Internet required	No (offline-first)	Yes (cloud-based)	Varies
Two-wheeler support	Yes (primary focus)	Limited	Rare
Real-time feedback	Yes (real-time)	Mostly post-trip	Varies
ML deployment	On-device (TFLite)	Cloud-based	Sometimes
Evidence capture	Yes (photo-based)	No	No
Financial loss estimate	Yes (per trip)	Limited/general	Rare
Privacy	Local processing only	Data sent to servers	Data collection required
Cost	No additional cost	Hardware + subscription	Research setup

10.9 Engineering Iterations Applied

Throughout development and testing, multiple engineering issues were identified through real ride observations and resolved through targeted design or parameter changes.

Table 10.6 summarises the most significant iterations that improved system behaviour.

Table 10.6 Engineering Issues and Applied Fixes

	Issue Identified	Fix Applied	Result
1	Excessive braking false positives at low speed	Revised speed gates for each event type	False detections reduced during slow movement
2	Instability detections on rough but straight roads	Increased instability threshold and added gyroscope validation	Improved distinction between rough roads and unstable riding
3	Rapid score collapse on rough roads	Replaced linear penalties with diminishing-returns scoring	Scores remained stable under sustained rough conditions
4	ML over-triggering during vibration	Added vibration baseline adaptation and expanded training data	Reduced vibration-related false positives
5	Detections during parking manoeuvres	Enforced minimum speed gating	Parking movements no longer triggered events
6	Multiple photos for one sustained event	Added evidence capture cooldown	One photo captured per sustained event
7	GPS speed spikes after signal recovery	Added smoothing and short-term speed history checks	GPS spikes no longer caused false events
8	Weak differentiation between moderate and severe events	Added severity-based penalty scaling	Severe events receive stronger penalties
9	Evidence gallery growth over time	Implemented automatic cleanup worker	Old evidence files removed automatically
10	Over-scoring during sustained rough conditions	Increased scoring cooldown interval	Repeated scoring reduced during prolonged events

These iterations were driven by observed behaviour during real rides and refined through measurable adjustments rather than arbitrary tuning. The final system therefore reflects an empirical engineering process grounded in practical testing conditions.

10.10 Discussion of Findings

The results presented in this chapter indicate that TeleDrive achieves its primary design objective: real-time, on-device riding behaviour analysis for two-wheelers using only smartphone sensors, without requiring external hardware or continuous internet connectivity.

Detection Performance. The hybrid detection pipeline, combining rule-based logic with a 1D-CNN classifier, achieved strong offline and real-world performance. This suggests that the model generalises effectively while additional pipeline controls such as confirmation filtering and speed gating improve reliability during practical use.

Scoring Effectiveness. The eco score model produced outputs that aligned with observed riding behaviour across the tested range. Smooth rides consistently received high scores, while aggressive or unstable rides received lower scores. The diminishing-returns penalty model prevented excessive score collapse during prolonged adverse conditions. The financial loss indicator further translated behaviour quality into a practical economic metric.

User Feedback Value. The Ride Summary, Trip History, Analytics, and Trip Details screens provided multiple levels of feedback: immediate trip results, historical trends, and detailed ride inspection. Coaching tips were aligned with the dominant behaviour type, helping riders identify specific improvement areas.

Evidence System. Automatic photographic evidence capture created a visual record of detected events without requiring rider interaction. Combined with timestamped trip data, this improved transparency and reviewability of system outputs.

System Efficiency. Low processing latency, modest battery usage, and stable memory consumption demonstrate that TeleDrive can operate within the resource limits of a typical Android smartphone during active rides.

Observed Limitations. Testing was performed using a single smartphone model, one mounting orientation, one motorcycle type, and one geographic region. Broader validation across different devices, vehicles, orientations, and road environments would strengthen generalisability. Unstable riding detection remains the most challenging category because of overlap with environmental vibration and road-surface noise.

10.11 Limitations

10.11.1 Introduction

This chapter critically examines the present boundaries of the TeleDrive system and identifies realistic directions for its further development. Although the results presented in the preceding sections of this chapter demonstrate that the application functions effectively as an offline, smartphone-based telematics solution for two-wheeler riding behaviour analysis, no practical system can be evaluated without recognising the constraints under which it was designed, implemented, and validated.

The discussion in this chapter distinguishes between four related aspects: current system limitations arising from the implemented design, technical and operational constraints imposed by the smartphone-based deployment model, limitations in the scope of field validation conducted for this project, and future enhancements that can logically extend the existing architecture. This distinction is important because some constraints are not defects, but deliberate design decisions made to preserve affordability, privacy, offline usability, and implementation feasibility within the scope of a final-year project.

Accordingly, the chapter presents an evidence-based evaluation of what the current version of TeleDrive does not yet address, where its present results should be interpreted with caution, and which enhancements are most appropriate for future development and research. The aim is not to diminish the value of the implemented system, but to position it accurately as a validated baseline for further technical refinement, broader deployment, and deeper academic investigation.

10.11.2 Current System Limitations

The current version of TeleDrive successfully delivers real-time riding behaviour analysis using only smartphone sensors and on-device processing. However, the implemented design introduces several practical limitations that affect detection robustness under certain operating conditions. These limitations do not invalidate the system's usefulness; rather, they define the conditions under which the reported performance should be interpreted.

Table 10.7 Current System Limitations

Limitation	Description	Practical Effect
Mounting orientation dependency	Forward-acceleration analysis assumes portrait handlebar mounting with consistent device-axis alignment. Automatic orientation calibration is not implemented.	Detection accuracy may decrease if the phone is rotated, tilted, mounted differently, or repositioned between rides.
Low-speed event suppression	Speed gates are used to reduce false positives and suppress event detection below defined thresholds.	Aggressive behaviour in dense stop-and-go traffic may not always be captured at very low speeds.
Dependence on GPS-assisted validation	Distance tracking, route mapping, fuel estimation, and some speed-aware filtering depend on GPS availability.	Poor signal reception can reduce trip-summary completeness and temporarily affect speed-dependent logic.
Instability class sensitivity	Road vibration and rider-induced instability can produce similar inertial patterns.	Rough roads or prolonged vibration may reduce unstable-ride classification reliability.
Evidence capture viewpoint limitation	Evidence photos depend on the mounted device camera orientation and available field of view.	Some images may provide limited contextual value if the road scene is not clearly visible.

Short-term evidence retention	Evidence files are automatically deleted after the configured retention period unless exported.	Supports privacy and storage control, but limits long-term review unless saved manually.
Local storage scalability boundary	Trip history uses SharedPreferences with JSON serialisation.	Suitable for project-scale usage, but less efficient for very large long-term datasets.

The most significant behavioural-analysis limitations are dependence on correct phone mounting, reduced sensitivity at low speeds, and the relative difficulty of separating genuine instability from environmental vibration. These are not accidental flaws, but expected trade-offs of a low-cost, privacy-preserving, smartphone-only telematics system without external vehicle hardware.

Several limitations also reflect deliberate engineering choices. Speed gating reduces false positives during near-stationary motion, while local-only storage and automatic evidence cleanup prioritise privacy and low infrastructure complexity. These limitations should therefore be understood as boundaries of the current design rather than contradictions of the project's objectives.

10.11.3 Technical Constraints

In addition to the current system limitations discussed in the previous section, TeleDrive operates under a number of technical constraints inherent to its smartphone-based deployment model. These constraints define the platform and environmental conditions under which the system can function as intended. They should therefore be interpreted as operational boundaries of the implemented architecture rather than as defects in the application itself.

Table 10.8 Technical Constraints

Technical Constraint	Description	Operational Impact
Android-only platform support	TeleDrive depends on Android components such as SensorManager, Foreground Services, WorkManager, CameraX, and Jetpack Compose.	The current version cannot run on iOS or other platforms without major redevelopment.
Sensor hardware dependency	Full operation requires a linear accelerometer, gyroscope, GPS receiver, and rear camera.	Devices with missing or low-quality sensors may experience reduced functionality or lower detection consistency.
Dependence on Google Play Services	Location tracking uses FusedLocationProviderClient.	Devices without compatible Google Play Services may have limited GPS-based features.
Foreground service requirement	Continuous monitoring depends on an active foreground service.	Behaviour may be affected by battery optimisation policies and device-specific process management.
Permission-dependent features	Core functions rely on runtime permissions for location, camera, and notifications.	If permissions are denied, some features degrade

		gracefully but monitoring completeness is reduced.
Initial network dependency for map tiles	Event Map tiles require initial download and caching.	First-time map use may be limited in fully offline conditions.
Local device storage model	All trip data and evidence are stored locally on the phone.	Improves privacy, but limits cross-device continuity and long-term storage scale.
On-device processing budget	Signal processing and ML inference execute in real time on the smartphone.	Future model complexity and background analytics must remain within mobile resource limits.

These constraints are closely linked to the project’s design philosophy. TeleDrive was intentionally developed as a low-cost, self-contained, offline-capable system without external hardware or cloud infrastructure. As a result, the system gains privacy, affordability, and simple deployment, but must operate within the performance, sensor, and lifecycle constraints of consumer Android smartphones.

From an academic perspective, these constraints help define both the strengths and boundaries of the implemented solution. The same design choices that make TeleDrive practical for everyday riders also limit its portability, hardware independence, and large-scale extensibility when compared with telematics platforms supported by dedicated hardware or backend infrastructure.

10.11.4 Deployment and Validation Limitations

Although TeleDrive was evaluated through real on-device rides rather than simulated data, the deployment and validation evidence presented in this project remains bounded by the scale of a final-year implementation. The reported results therefore demonstrate practical feasibility and functional correctness within the tested context, but they should not be interpreted as universal performance guarantees across all riders, vehicles, smartphones, and road environments.

Table 10.9 Deployment and Validation Limitations

Validation Limitation	Current Scope	Implication
Single smartphone model	Final validation used one Android smartphone model.	Performance may vary across devices with different sensor quality, processing power, or camera hardware.
Single vehicle platform	Testing was performed on one two-wheeler platform.	Additional validation is needed across scooters, motorcycles, and other vehicle classes.
Single mounting configuration	Testing used one primary phone mounting method.	Robustness across alternate mounting positions and vibration profiles is not yet established.
Single geographic region	Validation was conducted in one urban region.	Different traffic patterns, terrain, road quality, and GPS conditions may affect results elsewhere.
Limited ride dataset	Final production-build validation used a relatively small number of rides and total distance.	Sufficient for end-to-end verification, but not for long-term reliability or rare-edge-case analysis.
Short-duration sessions	Many validation rides were short and behaviour-focused.	Extended daily use and multi-hour monitoring require further study.
Limited user acceptance sample	Usability feedback came from a small rider group.	Findings are indicative but not statistically broad.
No external benchmark trial	No side-by-side field comparison with commercial telematics systems was conducted.	Internal effectiveness is shown, but comparative superiority is not yet established.

These limitations are important when interpreting the strong performance figures reported in earlier chapters. The results are credible within the tested environment because they were obtained under real riding conditions using the implemented system. However, broader deployment claims

would require more diverse validation across multiple devices, riders, road conditions, and vehicle types.

From an academic perspective, this does not weaken the project contribution. Rather, it defines the current evidential boundary of the work. TeleDrive can be defended as a validated proof-of-implementation and a workable baseline system, but not yet as a fully generalised telematics solution for all two-wheeler deployment contexts.

10.11.5 Privacy and Ethical Considerations

TeleDrive was intentionally designed as an offline-first system in which sensing, event detection, scoring, and data storage occur locally on the user's smartphone. This architecture reduces several common privacy risks associated with cloud-based telematics systems, particularly continuous remote tracking, third-party data sharing, and dependence on persistent internet connectivity. From a privacy perspective, this is one of the system's key strengths.

However, local processing does not eliminate privacy and ethical considerations altogether. The application still captures and stores sensitive behavioural and contextual data, including route history, riding event records, timestamps, location coordinates, and event-linked photographic evidence. Even when such data remains on the device, it may reveal patterns about a rider's movements, habits, visited locations, and riding behaviour. Privacy protection in TeleDrive should therefore be understood as data minimisation through local processing, not as the absence of sensitive data collection.

A further ethical consideration arises from the evidence capture mechanism. Although intended to improve transparency and post-trip review, captured images may include other road users, surrounding vehicles, or location-specific details. For this reason, the current design limits evidence retention through automatic cleanup and avoids continuous recording. Nevertheless, the feature should be used with clear user awareness and caution regarding how evidence is stored, exported, or shared.

There is also an ethical boundary in the interpretation of system outputs. TeleDrive is designed as a rider-assistance and behavioural-awareness tool, not as a legal, insurance, or enforcement

instrument. Its scores, detected events, and financial loss estimates are computational assessments derived from smartphone sensor data and defined rule or model logic. They should therefore be interpreted as informative indicators rather than definitive proof of negligence, road fault, or accident liability.

From a responsible deployment perspective, three principles are important: informed user consent for data and image capture, transparent communication that outputs are advisory rather than punitive, and careful handling of stored trip records and exported evidence. These considerations would become even more important if future versions introduce cloud backup, fleet supervision, or third-party integration. In its current form, TeleDrive adopts a privacy-conscious design, but ethical use still depends on how collected data is managed and interpreted in practice.

10.12 Future Enhancements

The current implementation of TeleDrive establishes a functional baseline for smartphone-based two-wheeler telematics, but several enhancements can be introduced to improve robustness, usability, scalability, and deployment readiness. These enhancements arise directly from the limitations identified in the previous sections and from opportunities observed during testing and user feedback.

Table 10.10 Future Enhancements for TeleDrive

Proposed Enhancement	Rationale	Expected Benefit	Priority
Automatic mounting orientation calibration	Current detection assumes consistent phone alignment.	Improves robustness across different mounting positions and device orientations.	High
Improved low-speed event handling	Existing speed gates may suppress valid events in heavy traffic.	Better behavioural detection in urban stop-and-go conditions.	High
Enhanced instability classification	Unstable riding remains the most challenging event class.	Stronger separation between rider instability and road vibration.	High
Migration to Room database	SharedPreferences is less suitable for large long-term datasets.	Better scalability, faster queries, and richer analytics.	High
Real-time audio or voice coaching	Riders should not depend on screen glances during trips.	Safer and more immediate behavioural feedback.	Medium

User-configurable evidence retention	Current cleanup follows a fixed policy.	Greater control over privacy, storage, and evidence preservation.	Medium
Optional encrypted backup and restore	Current data remains tied to one device.	Secure continuity across device changes or resets.	Medium
Flexible financial estimation settings	Cost outputs depend on fixed assumptions and user inputs.	More personalised and accurate cost estimates.	Medium
Expanded route analytics	Current analytics are functional but limited in depth.	Better trend analysis, hotspot review, and progress tracking.	Medium
Optional fleet or hardware integration	Diagnostics and multi-user features are outside current scope.	Extends TeleDrive into commercial or fleet scenarios.	Low/ Long-term

The most immediate priorities are orientation calibration, improved low-speed detection, stronger instability classification, and migration to a more scalable persistence layer. These changes would improve technical reliability without changing the system’s core identity as a smartphone-only, privacy-conscious, offline-capable platform.

Other enhancements such as audio coaching, configurable retention, and encrypted backup would mainly improve user experience and long-term adoption. Longer-term additions such as external hardware integration or fleet support should be viewed as expansion paths rather than immediate requirements. Overall, the strongest next step for TeleDrive is to refine the existing pipeline and extend it in ways that remain consistent with the project’s original objectives.

10.13 Summary of Work Completed

The TeleDrive project progressed from the identification of a practical transportation problem to the delivery of a working Android-based telematics system for two-wheeler riding behaviour analysis. The work began by examining the limitations of existing telematics solutions for individual riders, particularly dependence on dedicated hardware, subscription-based cloud services, and detection approaches primarily designed for four-wheelers. On this basis, the project defined a clear objective: to create a low-cost, privacy-conscious, smartphone-based system capable of monitoring riding behaviour in real time and converting sensor data into meaningful rider feedback.

To achieve this objective, the project first established a formal specification through the Software Requirements Specification, workflow modelling, structured design models, and layered system architecture. These stages defined the system scope, functional behaviour, constraints, data flows, persistent entities, and inter-module relationships required for a complete telematics application. The design process also justified key architectural choices, including offline-first processing, local storage, foreground-service monitoring, and a modular pipeline separating acquisition, signal processing, classification, scoring, evidence handling, analytics, and user interface layers.

The implementation stage translated this design into a functioning Android application developed in Kotlin. The final system acquires accelerometer, gyroscope, GPS, and camera data through smartphone hardware, processes one-second sensor windows in real time, and classifies riding behaviour using a hybrid detection pipeline combining rule-based logic with on-device machine learning through TensorFlow Lite. Beyond event detection, the system also implements eco score computation, fuel and financial impact estimation, evidence capture, trip persistence, contextual coaching, route visualisation, analytics, and rider profile management.

The project further included structured testing across unit, integration, validation, output, user acceptance, and system-level stages, followed by analysis of real ride results from live on-device sessions. These activities verified that the implemented system functions as an end-to-end telematics platform rather than as isolated modules. The completed work therefore represents a

full-cycle final-year project contribution encompassing problem definition, requirements engineering, design, implementation, empirical refinement, and real-world evaluation.

10.14 Key Technical Contributions

The TeleDrive project makes several meaningful technical contributions within the domain of smartphone-based two-wheeler telematics. The first contribution is the development of a complete on-device riding behaviour analysis pipeline tailored specifically to two-wheelers. Unlike generic smartphone telematics solutions that are often designed around four-wheeler motion characteristics, TeleDrive was structured around the vibration profile, directional movement, and road-surface sensitivity of two-wheeler riding. This gives the project a clear domain-specific contribution beyond general mobile sensing.

A second major contribution is the hybrid event-detection architecture that combines physics-based rule logic with an on-device one-dimensional Convolutional Neural Network deployed through TensorFlow Lite. The rule-based component provides deterministic and interpretable detection grounded in acceleration thresholds, speed validation, and event persistence checks, while the machine learning component captures temporal signal patterns that are difficult to express through fixed thresholds alone. The integration of these two approaches into a single real-time detection framework constitutes a central technical contribution of the implemented system.

A third contribution is the design of an offline-first telematics architecture in which sensing, signal processing, classification, scoring, evidence handling, and storage all occur locally on the smartphone. This design eliminates the need for external hardware, backend infrastructure, and continuous internet connectivity for core system operation. From an engineering perspective, this demonstrates that practical riding behaviour analysis can be achieved within the computational and energy constraints of an ordinary Android device while preserving user privacy through local-only data handling.

The project also contributes a rider-oriented behavioural evaluation model that extends beyond raw event detection. Through the eco score engine, fuel wastage estimation, and financial loss calculation, TeleDrive translates low-level sensor events into outputs that are more understandable

and practically relevant to everyday riders. This is further strengthened by contextual outputs such as coaching tips, trip history, analytics trends, event mapping, hotspot detection, and evidence-linked review, all of which improve interpretability and post-trip reflection.

Finally, TeleDrive contributes an empirically refined implementation methodology. The system was not configured only from theoretical assumptions; its thresholds, validation logic, and model behaviour were refined through repeated ride observations, real sensor data collection, and iterative testing on an actual Android deployment target. As a result, the contribution of the project lies not only in proposing a telematics concept, but in demonstrating a validated baseline system for smartphone-based two-wheeler riding behaviour analysis.

10.15 Major Findings and Outcomes

The testing and evaluation results indicate that TeleDrive successfully achieved its core objective of providing real-time, smartphone-based riding behaviour analysis for two-wheelers within the defined scope of the project. The implemented system was able to acquire sensor data continuously during active rides, process one-second motion windows on-device, classify riding events in real time, and generate meaningful trip-level outputs without relying on external telematics hardware or cloud infrastructure.

One of the most significant findings was that the hybrid detection pipeline produced strong classification performance under practical conditions. The machine learning subsystem achieved high offline evaluation performance, while the deployed system also performed strongly during real-world validation. When combined with confirmation logic, speed-aware validation, and consensus handling between rule-based and machine learning detection, the overall pipeline demonstrated reliable behaviour suitable for live smartphone deployment.

A second major outcome was that the eco scoring model produced meaningful differentiation across riding styles. Smooth rides with no confirmed aggressive events retained very high or perfect scores, while rides involving repeated harsh acceleration, harsh braking, or instability produced progressively lower scores and higher estimated behavioural cost. This confirms that the scoring model functioned not merely as a display metric, but as a behaviour-sensitive summary of

trip quality. The associated fuel and financial estimation outputs further improved the practical interpretability of these results.

The evaluation also showed that TeleDrive operated efficiently within the limits of typical Android smartphone hardware. Processing latency remained within real-time constraints, memory usage remained stable during foreground monitoring, and battery consumption was sufficiently modest for practical trip recording. These findings demonstrate that the project's offline-first design is technically viable on consumer mobile hardware without requiring specialised embedded systems or remote computation.

At the application level, the results confirmed that the major functional modules worked together coherently as a complete telematics platform. Ride summaries, trip history, analytics, coaching feedback, event-linked evidence capture, and map-based visualisation all operated as integrated outputs of the same sensing and analysis pipeline. User acceptance observations further indicated that the generated feedback was understandable and useful, while also revealing realistic areas for future improvement.

Overall, the major outcome of the project is that TeleDrive was not only conceptually designed, but practically demonstrated as a functioning two-wheeler telematics system that can analyse riding behaviour, generate interpretable rider feedback, and operate in real time using only a smartphone. Within the scope of this study, the results support the conclusion that the system is a validated baseline implementation for offline, on-device riding behaviour analysis.

10.16 Practical Relevance

The practical relevance of TeleDrive lies in its ability to convert widely available consumer smartphone hardware into a usable behavioural telematics tool for two-wheeler riders. This is important because the problem addressed by the project is not purely technical; it is also economic and behavioural. Many individual riders, especially students, commuters, and gig-economy delivery workers, do not have access to conventional telematics systems because such systems typically depend on dedicated hardware, subscription models, or fleet-level infrastructure. TeleDrive addresses this gap by providing a self-contained alternative based on a device that riders already possess.

From a user perspective, the system offers practical value in three forms. First, it provides behavioural visibility by identifying harsh acceleration, harsh braking, and unstable riding events that may otherwise go unnoticed during routine travel. Second, it converts those detections into understandable indicators such as eco score, fuel impact, monetary loss, and contextual coaching feedback, making riding quality easier to interpret and improve. Third, it preserves ride context through trip history, analytics, event mapping, and evidence-linked review, allowing users to examine behavioural patterns over time rather than relying only on momentary feedback.

The project is also practically significant because it demonstrates that privacy-conscious and offline-capable telematics is feasible for this use case. By performing sensing, classification, scoring, and storage locally on the device, TeleDrive reduces dependence on cloud connectivity while preserving low-latency operation. This is particularly relevant in environments where connectivity may be unreliable, where recurring subscription costs are undesirable, or where users are reluctant to share continuous behavioural data with external platforms.

At the same time, the system's significance should be interpreted within its present scope. TeleDrive is best understood as a focused rider-assistance and behavioural-awareness system rather than a replacement for full fleet telematics, vehicle diagnostics, or regulatory monitoring infrastructure. Its value lies in making two-wheeler behaviour analysis more accessible, interpretable, and technically achievable at the individual rider level.

In this sense, the practical importance of TeleDrive extends beyond the final software artefact itself. The project demonstrates that low-cost mobile devices can support road-behaviour analytics for riders who are typically excluded from traditional telematics ecosystems. That contribution is technically relevant and practically useful within the context for which the system was designed.

10.17 Final Concluding Statement

In conclusion, the TeleDrive project demonstrates that a practical two-wheeler telematics system can be designed, implemented, and validated using smartphone sensors and on-device computation. Within the scope of this work, the system integrates real-time sensor acquisition, hybrid riding-event detection, eco scoring, fuel and financial estimation, contextual evidence capture, local persistence, trip analytics, and map-based review within a single Android application intended for everyday rider use.

The project addresses a clearly defined transport and rider-safety problem by offering an affordable and privacy-conscious alternative to telematics approaches that depend on dedicated hardware or cloud-based infrastructure. The work is grounded in requirements analysis, structured system design, implementation, and real-world testing, and can therefore be defended as a focused engineering contribution within the scope of this study.

Broader validation across devices, vehicles, and riding environments remains future work. Within the tested scope, TeleDrive establishes a workable baseline for offline, on-device analysis of riding behaviour for two-wheelers and demonstrates that meaningful telematics feedback can be delivered through a consumer smartphone platform.

On this basis, the project may be concluded as a validated final-year implementation with practical relevance and clear scope for future enhancement.

CHAPTER 11 BIBLIOGRAPHY

11.1 BOOK REFERENCES

- Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig
- Introduction to Algorithms by Thomas H. Cormen et al.
- Database System Concepts by Abraham Silberschatz et al.
- Operating System Concepts by Abraham Silberschatz et al.
- Computer Organization and Design by David A. Patterson and John L. Hennessy
- Machine Learning: A Probabilistic Perspective by Kevin P. Murphy

11.2 WEB REFERENCES

- <https://developer.android.com/guide/components/foreground-services>
- <https://developers.google.com/android>
- <https://www.tensorflow.org/lite>
- <https://kotlinlang.org/docs/home.html>
- <https://github.com/>
- <https://ieeexplore.ieee.org/>
- <https://www.researchgate.net/>
- <https://stackoverflow.com/>
- <https://www.openstreetmap.org>
- <https://www.osmdroid.org/>

CHAPTER 12 PLAGARISM REPORT



Page 1 of 136 - Cover Page

Submission ID trn:oid::3618:137405402

Sharan S

Edge-Based Riding Behaviour Analysis Using Smartphone Inertial Sensors and On-Device Machine Learning

 Srinivas University, Mangaluru

Document Details

Submission ID

trn:oid::3618:137405402

Submission Date

May 2, 2026, 12:47 PM GMT+5:30

Download Date

May 2, 2026, 12:52 PM GMT+5:30

File Name

Teledrive_plagarism.docx

File Size

1.5 MB

132 Pages

24,470 Words

153,295 Characters



Page 1 of 136 - Cover Page

Submission ID trn:oid::3618:137405402

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **12 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 2%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
2183 suspect characters on 55 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

GLOSSARY

The following glossary defines the principal technical, domain, software engineering, and TeleDrive-specific terms used throughout this report. Each definition is written in the context of the implemented TeleDrive system to support clear interpretation of the design, implementation, testing, results, appendices, and user manual.

Definitions, Acronyms, and Abbreviations Used in TeleDrive

No.	Term	Definition
1	1D-CNN (One-Dimensional Convolutional Neural Network)	The sequence-based machine learning model used in TeleDrive to classify riding windows into NORMAL, HARSH_ACCELERATION, HARSH_BRAKING, or UNSTABLE_RIDE.
2	Activity	An Android application component that represents a user-facing screen, such as MainActivity, LiveTripActivity, or EventMapActivity in TeleDrive.
3	AI Assist Mode	A TeleDrive detection mode in which only the MLAnalyzer and the deployed 1D-CNN model are used for event classification.
4	Analytics Dashboard	The TeleDrive screen that presents riding trends, recent eco score behaviour, and summary intelligence from previous trips.
5	API Level	The Android platform version identifier used to define compatibility; TeleDrive targets Android API 26 and later.
6	APK (Android Package Kit)	The installable package format used to distribute the TeleDrive Android application.
7	Architecture	The high-level structural organisation of the system; in TeleDrive this refers to the layered pipeline

		separating sensing, processing, detection, scoring, storage, and UI.
8	CameraX	The Android Jetpack camera library used by TeleDrive to capture event evidence photographs from the rear camera.
9	Confusion Matrix	A table used to compare predicted classes with actual classes in model evaluation; TeleDrive uses it to analyse classification accuracy and misclassifications.
10	Detection Mode	The user-selectable operating mode that determines whether TeleDrive uses Rule-Based, AI Assist, or Hybrid event detection.
11	Edge AI	Artificial intelligence executed locally on the device rather than on a cloud server; TeleDrive performs ML inference on the smartphone itself.
12	Eco Score	TeleDrive's trip-performance score from 0 to 100 that summarises riding smoothness based on confirmed events and severity.
13	ER Diagram (Entity-Relationship Diagram)	A logical data-model diagram showing how TeleDrive's persistent entities such as UserProfile, TripSummary, and EvidenceItem relate to one another.
14	Event Confirmation	The state-machine process that requires consistent detections across windows before TeleDrive records an event, reducing false positives.
15	Event Map	The TeleDrive map interface that displays the trip route, event markers, matched evidence, and hotspot overlay.

16	Evidence Gallery	The TeleDrive screen that lists captured event photographs and allows review or deletion of stored evidence.
17	FeatureVector	The structured set of extracted numerical descriptors computed from a one-second sensor window for rule-based analysis and scoring.
18	Foreground Service	An Android service that stays active with a persistent notification; TeleDrive uses this to continue trip monitoring during active rides.
19	FusedLocationProviderClient	The Google Play Services location component used by TeleDrive to obtain GPS speed, position, and bearing.
20	Geocoder	The Android service used to convert latitude and longitude into human-readable place names for trip summaries.
21	GPS (Global Positioning System)	The satellite-based positioning source used in TeleDrive for location, speed, and route tracking.
22	Gson	The JSON serialization library used by TeleDrive to store and retrieve trip records and user profile data in local storage.
23	Harsh Acceleration	A riding event representing unusually strong positive forward acceleration, detected by TeleDrive as aggressive throttle behaviour.
24	Harsh Braking	A riding event representing unusually strong negative acceleration, detected by TeleDrive as sudden braking behaviour.
25	Hold-Out Set	The portion of the balanced ML dataset reserved for offline evaluation and not used during training.

26	Hotspot Detection	The TeleDrive process that identifies locations where risky events recur across multiple trips.
27	Hybrid Mode	The default TeleDrive detection mode in which the rule engine and ML model run together and resolve outputs through consensus rules.
28	Inference	The runtime process of using a trained model to produce a prediction from live sensor input on the Android device.
29	Insight Engine	The TeleDrive component that generates post-trip feedback such as comparisons, trends, streaks, and improvement suggestions.
30	Integration Testing	Testing that verifies whether two or more connected TeleDrive modules work correctly together as a pipeline.
31	Jetpack Compose	The declarative Android UI toolkit used to build TeleDrive's screens and user interface components.
32	JSON (JavaScript Object Notation)	The lightweight structured data format used in TeleDrive to store trip and profile records through Gson.
33	Local Storage	Storage kept directly on the smartphone rather than on a remote server; TeleDrive stores trip data and evidence locally.
34	Macro F1-Score	An evaluation metric that averages F1-scores across all classes equally; TeleDrive reports this to summarise ML performance across riding-event categories.
35	MLAnalyzer	The TeleDrive runtime classifier that builds the input tensor, runs TensorFlow Lite inference, and maps predictions to event labels.

36	Module	A distinct functional part of the system with a defined responsibility, such as scoring, evidence handling, analytics, or classification.
37	Offline-First	A design approach in which core application functions operate without continuous internet access; TeleDrive uses this for sensing, detection, scoring, and storage.
38	OpenStreetMap	The open map data source used by TeleDrive for route display and map tile rendering through osmdroid.
39	osmdroid	The Android mapping library used by TeleDrive to render the Event Map with OpenStreetMap tiles.
40	Permission	An Android runtime access right required for sensitive features; TeleDrive requests permissions such as location, camera, and notifications.
41	Precision	An ML evaluation metric that indicates how many predicted events of a given class were actually correct.
42	Recall	An ML evaluation metric that indicates how many actual events of a given class were successfully detected by the model.
43	Ride Summary	The TeleDrive results screen shown after a trip, containing eco score, event counts, fuel impact, loss estimate, and insights.
44	Risk Hotspot	A repeated hazardous location identified by TeleDrive when similar events occur in the same area across multiple trips.
45	Route Polyline	The line drawn on the Event Map to represent the recorded path of a trip from start to end.

46	SensorManager	The Android framework class used by TeleDrive to access accelerometer and gyroscope sensor events.
47	SharedPreferences	The Android local key-value storage mechanism used by TeleDrive to persist trip summaries and profile data.
48	Signal Conditioning	The preprocessing stage that cleans raw sensor data before classification, including spike rejection, median filtering, and moving average smoothing.
49	Softmax	The output function used by the TeleDrive CNN model to convert raw scores into class probabilities.
50	SRS (Software Requirements Specification)	The formal requirements document that defines TeleDrive's functional, non-functional, interface, and performance expectations.
51	Telematics	The use of sensing, computation, and communication technologies to monitor vehicle behaviour; in TeleDrive this refers to smartphone-based riding analysis.
52	TensorFlow Lite	The mobile machine learning framework used to deploy and run TeleDrive's trained 1D-CNN model on Android.
53	Trip Event	A single detected riding incident stored within a trip record, including type, time, severity, and GPS context.
54	Trip History	The TeleDrive record of previously completed rides, used for review, analytics, and comparison over time.
55	Trip Summary	The structured stored result of one completed ride, including metrics, event counts, timestamps, and related outputs.

56	Unit Testing	Testing that checks an individual component of TeleDrive in isolation to verify correctness for known inputs.
57	User Acceptance Testing (UAT)	Testing performed from the user perspective to confirm that TeleDrive is understandable, usable, and practically useful in real riding conditions.
58	Validation Testing	Testing used to confirm that TeleDrive's outputs and behaviour match real-world expectations and documented requirements.
59	WorkManager	The Android background task framework used by TeleDrive to schedule automatic cleanup of old evidence files.

End of Report